



FernUniversität in Hagen

– Fakultät für Mathematik und Informatik –
Lehrgebiet Datenbanken und Informationssysteme

Extending Static Code Analyzers with PostgreSQL-Specific Rules: An Analysis of Tools and Use Cases

Abschlussarbeit zur Erlangung des akademischen Grades
Bachelor of Science (B.Sc.)

vorgelegt von

Mag. Bernd Reiß, BA MA

Matrikelnummer: 3223442

Referentin : Prof. Dr. Uta Störl
Betreuer : Dr. Jan Nidzwetzki

ABSTRACT

PostgreSQL is one of the most flexible and widely adopted database management systems. Its extensibility and widespread use make it a critical component of modern infrastructure, whose reliable operation is of great importance to a broad range of users and systems. The core of PostgreSQL is implemented in the C programming language, which lacks built-in mechanisms for safe memory management. Consequently, memory-related errors—particularly those involving the PostgreSQL API—can have severe consequences.

This thesis explores how existing static analysis tools can be adapted to address PostgreSQL-specific challenges. In particular, it examines the Clang Static Analyzer framework and the semantic patch system Coccinelle. Both enable source code analysis without execution, allowing potential bugs to be detected early. As part of this work, the Clang-based `PostgresChecker` was developed to analyze memory management patterns in the PostgreSQL API and detect common issues such as use-after-free and double-free errors.

The methodology involved first identifying PostgreSQL-specific memory management patterns, followed by a systematic examination of the PostgreSQL codebase to find functions matching these patterns. The resulting function catalog was divided into five categories, which served as the basis for evaluating the suitability of Clang and Coccinelle. Building on this foundation, the `PostgresChecker` was implemented and assessed in terms of its detection capabilities, while alternative approaches using Coccinelle were demonstrated for certain pattern-based scenarios.

Applied to the PostgreSQL codebase and selected extensions, the checker successfully identified both real bugs and potentially problematic coding patterns, with an estimated false-positive rate of about 2%. Notably, one of the discovered issues resulted in an accepted patch in the PostgreSQL codebase, highlighting the practical value of the approach. Overall, the Clang Static Analyzer framework proved to be a powerful foundation for precise error detection in PostgreSQL, while Coccinelle showed particular strengths in pattern-based problems and offers a lightweight complement to more complex analysis techniques.

ZUSAMMENFASSUNG

PostgreSQL ist eines der flexibelsten und am weitesten verbreiteten Datenbankmanagementsysteme. Seine Erweiterbarkeit und große Verbreitung machen es zu einer wichtigen Infrastrukturkomponente, deren zuverlässige Funktion für eine Vielzahl von Anwender:innen und Systemen von zentraler Bedeutung ist. Der Kern von PostgreSQL ist in der Programmiersprache C implementiert, die keine inhärenten Mechanismen für sicheres Speicher-Management bereitstellt. Diese Eigenschaft führt dazu, dass Speicherfehler — insbesondere im Umgang mit der PostgreSQL-API — schwerwiegende Auswirkungen haben können.

Die vorliegende Arbeit untersucht, inwiefern sich bestehende statische Analyse-Tools an PostgreSQL-spezifische Problemstellungen anpassen lassen. Dabei werden insbesondere das Clang Static Analyzer Framework und das semantische Patchsystem Coccinelle analysiert. Beide Werkzeuge ermöglichen es, Quellcode ohne Ausführung zu untersuchen und dadurch potenzielle Fehler frühzeitig zu erkennen. Im Rahmen der Arbeit wurde der auf Clang basierende PostgresChecker entwickelt, der gezielt Speicher-Management-Muster der PostgreSQL-API überprüft und typische Fehler wie use-after-free oder double-free identifiziert.

Methodisch wurden zunächst PostgreSQL-spezifische Muster im Speicher-Management herausgearbeitet. Anschließend erfolgte eine systematische Untersuchung der PostgreSQL-Codebasis, um Funktionen zu identifizieren, die diesen Mustern entsprechen. Der resultierende Funktionenkatalog wurde in fünf Kategorien eingeteilt, welche als Grundlage für die Evaluierung von Clang und Coccinelle dienten. Darauf aufbauend wurde der PostgresChecker implementiert und hinsichtlich seiner Erkennungsleistung evaluiert. Ergänzend wurden für bestimmte Muster alternative Lösungsansätze mit Coccinelle vorgestellt.

Bei der Anwendung auf die PostgreSQL-Codebasis und ausgewählte Erweiterungen konnte der Checker sowohl tatsächliche Fehler als auch potenziell problematische Programmiermuster identifizieren, bei einer geschätzten False-Positive-Rate von rund 2%. Einer der entdeckten Fehler führte zu einem akzeptierten Patch in der PostgreSQL-Codebasis, was den praktischen Nutzen des Ansatzes unterstreicht. Das Clang Static Analyzer Framework erwies sich somit als leistungsfähige Grundlage für präzise Fehlererkennung im Kontext von PostgreSQL, während Coccinelle seine Stärken insbesondere bei musterbasierten Problemstellungen zeigte und eine leichtgewichtige Ergänzung zu komplexeren Analyseverfahren darstellt.

ACKNOWLEDGMENTS

First and foremost, I would like to thank Jan Nidzwetzki and Kevin Kramer for their invaluable input and outstanding supervision—I could not have hoped for better guidance. I also extend my thanks to Stefanie Scherzinger for her thought-provoking impulses during my presentation and David Rowley for reviewing the proposed patch. Most importantly, I am deeply grateful to Uta Störl for introducing me to the fascinating world of databases.

CONTENTS

1	Introduction	1
1.1	Motivation	1
1.2	Overview	2
1.2.1	Research Objectives	2
1.2.2	Main Contributions	3
1.2.3	Structure of the Thesis	3
2	Related Work	5
2.1	Memory Management: The Case of C	5
2.1.1	Basic Usage	5
2.1.2	A Look Into the Memory Allocator	6
2.1.3	Potential Problems in Memory Management	6
2.2	Solutions to Memory Management Issues	7
2.2.1	Approaches to Memory Safety Within Programming Languages	8
2.2.2	Static Analysis for C and PostgreSQL	8
2.2.3	Static Analysis for (Postgre)SQL	9
3	Static Analysis	10
3.1	A Look Inside the Compiler	10
3.1.1	Overview	10
3.1.2	Definitions	12
3.2	Static Analysis	13
3.2.1	What It Does and How It Works	14
3.2.2	Implementations of Static Analyzers	17
3.3	Clang	19
3.3.1	Clang as a Compiler	19
3.3.2	Clang Static Analyzer	20
3.3.3	Limitations	21
3.4	Coccinelle	22
3.4.1	Semantic Patches	22
3.4.2	A Closer Look	24
3.4.3	Limitations	25
3.5	Summary	26
4	PostgreSQL and Domain-Specific Patterns	27
4.1	PostgreSQL vs. Standard C	27
4.2	Data Basis	30
4.2.1	Approach	30
4.2.2	Implementation	31
4.3	Categories	33
4.3.1	C1: Strict Functions	35
4.3.2	C2: Dependent Functions	37
4.3.3	C3: Arbitrary Functions	38
4.3.4	C4: Reassigning Functions	39

4.3.5	C5: Functions Returning Boolean	40
4.4	Summary	40
5	Implementing a Static Analyzer	41
5.1	Detecting Use-after-frees: General Considerations	41
5.1.1	Coccinelle: Implementation by Stuart	41
5.1.2	Clang: The MallocChecker	42
5.2	Categories	43
5.2.1	C1: Strict Functions	44
5.2.2	C2: Dependent Functions	46
5.2.3	C3: Arbitrary Functions	48
5.2.4	C4: Reassigning Functions	49
5.2.5	C5: Functions Returning Boolean	49
5.3	Summary	50
6	Discussion and Evaluation	51
6.1	Comparing Coccinelle and Clang	51
6.2	The PostgresChecker	52
6.2.1	True Positives	53
6.2.2	False Positives	55
6.2.3	Suboptimal Coding Patterns	56
6.3	Summary	57
7	Conclusion	58
7.1	Extending the Checker: PostgreSQL-specific Issues	58
7.2	Extending the Checker: Issues Independent of PostgreSQL	60
 Appendix		
A	Scripts and Templates	62
A.1	Recursive Bash Script	62
A.2	Strict Functions Coccinelle Template	66
A.3	Dependent Functions Coccinelle Template	70
A.4	Functions With Two Parameters of Same Type Coccinelle Template	74
A.5	Collect Results Bash Script	75
A.6	Template to Retrieve the Position of Arguments	78
A.7	Script to Retrieve Argument Position	79
A.8	Coccinelle Script to Address Category 5	80
A.9	Coccinelle Script to Report Issues from Category 5	80
A.10	Script to Analyze Directories	81
B	The PostgresChecker	86
B.1	Implementation	86
B.2	Tests	103
 Bibliography		 112

LIST OF FIGURES

Figure 1.1	Example of a bitmap for the value range 0–7	2
Figure 1.2	Developing the PostgresChecker	4
Figure 2.1	A block of memory newly allocated by malloc	6
Figure 2.2	A block of memory allocated by malloc and assigned a string	6
Figure 3.1	A sample syntax tree	11
Figure 3.2	Two-stage mapping from names to values	13
Figure 3.3a	Example of a control-flow graph	17
Figure 3.3b	Example of a path-sensitive control-flow graph	17
Figure 3.4	Program with three states and specified transitions . .	24
Figure 3.5	Infinite tree of all computation paths beginning in a particular state	25
Figure 4.1	Phase 1 of developing the PostgresChecker	27
Figure 4.2	The directory layout of the scripting environment . . .	34
Figure 5.1	Phase 2 of developing the PostgresChecker	41
Figure 5.2	Resulting output of the PostgresChecker	46
Figure 6.1	Phase 3 of developing the PostgresChecker	51

LIST OF TABLES

Table 2.1	Overview of existing static analyzers for PostgreSQL . . .	9
Table 3.1	Comparison of soundness vs. completeness	15
Table 3.2	Relationship between positives and negatives as seen by the analyzer	15
Table 3.3	Comparison of different static analysis tools with re- gard to maintenance, integration, and customizability .	18
Table 3.4	Comparison of Coccinelle, the Clang Static Analyzer, and Clang-Tidy	26
Table 4.1	Number of functions by category	40
Table 6.1	Comparison of Coccinelle and Clang for static analysis	52
Table 6.2	Results of the experimental evaluation	53

LISTINGS

2.1	Minimal program in C including memory management	5
2.2	Case of use-after-free	7
2.3	Minimal program in Rust	8
3.1	Example of IR in the form of three-address code	12
3.2	Example of a compound statement	12
3.3	Sample C code for the control-flow graph	16
3.4	Hello World program in C	19
3.5	A C program including a separate file	21
3.6	C code with use-after-free	23
3.7	Example of a patch file	23
3.8	Snippet of function with suboptimal code	24
3.9	Coccinelle patch for the code snippet	24
4.1	Memory management in traditional C programming patterns .	28
4.2	Implementation of the strcat function in glibc	28
4.3	Function reallocating memory internally in PostgreSQL	29
4.4	Problematic use of a PostgreSQL-style function	29
4.5	Memory management and functions in PostgreSQL	29
4.6	Template script for finding all occurrences of a function	31
4.7	Matched functions	32
4.8	Unmatched functions	32
4.9	Python output of the Coccinelle scripts	32
4.10	Basic template for strict functions	36
4.11	Template for strict functions with NULL check	36
4.12	Template for strict functions with valid pointer check	36
4.13	Matched functions	36
4.14	Unmatched functions	36
4.15	Function where freeing depends on another parameter	37
4.16	Template for functions depending on a passed parameter . . .	37
4.17	Template for reversed parameter order	38
4.18	Template for dependencies on parameter fields	38
4.19	Function where freeing of a parameter is arbitrary	38
4.20	Function where return value should be reassigned	39
5.1	Coccinelle template to find use-after-free locations	42
5.2	Example of an annotation in Clang	43
5.3	Map storing strict function names and the position of the parameter	44
5.4	Signature of the PostgresChecker class	46
5.5	The structure to store dependency-related information	47
5.6	A PostgreSQL function being arbitrary when the dependent value is false	47
5.7	Example of an entry in the DependentMap	48

5.8	Coccinelle script to address Category 4-related issues	49
5.9	Coccinelle script to address incorrect usage of the function ecpg_check_PQresult	50
6.1	Confirmed case of use-after-free detected by the PostgresChecker	54
6.2	Confirmed case of use-after-free detected by the PostgresChecker	54
6.3	Possible use-after-free case in the Age extension	55
6.4	Suboptimal use of if statements detected due to false positive .	57
6.5	Suboptimal use of local variable detected due to false positive .	57
A.1	Bash script for recursing over the code base	62
A.2	Coccinelle template for Strict Functions	66
A.3	Coccinelle template for Dependent Functions	70
A.4	Coccinelle template for functions with two parameters of the same type	74
A.5	Bash script for collecting results	75
A.6	Coccinelle template retrieving the position of arguments	78
A.7	Bash script to retrieve the position of arguments in signatures .	79
A.8	Full Coccinelle script to address Category 5	80
A.9	Coccinelle script to report Category 5 issues	80
A.10	Bash script to run Clang against a directory	81
B.1	Implementation in the Clang Static Analyzer framework	86
B.2	Tests used to validate the checker	103

INTRODUCTION

“路漫漫其修远兮，吾将上下而求索”

—屈原

“The road ahead is long and arduous, but I will search high and low.”

—Qu Yuan

1.1 MOTIVATION

There is no question that PostgreSQL is one of the most relevant database systems in existence. Its long history, widespread deployment, and active community make it one of the most trusted and reliable systems in its domain. However, PostgreSQL takes a very broad approach to data management, serving as a general-purpose database. In some cases, more specialized solutions are required. This is where extensions can provide valuable additional features. These range from handling spatial data efficiently to introducing cryptographic functions [Tim25g]. The PostgreSQL ecosystem, in turn, benefits from such extensions, as they help make the system relevant to a wider audience.

When creating extensions, developers work directly with the PostgreSQL codebase, which is written in C—a relatively low-level programming language that often facilitates the introduction of subtle and hard-to-detect bugs. To help developers avoid such pitfalls, tools like static code analyzers are commonly used.

Static analysis is concerned with characteristics of code that can be deduced without actually executing the program. It is primarily used during compilation to verify correctness and optimize programs. Modern IDEs even incorporate static analysis to provide real-time feedback as developers write code. Common examples of such checks include type mismatches and the use of undeclared variables. These errors are detected at compile-time without requiring the program to run. However, static analysis can go beyond detecting such obvious issues and instead identify more subtle problems.

Some programming mistakes are syntactically correct and compile successfully but lead to runtime errors—or worse, undefined behavior. A prominent example is the dereferencing of dangling pointers. In C, such pointers might have already been freed or reassigned. In the best case, the issue occurs consistently and can be easily traced back to a specific function or code block. But not every incorrect use of a pointer is so clear-cut. What if the pointer is freed within a called function? What if it is only freed under certain conditions,

such as when $x > 42$? The problem may not surface every time the program runs—it might only manifest once in a thousand executions. Such bugs are notoriously difficult to detect, and the root cause may be far from obvious.

When developing a PostgreSQL extension, encountering this kind of problem is not unlikely. For example, a `Bitmapset` [Pos250] is a data structure used to store a set of nonnegative integers as a bitmap. Figure 1.1 illustrates the concept: members of the set are represented by 1, meaning the set in the example consists of the values $\{1, 2, 4, 7\}$. It provides various functions, such as `bms_add_member`, which takes a pointer `Bitmapset *a` and a number x as input and adds the bit corresponding to x to the bitmap. This seems straightforward—until x does not fit into a . In that case, a new `Bitmapset` is allocated internally, and the pointer to the original memory region becomes dangling. In the example in Figure 1.1, inserting a value ≥ 8 would require such an action. Therefore, the validity of the passed pointer depends on a seemingly arbitrary number x —exactly the kind of subtle, hard-to-detect issue described earlier. The documentation of `Bitmapset` even warns developers about this behavior and advises always using the result returned by the function rather than assuming the original pointer remains valid [Pos25a]. But even when the warning is read, mistakes can still happen. This is where static analysis can help by offering a systematic approach to ensure code quality and safety.

0	1	1	0	1	0	0	1
0	1	2	3	4	5	6	7

Figure 1.1: Example of a bitmap for the value range 0–7

This thesis explores the capabilities of static analysis tools in addressing issues specific to the PostgreSQL codebase. Two tools in particular are examined: Clang and Coccinelle.

1.2 OVERVIEW

This section presents the research questions and main contributions, followed by an outline of the thesis structure.

1.2.1 Research Objectives

The following research questions, derived from the introduction, form the foundation of this thesis:

1. What PostgreSQL-specific memory management pitfalls exist for developers of PostgreSQL extensions that can be addressed through static program analysis?
2. Which approaches are most suitable for tackling these problems, and what are the benefits and limitations of the various methods?

3. How can existing tools be extended to address PostgreSQL-specific issues?

The mentioned pitfalls refer exclusively to bugs arising in the context of the C programming language. Extensions written in Rust, as well as issues related to SQL, are beyond the scope of this thesis.

1.2.2 Main Contributions

The main contributions of this thesis are:

- **Function Catalog:** an approach for systematically deriving a function catalog for static checks from an existing codebase.
- **Coccinelle and Clang for PostgreSQL:** an assessment of the capabilities of Coccinelle and Clang in detecting PostgreSQL-specific memory management issues.
- **Checker:** the implementation of a static checker capable of reporting basic *use-after-free* and *double-free* issues¹ when using the PostgreSQL API.
- **Evaluation:** an experimental evaluation of the checker on the PostgreSQL codebase and selected extensions.

The remainder of this section outlines the structure of the thesis.

1.2.3 Structure of the Thesis

To develop a PostgreSQL-specific checker capable of detecting memory-management-related issues, this thesis follows a structured approach, with each step building on the previous one. Figure 1.2 illustrates the development process of the static checker, referred to in this thesis as the *PostgresChecker*. In the figure, blue boxes represent resources, red boxes represent processes, and green boxes denote outputs. The labels on the edges indicate artefacts transferred between steps.

The core idea is to analyze the PostgreSQL codebase for specific function patterns related to memory management. The identified functions are categorized into groups that require different handling by the static checker. Based on this foundation, a checker implementing these checks is developed and applied to the PostgreSQL codebase and selected extensions to gather feedback in a real-world scenario. Any false positives encountered are then used to identify potential improvements to the checker.

GROUNDWORK Chapters 2 and 3 lay the groundwork for the development of the static checker. Chapter 2 discusses the problem of memory management in C and presents established approaches to static analysis in general, as well

¹ These issues are elaborated on in Chapter 2.

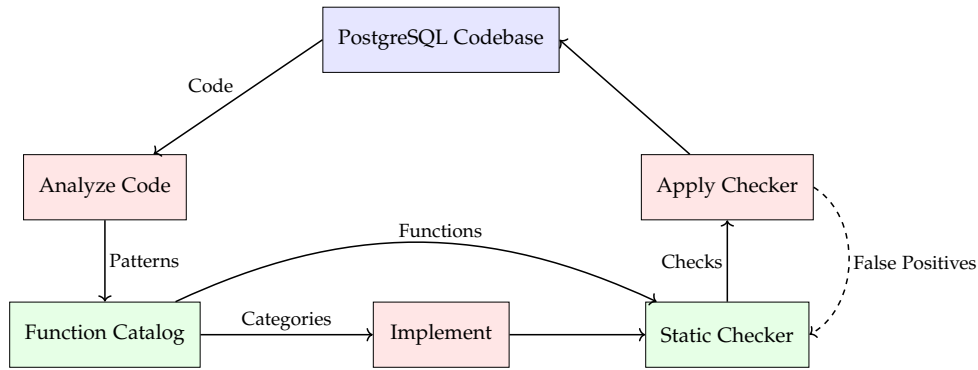


Figure 1.2: Developing the PostgresChecker

as those related to PostgreSQL. Chapter 3 lays the theoretical foundation for the thesis by presenting key concepts in compiler technologies and static analysis. It introduces various existing implementations and motivates the choice to focus on Coccinelle and Clang as the most promising candidates for implementing the checker. Their capabilities and limitations are discussed in detail.

FUNCTION CATALOG In Chapter 4, the process illustrated in Figure 1.2 begins. The PostgreSQL codebase is explored for issues that can be addressed by static analysis. Relevant patterns are identified, and the approach used to systematically extract functions containing these patterns is described, resulting in a function catalog. From these functions, categories are deduced, leading to five distinct groups. The identified categories and their implications for the requirements of a static checker are discussed.

IMPLEMENTATION Chapter 5 analyzes Coccinelle and Clang with respect to the previously defined categories. Their strengths and limitations are assessed in light of the challenges posed by these categories. The chapter also presents the actual implementation of the static checker.

REFLECTION Chapter 6 summarizes the findings from the previous chapter and discusses the validity of the developed checker. It presents real-world problems discovered by the PostgresChecker while applying the implemented checks to the PostgreSQL codebase and selected extensions, identifying three cases of use-after-free. Particular attention is given to false positives, from which suggestions for improving the checker are derived. The chapter also examines problematic coding patterns identified in the PostgreSQL codebase. Finally, Chapter 7 summarizes the overall findings of this thesis, proposes ideas for extending the developed checker, and highlights promising directions for future research.

RELATED WORK

This chapter first discusses memory management issues in programming languages, using C as a representative example, before presenting established solutions, with a particular focus on static analysis.

2.1 MEMORY MANAGEMENT: THE CASE OF C

C is a programming language well known for its challenges regarding memory management. Unlike many modern languages, it does not offer garbage collection, nor does it provide constructs similar to smart pointers in C++. Instead, dynamically allocated memory must be managed manually by the programmer. Improper memory management can lead to serious issues, including memory leaks, security vulnerabilities, data corruption, and undefined behavior. Therefore, programming in C demands great care and diligence. This section explores memory management functions in C and their implications for memory safety.

2.1.1 *Basic Usage*

The most important functions for managing memory are `malloc`, used to allocate memory; `realloc`, used to resize a memory region; and `free`, used to release memory. A minimal example of memory management in C is shown in Listing 2.1.

Listing 2.1: Minimal program in C including memory management

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main(){
6     char *str = malloc(sizeof(char) * 13);
7
8     strcpy(str, "Hello World\n");
9     printf("%s", str);
10
11     free(str);
12
13     return 0;
14 }
```

In this example, memory is allocated in Line 6. The programmer must explicitly specify how much memory to allocate. This is done by defining the size of the type—determined via the `sizeof` operator—multiplied by the number of elements needed. In this case, memory is allocated to store the string `Hello World\n`, consisting of ten letters, a space, and a newline

character. The thirteenth allocated byte is reserved for the special string terminator `\0` in C, which marks the end of the string.¹

In Line 8, the string is copied to `str` using the dedicated `strcpy` function. The function takes a pointer to memory (`str`) and a char array, copying the array—including the `\0` string terminator—into the memory region pointed to by `str`.² Line 9 prints the string to standard output, and Line 11 releases the allocated memory.

2.1.2 A Look Into the Memory Allocator

`malloc` returns the address of a memory block reserved for the process, which can be accessed using a pointer—in this example, `str`. As shown in Figure 2.1, the fields in this chunk normally contain random values, since `malloc` does not initialize memory—the contents of the returned memory are undefined.

A	9	\$	s	?	1	#	4	2	!	a	E	2
00	01	02	03	04	05	06	07	08	09	10	11	12

Figure 2.1: A block of memory newly allocated by `malloc`

The use of `strcpy` results in the memory region shown in Figure 2.2. The `printf` function then accesses this memory, and `free` releases the memory region pointed to by `str`. This means the memory is “given back” to the allocator and can be reused by other processes. Therefore, the contents of this memory after release are undefined: it may still hold the previously assigned values, it may have been overwritten by the allocator, it may have been reused by other allocations, or it may already be unmapped and therefore no longer belong to the process.

H	e	l	l	o		W	o	r	l	d	\n	\0
00	01	02	03	04	05	06	07	08	09	10	11	12

Figure 2.2: A block of memory allocated by `malloc` and assigned a string

2.1.3 Potential Problems in Memory Management

If we attempt to access the memory region after freeing it, a range of potential issues may arise [OP23]:

- **Garbage values:** The software might access values that are no longer valid, leading to unexpected results.

¹ Strings in C are essentially char arrays, which are, in turn, pointers to memory locations. The convention of zero termination allows functions to reliably determine where the string ends.

² All string literals in C are, by definition, zero-terminated, as the value zero is appended at compile time [ISO07, p. 62]. However, when copying from another pointer, the `\0` terminator must already be present; otherwise, the behavior is undefined.

- **Unreliable software:** The expected values might *sometimes* still exist, resulting in unpredictable behavior that depends on factors such as the operating system and the allocator.
- **Corrupted data:** If the program writes to freed memory, a memory region used by another part of the process could be overwritten, potentially affecting other data or functions.
- **Security issues:** If sensitive data—such as access rights—is stored in the reused memory region, accessing it may introduce serious vulnerabilities. Additionally, writing to memory regions belonging to other parts of the process may influence control structures and, therefore, alter program flow.
- **Program crashes:** Accessing already unmapped memory may cause a *segmentation fault*, crashing the program.

A typical case of a *use-after-free* bug is shown in Listing 2.2. The program works exactly like the one in Listing 2.1, with the important difference that the `str` pointer is accessed in Line 11 *after* being freed in Line 9. In this example, there is no way to predict whether `printf` will print “Hello World”, random characters, or cause a crash—the behavior is undefined. Another common issue is calling `free` twice on the same memory region, also known as a *double-free*. Since the memory region may already have been reallocated—e.g., by another part of the program—this can lead to memory corruption, unexpected use-after-free scenarios elsewhere in the program, and even security vulnerabilities. As a result, the program may encounter segmentation faults, crashes, or other forms of undefined behavior.

Listing 2.2: Case of use-after-free

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6      char *str = malloc(sizeof(char) * 13);
7
8      strcpy(str, "Hello World\n");
9      free(str);
10
11     printf("%s", str);
12
13     return 0;
14 }
```

2.2 SOLUTIONS TO MEMORY MANAGEMENT ISSUES

Although memory management is a challenge that every program must address, different programming languages offer different solutions. A large number of legacy systems are written in C, which lacks modern safety mechanisms. For this reason, static analysis has become a major focus area

for helping C developers write safer, more reliable software. Static analyzers designed for PostgreSQL development also build on these tools.

This section first explores approaches adopted by other programming languages and then discusses solutions for C and PostgreSQL in more detail.

2.2.1 *Approaches to Memory Safety Within Programming Languages*

GARBAGE COLLECTION Most modern languages address memory management issues by implementing a garbage collector. In such languages, developers do not explicitly manage memory allocation and deallocation; instead, the runtime automatically handles it. In Java, for example, the Java Virtual Machine (JVM) tracks heap objects and automatically reclaims the memory of unused objects. However, this convenience comes at a cost: the runtime must spend resources on garbage collection, which introduces performance overhead. While this trade-off is acceptable for many enterprise and end-user applications, it becomes problematic for infrastructure software such as operating systems or databases, where performance is critical.

RUST AND OWNERSHIP Rust [Rus25a] takes a fundamentally different approach by introducing the concept of *ownership* [Rus25b]. Every value in Rust has exactly one *owner*, and when that owner goes out of scope, the value is automatically deallocated. The program shown in Listing 2.1 would look like Listing 2.3 in Rust. In this version, there is no explicit memory management by the developer: in Line 2, a variable is declared and automatically freed when it goes out of scope. Instead of relying on runtime checks, Rust enforces memory safety at compile time, preventing common issues such as use-after-free and double-free errors. The main drawbacks of this approach are the steep learning curve and longer compilation times.

Listing 2.3: Minimal program in Rust

```

1 fn main() {
2     let str = String::from("Hello World\n");
3     print!("{}", str);
4 }
```

2.2.2 *Static Analysis for C and PostgreSQL*

As this thesis uses Clang and Coccinelle for static analysis, this section focuses on related work involving these tools.

CLANG Beyond the basic checkers provided by Clang as a C/C++ compiler, there is also research on extending Clang-based tools to address domain-specific problems. A custom checker for the MPI API was developed by Droste, Kuhn, and Ludwig, targeting domain-specific issues through path-sensitive analysis [DKL15]. These include problems related to non-blocking operations—such as not waiting for them to finish—data type safety, and deadlock detection. Bari et al. introduced a checker for memory-related

bugs [Bar+21] within the OpenSHMEM framework—a standardized parallel programming API specification [Sil25]. The checks include type checks, synchronized access checks, and double-free detection. Arroyo, Chiotta, and Bavera describe a general approach to static taint checking for API analysis using Clang [ACB16].

COCCINELLE Coccinelle, although primarily used for refactoring large code bases, has also seen attempts at using it as a framework for static analysis. Stuart et al. describe its use for bug detection in Linux code [Stu+07], while Stuart specifically adapts Coccinelle to identify use-after-free issues [Stu08]. Olesen et al. combine Clang and Coccinelle to support compliance with the CERT C Secure Coding Standard. Their approach leverages Clang’s analysis features and the usability of Coccinelle’s SmPL language [Ole+10]. Dura and Reichenbach present Clog, a declarative language inspired by SmPL but backed by Clang, combining Clang’s analytical power with a higher-level syntax [DR24].

POSTGRESQL The `pg_ladybug` project [Tim25d], a Clang-Tidy plugin developed by Timescale [Tim25f], implements a static analysis tool for PostgreSQL API development. The same company also uses Coccinelle in their PostgreSQL-based TimescaleDB [Tim25b] to identify problematic C code patterns. Another project called `sqlsmith` [STM25b], although not a static checker in the traditional sense, takes an interesting approach to detecting bugs in the PostgreSQL code base: it randomly generates SQL queries and, by executing them, uncovers bugs and crashes. Although not a strictly structured approach, `sqlsmith` has been able to identify numerous bugs in PostgreSQL (see [STM25a]).

2.2.3 Static Analysis for (Postgre)SQL

Although PostgreSQL is implemented in C, most developers interact with it through SQL. Various static analysis tools focus on SQL usage in PostgreSQL. Table 2.1 presents prominent examples. However, none of these tools analyze extensions written in C.

Tool	Description
<code>dblint</code> [Dec25]	Checks database schemas rather than code.
<code>Holistic</code> [Hol25]	Static SQL code analysis, not C code.
<code>plpgsql_check</code> [Ste25]	Linters for PL/pgSQL, not C.
<code>pg-index-health</code> [Van25]	Analyzes database structures, not source code.

Table 2.1: Overview of existing static analyzers for PostgreSQL

Regarding Coccinelle, the DeBinelle project illustrates how its concepts can also be applied to databases. It introduces the notion of *collateral evolutions* in database schemas, using semantic patches in SQL [SMK21].

“A static program analyzer is a program that reasons about the behavior of other programs. For anyone interested in programming, what can be more fun than writing programs that analyze programs?”

—Møller and Schwartzbach [MS25, p. v]

This chapter provides the theoretical background necessary to understand the goals and functionality of static analysis. It defines key terminology and introduces the tools examined in this thesis. To that end, the first section briefly discusses the most important aspects of compilers. The second section introduces the concept of static analysis and reviews widely used implementations. The third section presents Clang as part of the LLVM framework, while the fourth section introduces Coccinelle.

3.1 A LOOK INSIDE THE COMPILER

Compilers are the backbone of modern software. They must reliably transform source code into machine instructions: errors in this translation lead to incorrect program behaviour. At the same time, compilers aim to optimize program performance. A correct but inefficient translation results in slow program execution. For these reasons, compilers are immensely complex pieces of software. The science behind them fills entire books, with the famous Dragon Book¹ [Aho+07] spanning well over a thousand pages.

This section provides a high-level overview of the compilation process, focusing on the stages most relevant to static analysis. Many sophisticated concepts involved in constructing these steps are, however, beyond the scope of this thesis.

3.1.1 Overview

Before actually compiling a program, a preprocessor prepares the input for the compiler. In C, for example, this includes resolving macros. The compiler then typically proceeds through the following phases [Aho+07, pp. 5–11]:

- **Lexical analysis:** The input is transformed into distinguishable units called tokens (e.g., keywords, identifiers, or operators).
- **Syntactic analysis:** Every programming language follows a context-free grammar that defines correct syntax. The goal of this step is to check

¹ The name is derived from the book cover displaying a dragon.

for syntax errors, resulting in the construction of an (abstract) syntax tree (AST).

- **Semantic analysis:** The syntax tree is checked for semantic inconsistencies, and necessary transformations are performed. This may include type checking or converting integers to floating-point numbers.
- **Intermediate code generation:** An intermediate representation (IR) of the code is generated—typically in the form of *three-address code* (see Listing 3.1). This IR is optimized independently of the target machine.
- **Code generation:** The IR is translated into the target language, such as machine code.²

To illustrate these phases, consider the following C statement:

```
float circumference = 2 * (length + width);
```

This would be tokenized into

```
float id(1) = const * ( id(2) + id(3) ) ;
```

Tokens represent elementary units of significance. The validity of combining these elements is determined by the grammar of the programming language. For this purpose, an (abstract) syntax tree is constructed, during which violations of grammar rules are checked. A syntax tree for the example could look like Figure 3.1.

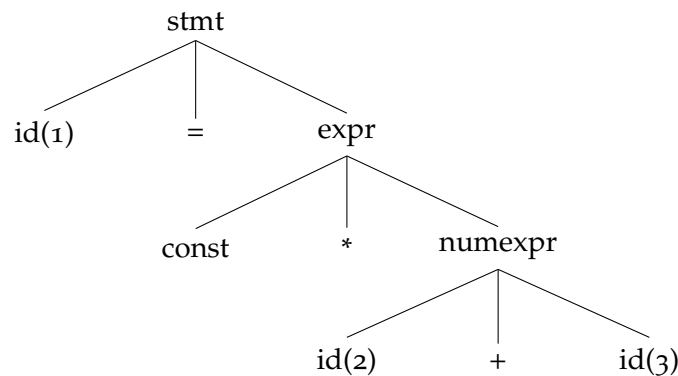


Figure 3.1: A sample syntax tree

The compiler then performs semantic analysis on the tree: among other things, it checks for type mismatches and converts the integer constant to a floating-point format. The machine-independent IR in the form of three-address code would look like Listing 3.1. From this IR, the actual machine-dependent code can be generated.

² The target could also be more abstract, for example, resulting in assembly or bytecode for a virtual machine.

Listing 3.1: Example of IR in the form of three-address code

```

1  t1 = id2 + id3
2  t2 = t1 * 2
3  id1 = t2

```

All phases up to and including the generation of an IR are typically referred to as the *frontend* of the compiler, while the *backend* encompasses machine-specific optimizations and code generation [Aho+07, p. 357].

3.1.2 Definitions

It is worth noting that in the process of creating the syntax tree above, some terms were used that have not yet been properly defined. This section provides these definitions, along with other important concepts relevant to static analysis.

3.1.2.1 Statements

A statement is defined as a syntactic unit that is explicitly terminated and either represents a declaration or an instruction, including any necessary operands and the usage of results [ISO15]. A statement therefore represents a basic functional unit in programming. In C, statements are typically terminated by the delimiter `;`, as in `x += 42;`. However, a statement can also be much more complex and consist of other statements and elements. An example of a so-called *compound statement*—also referred to as a block—is shown in Listing 3.2. In the example, the statement representing the function body includes a *declaration statement*, an *if statement*, and a *return statement*. The if statement itself consists of a *binary operator* and another compound statement: `x += 42;`. Therefore, statements are highly expressive and can take on various forms.

Listing 3.2: Example of a compound statement

```

1  int add42(int y){
2      int x = y;
3      if (x < 42){
4          x += 42;
5      }
6      return x;
7  }

```

3.1.2.2 Expressions

An expression is defined as a

language construct that defines the computation of a data value as a result from one or more operands[.] [ISO15]

Expressions are any constructs that represent a value. For example, `x += 42;` is an expression, whereas `return x + 42;` is a statement that contains an expression.

3.1.2.3 Declarations

A declaration is defined as an

explicit language construct that introduces one or more identifiers into a program and specifies how these identifiers are to be interpreted[.] [ISO15]

A typical example in C would be a forward declaration, such as `int square(int number);`.

3.1.2.4 Variables

Variables represent locations in memory and are denoted by names called *identifiers*³ [Aho+07, p. 28]. As shown in Figure 3.2, these names are mapped to memory locations via an *environment* [Aho+07, p. 26]. The environment is determined by the scoping rules of the programming language. Locations are then mapped to values via the program's *state*. The memory location and the values they hold are often referred to as *l-values* and *r-values*, respectively [Aho+07, p. 26].

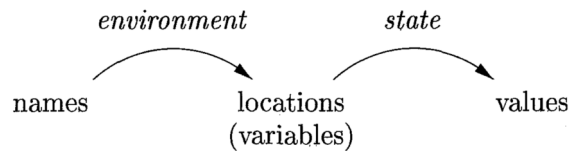


Figure 3.2: Two-stage mapping from names to values

Source: [Aho+07, p. 26]

3.1.2.5 Symbol Tables

Symbol tables keep track of identifiers. They contain important information about source program constructs, such as the lexeme (the original string), the type, and the storage location [Aho+07, p. 86]. A separate symbol table is created for each scope. These tables are managed and used during lexical analysis, parsing, and semantic analysis [Aho+07, p. 86].

3.2 STATIC ANALYSIS

The first part of this section discusses static analysis as part of the compilation process as well as a standalone tool and introduces important concepts. The second part presents widely used implementations for the C programming language.

³ Identifiers can also refer to expressions.

3.2.1 What It Does and How It Works

In static program analysis, the program is analyzed without actually executing it. Some static checking already occurs during the translation of source code to IR, including syntactic and type checking [Aho+07, 97f.]. However, most static checks performed at compile time are related to optimization. These include eliminating dead or redundant code, reasoning about register allocation, or constant folding⁴ [AG04, p. 383].

Static analysis, however, can go beyond what a compiler typically does. Møller and Schwartzbach [MS25, pp. 2–3] identify two additional areas where developers can benefit from static analysis:

- **Program correctness checks:**
 - Null pointer dereferences, division by zero, arithmetic overflow.
 - Proper initialization of variables.
 - Out-of-bounds array accesses.
 - Dangling pointers.
 - etc.
- **Analysis for program development (debugging, refactoring, etc.):**
 - Which functions may have been called at a certain point in the program?
 - What types of values can a variable hold (e.g., in Python)?
 - etc.

Furthermore, more specific use cases are possible, such as analyses tailored to particular libraries (e.g., checking whether a function throws a specific exception) or checks for parallel programs (see also Section 2.2.2).

3.2.1.1 Soundness and Completeness

Checks for program correctness, in particular, can be an invaluable resource for bug prevention. However, the devil lies in the details. From *Rice's theorem* [Ric53] Møller and Schwartzbach [MS25, pp. 5–6] deduce that correctness cannot be fully asserted for any nontrivial program, even if it halts on every input. It is only possible to *approximate* correctness. For example, if we want to determine whether a variable always has the constant value 5 and can therefore be replaced by that constant, we can identify cases that are definitely constant. In other cases, however, we might only be able to say *maybe constant*, and the compiler must be very careful not to include such uncertain cases, as this could alter the program's semantics—an unacceptable outcome during compilation, as it may change the program's behavior. Static analysis is therefore always a trade-off between *soundness* and *completeness*: it must

⁴ Constant folding refers to the process of evaluating expressions at compile time that represent a constant (e.g., `x = 42 * 2`).

	Soundness	Completeness
Avoids	False negatives	False positives
Result	All bugs are reported	Only bugs are reported

Table 3.1: Comparison of soundness vs. completeness

	Positive	Negative
True	Reject incorrect program	Accept correct program
False	Reject correct program	Accept incorrect program

Table 3.2: Relationship between positives and negatives as seen by the analyzer

decide whether to report only valid results or whether to report all potentially relevant results. The precise definitions of these terms depend on the context. Still, soundness is typically defined as avoiding false negatives, while completeness means avoiding false positives⁵ [MS25]. Table 3.1 illustrates this distinction.

If a static analyzer, for example, checks for program safety (e.g., out-of-bounds array accesses), the analysis is *sound* if it never reports an unsafe program as safe. This may, however, lead to false positives: safe programs might be reported as potentially unsafe to ensure that all real bugs are detected. Soundness therefore means that no false negatives are reported—the problem is over-approximated, and all issues are detected. It would, on the other hand, be *complete* if all reported issues are real—false positives are avoided. This implies that some bugs may go undetected, resulting in false negatives.

Another way to think about this is that the goal of static analysis is to reject incorrect programs. Table 3.2 shows the relationship between positives and negatives. A true positive is the analyzer rejecting an incorrect program; a true negative is accepting a correct one. An analyzer is sound if it reports all bugs and avoids false negatives—i.e., it does not accept incorrect programs. It is complete if it only reports bugs and avoids false positives—i.e., it does not reject correct programs.⁶

⁵ Note: This definition might appear counterintuitive, as it contrasts with the notion of soundness in proof systems in formal logic. There, a deductive system is called *sound* if $\Sigma \vdash \phi \implies \Sigma \models \phi$, i.e., only semantically true propositions are syntactically provable. A system is *complete* if $\Sigma \models \phi \implies \Sigma \vdash \phi$, i.e., all semantically true propositions are provable [BPT17, 152f.]. A static analyzer, however, typically tries to reject incorrect programs, shifting the perspective.

⁶ The precise meaning of soundness and completeness can be subtle. Meyer [Mey19] shows that the definitions may vary depending on context. For instance, an analyzer that detects unreachable code is considered sound in the context of a compiler removing dead code if it only reports code that is truly unreachable. Reporting reachable code as unreachable would be *unsound*. This matters because the compiler must only accept correct programs; removing valid code would lead to accepting an incorrect one (a false negative). In contrast, a static checker in an IDE that reports unreachable code to a developer aims to detect incorrect code. If it reports reachable code as unreachable, it rejects a correct program, leading to a false positive and thus incompleteness.

Ideally, static analyzers aim to be conservative: they err on the side of caution, either by detecting all potential issues (favoring soundness) or by reporting only definite errors (favoring completeness), depending on the specific use case [MS25, p. 5].

3.2.1.2 Control-Flow Graphs

To better reason about a program and produce more accurate results, static analyzers—whether within a compiler or as a standalone tool—implement a control-flow graph (CFG). A CFG connects statements, represented as nodes, with edges indicating possible execution flows. If a statement y follows statement x , there is an edge from x to y [AG04, p. 218]. This graph is used in various dataflow analyses, such as liveness analysis, which in turn enables optimizations like dead code elimination [AG04, p. 383]. For example, the code in Listing 3.3 results in the CFG shown in Figure 3.3a. CFGs can be viewed as a collection of program points where checkers can analyze aspects such as control paths or variable values. Although branches are common, analysis typically focuses on specific linear execution orders, also known as *paths*.

Listing 3.3: Sample C code for the control-flow graph

```

1 char *message = malloc(sizeof(char) * 13);
2
3 int msg_exists = 0;
4
5 getmsg(message, msg_exists);
6
7 if (msg_exists)
8     printf(message);
9
10 free(message);

```

An analysis using a CFG is called *flow-sensitive*: it can distinguish that the variable `message` in Listing 3.3 might have different values at Line 5 and Line 8, instead of treating it as the same variable throughout the whole program. This allows checkers to make more informed decisions. Even more precise are *path-sensitive* checkers. In the CFG shown in Figure 3.3a, the same `free` node is always reached regardless of whether the `if` condition is true or false. Figure 3.3b illustrates a path-sensitive CFG: the true and false branches result in completely separate paths. The downside is increased graph size and higher analysis cost, as more paths need to be explored.

A specialized form of static analysis using a path-sensitive approach is *symbolic execution* [Kin76], which simulates program execution using symbolic values instead of concrete data. Expressions whose values are not known at compile time are represented symbolically. When the value of a variable is known, it is referred to as a *constant* or *concrete value*.

Checkers can perform even more refined analysis by being *context-sensitive*:

If the analysis produces facts that depend on the history of procedure calls [it] is said to be context-sensitive. [Aho+07, p. 959]

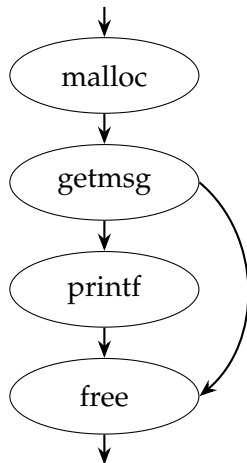


Figure 3.3a: Example of a control-flow graph

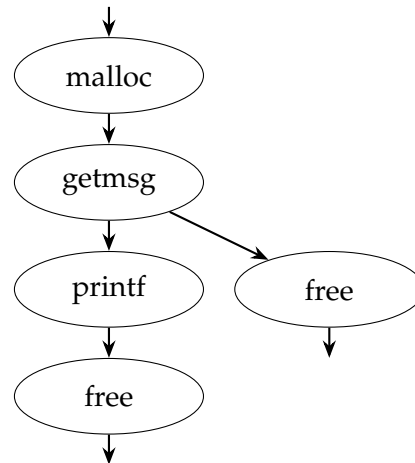


Figure 3.3b: Example of a path-sensitive control-flow graph

This, however, is a complex undertaking and beyond the scope of this thesis. Due to performance implications, compilers typically implement only flow-sensitive checks, while more intensive analyses are performed by separate tools, such as dedicated static analyzers.

3.2.2 Implementations of Static Analyzers

Table 3.3 compares various static analysis tools for the C programming language. Given the lack of financial resources, only open-source tools have been included in this thesis. In addition to a brief description, the table evaluates each tool with respect to the following desirable criteria:

- **Actively maintained:** Is the tool still being developed or maintained? If there has been no activity for at least a year, it is assumed to be unmaintained.
- **Integrable:** Can the tool be integrated into the development or compilation process to provide compiler-like feedback to the developer? Is the effort required for integration reasonable?
- **Customizable:** Can new, custom static analysis checks be added easily? Does the tool explicitly support extensibility?

The most promising tools can be grouped into full-fledged compilers (Clang, GCC), pattern matchers (Coccinelle, Semgrep), and formal verifiers (CPAChecker, Frama-C). Formal verifiers are beyond the scope of this thesis. Which tools, then, should be selected from the remaining categories?

Name	Description	Maint.	Integr.	Cust.
Clang [Cla25i]	The compiler frontend that uses LLVM as its backend provides a full framework for implementing static analysis checks.	✓	✓	✓
Clog [Lun25]	A declarative language bringing a Coccinelle-like interface to Clang.	✗	✗	✓
Coccinelle [Coc25]	A semantic patching tool originally created to ease driver code refactoring in the Linux kernel.	✓	✓	✓
CPAChecker [Sof25]	A static analysis tool for formal program verification.	✓	✓	✓
Frama-C [CEA25]	Similar to CPAChecker.	✓	✓	✓
GCC [GNU25b]	GNU's compiler includes several static analysis checks and provides a framework for implementing new ones.	✓	✓	?
Infer [Met25]	A static analysis tool developed at Facebook.	✓	✓	✗
Moose [Moo25]	A platform for software and data analysis.	✓	✗	✓
Semgrep [Sem25]	A pattern matcher with limited semantic understanding.	✓	?	✓
Sparse [For25]	A semantic parser written by Linus Torvalds and others for the Linux kernel. Creates a semantic parse tree.	✗	✓	✓
Splint [Uni25]	A "better lint." Uses source annotations to conduct static analysis.	?	✗	✓

Table 3.3: Comparison of different static analysis tools with regard to maintenance, integration, and customizability

In the compiler group, this thesis focuses on Clang. Although both Clang and GCC are well documented, Clang provides a dedicated framework for

developing static analyzers and is the more versatile choice. From the pattern matcher group, Coccinelle will be examined. While Semgrep is a relatively young project, Coccinelle is more mature, better documented, and supported by a larger body of academic literature, offering deeper insight into its inner workings.

Given the choice of Clang and Coccinelle, one might ask why Clog—which combines both—is not included. Although its approach of combining the best of both tools is promising, Clog unfortunately does not appear to be actively maintained. Furthermore, using it requires compiling the entire Clang project from source, which not only demands a strong computer, but also takes a considerable amount of time—especially given that most developers working with PostgreSQL already have a version of Clang installed. Therefore, the author does not consider it easy to integrate.

3.3 CLANG

Clang [Cla25c] is part of the LLVM project [LLV25a], a set of tools for building compilers and toolchains. Within that ecosystem, Clang serves as the frontend for the LLVM compiler. It was originally intended to be a unified parser for the C family of languages (i.e., C, Objective-C, C++, and Objective-C++) [Mur24, p. 6]. As part of LLVM, it includes many helpful tools, such as a debugger (LLDB) and the LLVM Integrated Tester (LIT) [LLV25b]. This section discusses the possibilities and limitations of static analysis in the context of Clang as a compiler.

3.3.1 *Clang as a Compiler*

As discussed in Section 3.1, Clang, as a compiler, performs lexical,⁷ syntactic, and semantic analysis, and translates the source code into IR. This IR is then passed to the LLVM backend and translated into machine code [Mur24, p. 44]. Suppose the code in Listing 3.4 is stored in a file called *hello.c*. Running `clang hello.c` will produce an executable file *a.out* that prints *Hello World* when executed, therefore, including all compilation phases.

Listing 3.4: Hello World program in C

```

1 #include <stdio.h>
2
3 int main() {
4     printf("Hello World\n");
5     return 0;
6 }

```

However, for static analysis, we only want to access the frontend and utilize the syntactic and semantic analyses. This can be achieved by passing `-cc1` or `-analyze` to Clang.⁸ This produces the AST constructed during the syntactic

⁷ Interestingly, lexical analysis is actually performed as part of the preprocessor in Clang [Mur24, p. 53].

⁸ Passing `-fsyntax-only` further restricts Clang to performing only syntactic analysis.

analysis,⁹ enriched with semantic information, without translating the code further. Clang also provides options for visualizing its internals: for example, passing `-Xclang -ast-print` outputs the constructed AST to the terminal.

3.3.2 Clang Static Analyzer

Clang provides two tools for static analysis: the Static Analyzer [Cla25g] and Clang-Tidy [Cla25h]. The Static Analyzer performs static program analysis as discussed in Section 3.2.1, whereas Clang-Tidy is a lightweight linter that accesses the AST but neither constructs a CFG nor performs symbolic execution. For this reason, only the Static Analyzer is discussed here.

The Static Analyzer is composed of various *checkers* implemented in C++ that analyze programs for specific issues. For example, the `MallocChecker` defines a set of memory-related checks that detect problems such as use-after-free, double-free, and memory leaks [Cla25l]. The LLVM framework and the Static Analyzer provide several classes [Cla25e] to aid checker development, including:

- `ASTContext`: holds the AST nodes.
- `ProgramState`: stores the program environment, including mappings from locations to values.
- `ExplodedNode`: stores a program point and a reference to the program state.
- `CheckerContext`: provides access to the program point and its predecessor as an `ExplodedNode`.
- `Stmt`: represents a statement.
- `Expr`: represents an expression as a subclass of `Stmt`.
- `FunctionDecl`: represents a function declaration or definition.
- `CallEvent`: abstracts a function call and stores the original expression, program state, location context, and a pointer to the corresponding CFG block (`CFGBlock`).
- `MemRegion`: abstraction used to represent memory regions.

This means that developers can access nearly all data processed by the compiler. So-called *checks* provide access to analyses on the CFG performed by Clang as part of static analysis and, therefore, to the elements described above. The checks used by a checker are declared via the class template [Cla25k] and include options such as `PreCall`, `PostCall`, and `DeadSymbols` [Cla25f]. These checks can then be used in the checker by implementing callbacks following the pattern `checkCheckName` [Cla25b]. For example, `checkPreCall`

⁹ This applies strictly to C. In certain languages like C++, semantic analysis is required during AST construction to support features such as template instantiation [Mur24, p. 36].

is triggered before a function call is made and provides access to both the `CallEvent` and the `CheckerContext`, which in turn provide access to elements such as the `FunctionDecl` and the original `Stmt`.

Beyond that, the Static Analyzer offers tools not typically available to the compiler. Its most powerful feature is symbolic execution, accessible via classes such as:

- `SVal`: a symbolic expression; can represent both l-values and r-values.
- `SymbolRef`: a reference to a symbolic expression.
- `SymbolicRegion`: a subclass of `SubRegion` (itself a subclass of `MemRegion`); serves as a placeholder for a real region and maps symbolic values to regions.

These elements can be retrieved via the `CheckerContext` and give developers of checkers powerful tools for creating sophisticated analyses.

3.3.3 Limitations

As Clang is a full-fledged compiler, it has few limitations regarding static analysis. The main limitation lies in its focus on a single *translation unit*:

*A C program is not required to be translated in its entirety at the same time. The text of the program is kept in units called source files (...). A source file together with all the headers and source files included via the preprocessing directive **#include** is known as a preprocessing translation unit. After preprocessing, a preprocessing translation unit is called a translation unit. [ISO24, p. 11]*

An example is shown in Listing 3.5: here, the file `otherFile.c` is included in Line 2, and a function from it is used in Line 6. Any file that is included by `otherFile.c` is also available, and so on.

Listing 3.5: A C program including a separate file

```

1  ...
2  #include "otherFile.c"
3  ...
4  int main(int argsNum, char **args){
5      ...
6      methodInOtherFile();
7      ...
8  }
```

All code processed by Clang during compilation is part of the translation unit,¹⁰ while precompiled code is not included by default. Cross-translation-unit analysis is possible by inlining function definitions [Cla25j], but this is beyond the scope of this thesis.

¹⁰ This unit is represented in the compiler as a top-level declaration called `TranslationUnitDecl` [Mur24, p. 59].

3.4 COCCINELLE

Coccinelle’s semantic patches enable automated code transformations using SmPL—a language developed specifically for this purpose. This section outlines Coccinelle’s motivation, features, logical foundations, and limitations.

3.4.1 *Semantic Patches*

Coccinelle was developed to address the increasing dependency of the Linux kernel on libraries and the corresponding need to keep interface invocations up to date to avoid breaking code [Mul+06, p. 1]. These changes are referred to as *collateral evolutions*, and they include [PLM07, p. 48]:

- changes in calls to library functions,
- changes in the number of arguments,
- changes in callback functions, and
- changes in the number of fields in data structures.

As of 2025, the Linux kernel exceeds 40 million lines of code [Sta25]. Manually editing all occurrences of an interface is both cumbersome and error-prone. Moreover, previously corrected patterns might inadvertently be reintroduced in later kernel versions. Automating this task is therefore desirable.

Coccinelle introduces so-called *semantic patches*, written in the *Semantic Patch Language* (SmPL) [PLM07], which was specifically developed to automate code revision.

A patch is a set of changes made to source code to fix bugs, add features, or improve functionality. Patches are typically distributed as unified *diff files* that show the differences between the original and modified versions of the code. They contain context lines (unchanged code) along with lines marked with + (additions) and - (deletions), allowing developers to understand exactly what changes were made and to apply them to their codebase using tools like `patch` or version control systems.

Consider the simple C function in Listing 3.6, which contains a use-after-free vulnerability. Here, a character array is allocated in Line 6, and a string is copied into it in Line 7. In Line 8, the pointer to the array is freed, but then accessed again in Line 9. Obviously, we would want to swap Lines 8 and 9 to fix this issue. The corresponding patch is shown in Listing 3.7. Line 3 indicates the context area being displayed: starting at Line 5 and showing 7 lines from both the original and fixed versions of the file. The patch relocates one line (`free(buf);`) from its original position to after the `printf` statement. The format shows which specific lines are removed (marked with -) and added (marked with +), while unmarked lines provide context. By applying patches to the source code, the process of fixing program errors can be automated.

Listing 3.6: C code with use-after-free

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main() {
6      char *buf = malloc(sizeof(char) * 12);
7      strcpy(buf, "Hello World");
8      free(buf);
9      printf("Data: %s\n", buf); // USE AFTER FREE
10     return 0;
11 }

```

Listing 3.7: Example of a patch file

```

1  --- originalFile.c      2025-07-22 16:55:11.520010475 +0200
2  +++ fixedFile.c 2025-07-22 16:55:29.392709977 +0200
3  @@ -5,7 +5,7 @@
4   int main() {
5       char *buf = malloc(sizeof(char) * 12);
6       strcpy(buf, "Hello World");
7   -   free(buf);
8       printf("Data: %s\n", buf); // USE AFTER FREE
9   +   free(buf);
10      return 0;
11  }

```

Semantic patches, however, go beyond traditional patches and enable matching specific patterns and replacing them with alternative code—referred to as *transformations* [PLMo7]. These transformations are independent of concrete line numbers and can match several occurrences across a code base. SmPL supports the definition of placeholders for constructs such as variables, structs, and functions. It is independent of code formatting and whitespace and allows for omitting parts of the code base in matching patterns [PLMo7, pp. 51–54]. Additionally, it can leverage type information [LM22, p. 77] and abstract away coding constructs such as loops by using CFGs and dataflow information. These abstractions, referred to as *isomorphisms*, allow structurally different but semantically equivalent code to be matched [PLMo7, p. 54].

SmPL is intentionally designed to resemble actual C code [PLMo7, p. 55], making it easier for developers and maintainers to write and understand patches.

As an example, consider the code in Listing 3.8. It shows memory allocation immediately followed by a call to `memset`, initializing the memory to 0. However, C provides the `calloc` function, which performs both actions in a single call:

```
struct foo *ptr = calloc(1, sizeof(struct foo));
```

This has the same effect as the code in Listing 3.8 (see also [LM22, pp. 74–78]). The latter version, however, is more concise, expressive, and might even enable better compiler optimizations. We would therefore like to replace all occurrences of this pattern in our code base.

Listing 3.8: Snippet of function with suboptimal code

```

1 struct foo *ptr = malloc(sizeof(struct foo));
2 if (ptr)
3     memset(ptr, 0, sizeof(struct foo));

```

Listing 3.9 shows a Coccinelle patch that performs this transformation. In the header, metavariables such as types and identifiers are defined and then used as placeholders in the patch body. The pattern to match is defined first, followed by the replacement code. Code lines prefixed by - are removed, while those prefixed by + are added.

Listing 3.9: Coccinelle patch for the code snippet

```

1 @@
2 type T;
3 identifier i;
4 @@
5 - T *i = malloc(sizeof(T));
6 - if (i)
7 -     memset(i, 0, sizeof(T));
8 + T *i = calloc(1, sizeof(T));

```

Using Coccinelle, large code bases can be efficiently patched, and flexible, reusable revisions can be implemented.

3.4.2 A Closer Look

The foundation for transformations in Coccinelle is a specialized form of Computation Tree Logic (CTL). In CTL, patterns are expressed as logical formulas, and the program is represented by a CFG. Model checking is used to identify nodes in the CFG that satisfy these formulas. As the name suggests, different paths of code execution are expanded into an infinite tree.

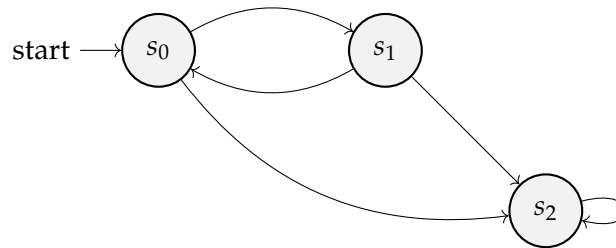


Figure 3.4: Program with three states and specified transitions

Figure 3.5 illustrates this concept, where the program shown in Figure 3.4 is represented—similarly to a CFG—by the states s_0 , s_1 , and s_2 , which are connected by transitions. Each state satisfies certain propositions (i.e., p , q , and r). The logic allows checks to compare these propositions against desired conditions on different paths using the quantifiers $\exists$ —represented by E —and $\forall$ —represented by A —, as well as operators derived from linear-time temporal logic [HR04, pp. 207–209]: a formal system that “models time as a sequence of states (...) consider[ing] several paths, representing different possible futures[.]” [HR04, p. 175] These operators include X , F , G , and U , representing

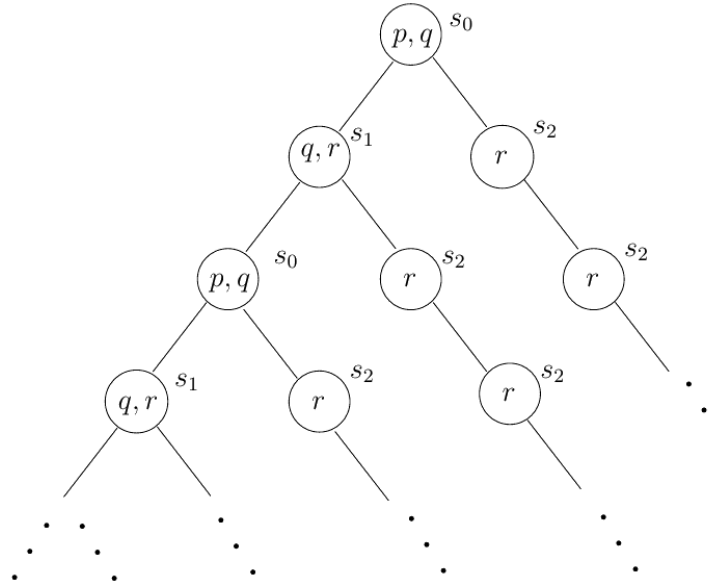


Figure 3.5: Infinite tree of all computation paths beginning in a particular state
 Source: [HR04, p. 180]

“neXt state”, “some Future state”, “Globally”, and “Until”, respectively [HR04, p. 208]. For instance, a formula using AX holds if all immediate successor states satisfy it; EG holds if there exists a path where every state satisfies the formula. Being able to discern different paths, CTL is path-sensitive.

Specialized forms of CTL enable increasingly nuanced analyses [Bru+09]:

- **CTL-FV:** introduces *free variables*, allowing the use of metavariables that are resolved via an environment.
- **CTL-V:** allows different *variable* bindings along different control-flow paths.
- **CTL-VW:** adds *witnesses*, which track where and how each part of a match occurs. This enables transformations to be applied even when only partial matches are found.

These extensions make CTL-VW a powerful tool for static analysis and code transformation: they enable the identification of complex patterns in control flow and allow precise tracking of how and where matches occur.

3.4.3 Limitations

Coccinelle was designed to be lightweight to efficiently handle large code bases such as the Linux kernel [LM18, p. 602]. However, this design comes with limitations. Although CTL can track variables, it does not have access to concrete values like a compiler does. Program analyses involving specific val-

ues or even value ranges are therefore not possible with Coccinelle¹¹ [Ole+10, p. 2].

Coccinelle also does not perform alias analysis and only offers a light version of type inference—a best-effort approach that omits information by not including header files [LM18, p. 603]. It cannot differentiate between types with and without qualifiers: for example, `const int` is treated the same as `int`.

Another limitation is the scope of the analysis: rules are only applied to individual files and, within those, to the individual functions or other top-level declarations. In other words, the rules are intraprocedural [LM18, p. 603].

3.5 SUMMARY

Table 3.4 provides a comparison of the tools discussed in this chapter. While Coccinelle and Clang-Tidy lack symbolic execution and either AST or CFG support, they can be considered lightweight compared to the Clang Static Analyzer. The latter, although offering access to AST, CFG, and symbolic execution, introduces a higher overhead than the other tools.

Tool	AST	CFG	Symb. Ex.	Overhead
Coccinelle	✗	✓	✗	low
Clang-Tidy	✓	✗	✗	low
Clang Static Analyzer	✓	✓	✓	high

Table 3.4: Comparison of Coccinelle, the Clang Static Analyzer, and Clang-Tidy

Another fundamentally different approach is to implement PostgreSQL extensions in Rust, which is possible via the dedicated driver [Fac25]. As shown in Section 2.2.1, this would eliminate the memory-related problems discussed in this thesis altogether. However, since the core functionality of PostgreSQL is implemented in C, writing extensions in C is expected to remain popular.

¹¹ Olesen et al. [Ole+10] work around this issue by using Clang to obtain program analysis information and incorporating it into Coccinelle.

POSTGRESQL AND DOMAIN-SPECIFIC PATTERNS

This chapter discusses the first phase of developing the Checker (see Figure 4.1). It covers the analysis of the PostgreSQL codebase and highlights PostgreSQL-specific patterns. To this end, the first section addresses the challenges arising from PostgreSQL's specific approach to memory management, with a focus on particular types of functions. The second section explains how relevant functions are extracted from the PostgreSQL codebase in order to derive a function catalog, which serves as the foundation for the analysis. The third section examines the different categories that have been identified and discusses their implications for static analysis.

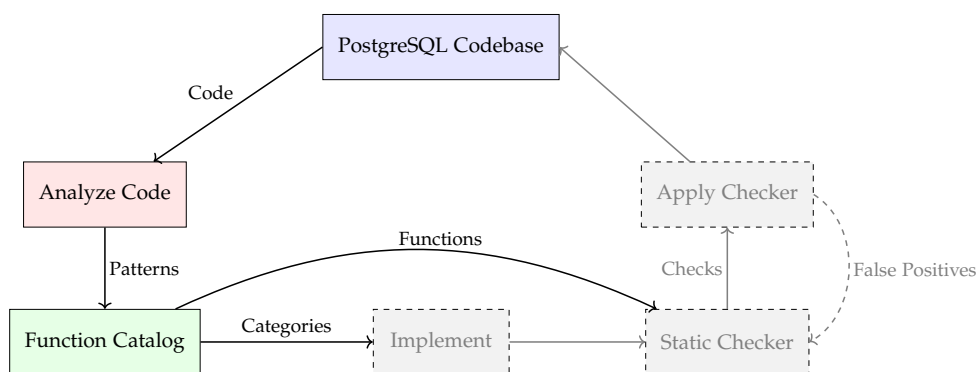


Figure 4.1: Phase 1 of developing the PostgresChecker

4.1 POSTGRESQL VS. STANDARD C

PostgreSQL is written in C,¹ which places the burden of memory management on the developer. This can lead to numerous potential bugs and security issues (see Chapter 2).

PostgreSQL addresses this challenge by introducing *memory contexts* [Albo3]: for every process, so-called memory contexts are created and expanded dynamically by allocating memory as needed. When the developer requires memory, they do not call `malloc` directly but instead request memory from the current context. The main advantage of this approach is that, at the end of a process, all allocated memory is automatically released by destroying the existing contexts, relieving the developer of this burden.

To achieve this, special functions are introduced: memory is allocated using `palloc`; the function `repalloc` resizes memory within the same context; and `pfree` can be used to manually release memory before the context ends. There

¹ PostgreSQL uses the C99 standard [Pos25j, p. 2452].

are also several variants of both `palloc` and `repalloc`, which are implicitly included in this discussion when referring to these functions.

At first glance, these PostgreSQL functions appear similar to their equivalents in standard C. However, in practice, there are significant differences in how memory management is handled. One notable example is the use of `pfree` and `repalloc` within function calls.

In typical C programs, memory management is very explicit. Functions generally do not allocate or free memory internally but instead rely on caller-allocated memory. The responsibility for providing pointers to appropriately sized memory regions for the task performed by the function lies with the caller. For example, the standard `strcat` function, shown in Listing 4.1, expects the caller to provide a destination buffer large enough to hold the concatenated string. The function does not call `malloc` or `realloc` internally to enlarge the provided char array; memory management is left to the caller, keeping it transparent and predictable.

Listing 4.1: Memory management in traditional C programming patterns

```

1  ...
2  size_t length = strlen(string1) + strlen(string2) + 1;
3  char *destination = (char *) malloc(sizeof(char) * length);
4  strcpy(destination, string1);
5  strcat(destination, string2);
6  ...
7  free(destination);
8  ...

```

The implementation of `strcat` in glibc [GNU25a], shown in Listing 4.2, further illustrates this principle. There are no calls to `free` or `realloc`; the function simply modifies existing memory. From a purely functional perspective, even returning the pointer is redundant—the function modifies the memory in place, and the pointer remains valid unless explicitly changed by the caller. A framework that makes heavy use of this pattern is the MPI API, mentioned in Section 2.2.2.

Listing 4.2: Implementation of the `strcat` function in glibc

```

1  char *
2  STRCAT (char *dest, const char *src)
3  {
4      strcpy (dest + strlen (dest), src);
5      return dest;
6  }

```

In contrast, PostgreSQL functions frequently handle memory reallocation internally. A prototypical example is shown in Listing 4.3, which displays the logic used in the `bms_add_member` function [Pos25p], used to add new members to a `Bitmapset` (see Chapter 1). In Line 1, the function receives a pointer to the `Bitmapset` and a number `x` to add. If `x` exceeds a certain threshold, the `Bitmapset` cannot hold the value and, in Line 4, needs to be enlarged, which causes the passed pointer to become dangling. Developers must be aware of how the function manages the passed pointer and handle it accordingly.

Listing 4.3: Function reallocating memory internally in PostgreSQL

```

1 Bitmapset * bms_add_member(Bitmapset *a, int x){
2     ...
3     if (x > SOME_THRESHOLD)
4         a = repalloc(a, NEW_SIZE);
5     ...
6     return a;
7 }

```

In this case, the return value is no longer redundant but is *supposed* to be assigned. If the programmer fails to use the returned pointer, they risk using a dangling pointer, which can lead to undefined behavior or memory errors. This pattern introduces an abstraction layer that hides memory management details, unlike traditional C practices where the caller maintains full control.

An example of problematic usage of the function described above is shown in Listing 4.4. In Line 3, the `Bitmapset` created in Line 1 is passed to the function, which may internally reallocate the memory region. Since the return value is not reassigned, in Line 5, a potentially dangling pointer is accessed, constituting a possible case of use-after-free.

Listing 4.4: Problematic use of a PostgreSQL-style function

```

1 Bitmapset *a = bms_make_singleton(42);
2 ...
3 bms_add_member(a, 64);
4 ...
5 if (bms_is_member(x, a)) {
6     ...

```

Resources may also be freed in a similar fashion. An example of such a function is shown in Listing 4.5 [Pos25e], which demonstrates how PostgreSQL handles the reference counting of tuple descriptors. The function decrements the reference count of the passed `TupleDesc` object and frees it once the count reaches 0. Consequently, the passed object cannot be safely used unless the caller is certain that the counter has not reached 0.

Listing 4.5: Memory management and functions in PostgreSQL

```

1 void
2 DecrTupleDescRefCount(TupleDesc tupdesc)
3 {
4     Assert(tupdesc->tdrefcount > 0);
5
6     ResourceOwnerForgetTupleDesc(CurrentResourceOwner, tupdesc);
7     if (--tupdesc->tdrefcount == 0)
8         FreeTupleDesc(tupdesc);
9 }

```

4.2 DATA BASIS

A static analyzer should warn developers when a freed resource is accessed again. To emit such warnings, the analyzer must be aware of which functions potentially free or reallocate a passed resource. The approach and implementation used to identify these functions are discussed in this section.

4.2.1 Approach

To address the memory-related issues discussed in the previous section, the data basis for this thesis consists of functions related to memory management in the PostgreSQL codebase (Version 17.2). To identify the relevant functions, a fixpoint algorithm is employed, as defined in Algorithm 1.

Algorithm 1 Allocation of memory management-related functions

```

1 output_set :=  $\emptyset$ 
2 iteration_set := {"free", "pfree", ...}
3 while output_set changes do
4   matches :=  $\emptyset$ 
5   for function in iteration_set do
6     matches  $\leftarrow$  {all functions that call function on a parameter}
7   end for
8   iteration_set :=  $\emptyset$ 
9   for match in matches do
10    iteration_set  $\leftarrow$  match.name
11  end for
12  output_set := output_set  $\cup$  iteration_set
13 end while
14 return output_set

```

In this algorithm, the *output_set* gathers all functions that “behave” like an initially provided set of functions (see Line 2). This is achieved by extracting all functions that internally call one of the functions already discovered. The while loop iteratively collects relevant functions from the codebase until the *output_set* no longer changes. A dedicated *iteration_set* keeps track of the functions detected in the previous iteration. While this is not strictly necessary, iterating over the entire *output_set* in every step would be costly.

The initial iteration typically includes only `free` or `realloc`, representing the base cases of memory management. However, to derive more fine-grained results, additional functions discovered in earlier runs may be added to the initial set. In Algorithm 1, for example, `pfree` is included alongside `free`.

4.2.2 Implementation

The implementation of Algorithm 1 uses Coccinelle scripts to detect relevant patterns in the PostgreSQL codebase (as in Line 6).²

For the purpose of generating Coccinelle scripts, templates are used. These templates allow a separate script to be automatically generated for each function in the `iteration_set`. Each template covers different patterns to check. The most basic case involves detecting any usage of `free` or `realloc` within a function. The template for this case is shown in Listing 4.6. The header in Line 1 includes the rule name “freeing” with the `exists` modifier, which ensures that the rule matches *any* path.

Listing 4.6: Template script for finding all occurrences of a function

```

1 @freeing exists@
2 type rt, t;
3 identifier f, i;
4 position p;
5 @@
6 rt f(..., t i, ...) {
7     <+...
8     __FUNCTION__@p(..., i, ...)
9     ...+>
10 }
11
12 @script:python@
13 f << freeing.f;
14 p << freeing.p;
15 t << freeing.t;
16 rt << freeing.rt;
17 @@
18 if t == __CHECKTYPE__:
19     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
20     print(f"Used: __FUNCTION__")
21     print(f"Ret. Type: {rt}")

```

The Coccinelle script defines the following metavariables to track information about each match:

- `rt`: the return type of the function
- `t`: the type of the parameter that is freed or reallocated
- `f`: the name of the function
- `i`: the name of the parameter
- `p`: positional information about the match

In Line 6, the rule itself matches all functions that call a freeing or re-allocating function on a passed parameter. `<+...+>` ensures the freeing or

² Why not simply use `grep`? Because Coccinelle scripts are significantly more flexible: they allow matching across either all or any paths, enabling more expressive searches and supporting constraints on these paths. Furthermore, Coccinelle abstracts away many syntactic details; for example, a function parameter can be matched simply as `(..., Type identifier, ...)`. This simplifies matching signatures that span multiple lines. Achieving similar functionality with tools like `grep` or `awk` would be much more complex.

reallocating function is called one or more times. For each function in the iteration set, a script is created from the template by replacing the placeholder `__FUNCTION__` with the actual function name.

Listings 4.7 and 4.8 show examples of matched and unmatched functions when using the template for the function `free`. Only the functions in Listing 4.7 are matched, as the first function in Listing 4.8 does not contain `free` on any reachable path, and the second one does not free a passed parameter.

Listing 4.7: Matched functions

```

1 void matched_one(char *message){
2     printf("%s", message);
3     free(message);
4 }
5 void matched_two(char *message){
6     if (message){
7         printf("%s", message);
8         free(message);
9     }
10 }
```

Listing 4.8: Unmatched functions

```

1 void unmatched_one(char *message){
2     printf("%s", message);
3     return;
4     free(message);
5 }
6
7 void unmatched_two(char *message){
8     char *str = malloc(10 * sizeof(char));
9     free(str);
10 }
```

Each Coccinelle template includes a Python script that prints important information (see Listing 4.6, Line 12). The script first checks whether the argument passed to the freeing or reallocating function is actually of the expected type. This is necessary to avoid false positives. For example, if a function `void somefunction(TypeA *a, TypeB *b)` only frees `a`, parameters of `TypeB` should not be considered in subsequent iterations. The script distinguishes between `__CHECKTYPE__` and `__PRINTTYPE__`. For the base case, `__CHECKTYPE__` is replaced with `t` because the type is arbitrary. This leads to a trivial `t == t` condition, which accepts all matches. `__PRINTTYPE__`, on the other hand, is still replaced by the actual type of the parameter.

A sample output of the Python script is shown in Listing 4.9.

Listing 4.9: Python output of the Coccinelle scripts

```

1 >function_name,type_freed, file:line:column
2 Used: function_used
3 Ret. Type: type_returned
```

The resulting set $\mathcal{A}$ of matched functions is gradually reduced by identifying aspects that constitute a category, resulting in the set $\mathcal{B}$ of functions satisfying that category's properties. Afterwards, $\mathcal{A} \setminus \mathcal{B}$ is examined, and the process continues until no further relevant categories can be found. For this purpose, more advanced Coccinelle scripts are applied that follow a similar pattern to Listing 4.6. These scripts will be introduced in Section 4.3 within the respective categories, and their differences will be highlighted. All scripts and results are available at [Rei25a].³

Figure 4.2 shows the overall structure of the scripting environment. The main folder is `scripts`, which contains a subfolder with all the templates.

³ When downloading and running the Docker container from [Rei25c], the results can also be reproduced by running `make` in `/workspaces/pg-dev-container/coccinelle`.

The `cocciCreator.sh` script implements Algorithm 1 and can be found in Appendix A.1. The separate script `runCocci.sh` is used to call the cocci creator with a specific function (`free` or `realloc`) and template. The script `collectResults.sh` (see Appendix A.5) prepares the results—the exact functionality of this and the remaining scripts will be explained in the following sections. The folder `scripts/exceptions` contains functions for templates that should be manually added or excluded. The `scripts/test` folder contains tests checking the correctness of the templates used.

In addition to the `scripts` folder, there are the `results` and `logs` folders, which are generated when the scripts are executed. All output from Coccinelle is logged. The `results` folder contains a subfolder for each template, which itself contains a subfolder for each iteration. The output of the Python script is saved in `results.out`. Finally, the `collected` folder contains results prepared by `collectResults.sh`, including not only the functions for each category but also files with statistical information.

4.3 CATEGORIES

Since all functions in the data basis are related to `free` or `realloc`, they can be generalized as use-after-free problems, as described in Section 2.1.3. However, what this implies for developers may differ from case to case. For instance, both `pfree` and `repalloc` can lead to dangling pointers. However, when a function uses `repalloc`, the return value should typically be reassigned. If the function instead uses `pfree`, the pointer should most likely not be used at all after the call. These two scenarios differ in quality, leading to the following questions:

- What categories of functions that actively manage memory can be identified?
- How do these categories differ?
- How should a static checker handle these categories?

Depending on the answers to these questions, a static checker needs to provide different information to the developer. This section explores the different categories found in the PostgreSQL codebase:

1. **Strict functions:** functions that always free or reallocate a passed parameter.
2. **Dependent functions:** functions where freeing or reallocation depends on another parameter.
3. **Arbitrary functions:** functions that may free or reallocate a parameter, but the condition cannot be deduced statically.
4. **Reassigning functions:** functions where the return value is supposed to be reassigned.

```

/
├── scripts/
│   ├── test/
│   │   ├── expected/
│   │   │   ├── free.out
│   │   │   ├── freestrict.out
│   │   │   └── ...
│   │   ├── test.c
│   │   └── runTest.sh
│   ├── exceptions/
│   │   ├── freeall.exclude
│   │   ├── freestrict.add
│   │   └── ...
│   ├── templates/
│   │   ├── template.cocci
│   │   ├── templatestrict.cocci
│   │   └── ...
│   ├── cocciCreator.sh
│   ├── runCocci.sh
│   ├── collectResults.sh
│   ├── getArgumentPosition.sh
│   └── getDuplicates.sh
├── results/
│   ├── free/
│   │   ├── Iteration_0/
│   │   ├── Iteration_1/
│   │   ├── ...
│   │   ├── results.out
│   │   ├── functionnamesonly.out
│   │   └── functionnamesonlywithoutstatic.out
│   ├── freestrict/
│   ├── ...
│   ├── collected/
│   │   ├── C1_free.out
│   │   ├── C1_free_functiononly.out
│   │   ├── C1_realloc.out
│   │   ├── ...
│   │   ├── stats_free.out
│   │   └── stats_realloc.out
│   ├── all.out
│   └── allDuplicate.out
├── logs/
└── Makefile

```

Figure 4.2: The directory layout of the scripting environment

5. **Functions returning boolean:** functions that indicate, via their return value, whether a parameter was freed or reallocated.

After applying Algorithm 1 with the basic template, 426 functions that free and 30 functions that reallocate a passed parameter were identified. However, many of these functions are used only *internally* in PostgreSQL: they use the keyword `static`, meaning they are visible only within the file where they are defined and cannot be used externally.⁴

After excluding static functions, 235 functions remain for free and 25 for `realloc`. Static functions were initially included in the search because non-static functions can still invoke them; the removal is performed later by the `collectResults.sh` script. All numbers presented in the following categories assume that static functions have already been removed.

Two issues remain in this setup. First, a freeing or reallocating function may take two parameters of the same type but free or reallocate only one of them. Currently, both parameters are assumed to be freed or reallocated, which may lead to false positives. To address this, a dedicated template was used to identify such functions (see Appendix A.4). Since only eight functions for free and two for `realloc` were affected, they were reviewed manually.

Second, different files can define functions with identical names but distinct semantics. Some of these duplicates may belong to different categories or be irrelevant entirely. The `getDuplicates.sh` script detects such cases by collecting all function names in the PostgreSQL codebase,⁵ extracting duplicates, and reviewing all affected matches manually.

4.3.1 C1: Strict Functions

The first useful constraint is to consider only functions that *always* free or reallocate a passed parameter. These functions are referred to as *strict*. To identify them, the template ensures that the freeing or reallocating function call occurs along *all* execution paths. This is achieved using the `forall` keyword in Coccinelle. Additionally, the template excludes functions where a return statement might occur before the relevant function call. The basic template is shown in Listing 4.10. The `when` condition in Line 8 excludes all matches where a return occurs before the function call. The code from Line 10 on allows for zero or more further occurrences.

⁴ Why remove them, though? Adding additional functions brings additional complexity to the checker, increasing the potential for errors. Furthermore, the function names introduced to the checker could potentially collide with function names used by the developer. Moreover, we do not gain anything from including static functions. The only case in which they would be relevant is when checking the PostgreSQL codebase itself. But in that special case, we are already operating within the file and can see the function body. The only reason we need to introduce function names to the checker is that, in C, functions are often declared in header files without a body. However, when analyzing the file with the actual code, the checker can see whether functions contain a freeing or reallocating function (like `pfree`) and can handle it correctly.

⁵ Fun fact: the PostgreSQL codebase contains 26,146 functions.

Listing 4.10: Basic template for strict functions

```

1 @freeing forall@
2 type rt, t;
3 identifier f, i;
4 position p;
5
6 @@
7 rt f(..., t i, ...) {
8   ... when != if(...){ ... return ...;}
9   __FUNCTION__@p(..., i, ...)
10  <... when any
11  __FUNCTION__(..., i, ...)
12  ...>
13 }

```

However, this approach excludes functions with valid patterns such as `if (i == NULL) {return;}` or `if (i){free(i);}`. These cases are captured by additional templates shown in Listing 4.11 and Listing 4.12. Variants using `NIL` instead of `NULL` and `while (i)` instead of `if (i)` are also supported. The complete template, including all variants, is provided in Appendix A.2.

Listing 4.11: Template for strict functions with NULL check

```

1 ...
2 if (i == NULL) {... return ...;}
3 ... when != if(...){ ... return ...;}
4 __FUNCTION__@p(..., i, ...)

```

Listing 4.12: Template for strict functions with valid pointer check

```

1 ... when != if(...){ ... return ...;}
2 if (i){...
3 __FUNCTION__@p(..., i, ...)
4 ...}

```

Listings 4.13 and 4.14 illustrate matched and unmatched cases. The first three matched functions always free message when it is not NULL. In contrast, `unmatched_one` frees message only under certain conditions, and the function `unmatched_two` has a return statement preventing certain paths from reaching the freeing function.

Listing 4.13: Matched functions

```

1 void matched_one(char *message){
2   free(message);
3 }
4 void matched_two(char *message){
5   if (message)
6     free(message);
7 }
8 void matched_three(char *message){
9   if (message == NULL)
10    return;
11   free(message);
12 }

```

Listing 4.14: Unmatched functions

```

1 void unmatched_one(char *message){
2   if (strlen(message) > 5)
3     free(message);
4 }
5
6
7
8 void unmatched_two(char *message, int x){
9   if (x > 5)
10    return;
11   free(message);
12 }

```

Using these rules and excluding static functions, 186 strict functions were identified for free and 19 for realloc.

Incorrect use of strict functions should trigger the following warnings:

- **use-after-free:** Attempt to use freed memory
- **double-free:** Attempt to free released memory

After removing strict functions from the initial set, the remaining functions are reduced to 49 for `free` and 6 for `realloc`, representing a significant reduction.

4.3.2 C2: *Dependent Functions*

The remaining functions *may* modify the passed parameter, but only under certain conditions. In many cases, freeing or reallocation depends on another parameter or on a field of a structure. An example is shown in Listing 4.15 [Pos25f], where the parameter `mtup` is freed only if `shouldFree` is set to `true`. Such functions are referred to as *dependent*.

Listing 4.15: Function where freeing depends on another parameter

```

1 void
2 ExecForceStoreMinimalTuple(MinimalTuple mtup,
3                             TupleTableSlot *slot,
4                             bool shouldFree)
5 {
6     ...
7     if (shouldFree)
8     {
9         ExecMaterializeSlot(slot);
10        pfree(mtup);
11    }
12    ...

```

To identify dependent functions, the template in Listing 4.16 was used. Here, the call to the freeing or reallocating function depends on the parameter `b` in Line 9. The variation in Listing 4.17 handles the case where the control parameter is declared before the freed or reallocated parameter. To catch dependencies on fields of parameters (e.g., `b->a`), Listing 4.18 was added. Templates for dependencies like `i->a` were also included, signifying that the dependent value is a field of the freed or reallocated parameter itself.

Listing 4.16: Template for functions depending on a passed parameter

```

1 @freeing0 exists@
2 type rt, t1, t2;
3 identifier f, i, b;
4 position p;
5
6 @@
7 rt f(..., t1 i, ..., t2 b, ...) {
8     <+...
9     if (b) {... __FUNCTION__@p(..., i, ...) ...}
10    ...+>
11 }

```

Listing 4.17: Template for reversed parameter order

```

1 rt f(..., t2 b, ..., t1 i, ...) {
2   <+...
3   if (b) {... __FUNCTION__@p(..., i, ...) ...}
4   ...+>
5 }

```

Listing 4.18: Template for dependencies on parameter fields

```

1 <+...
2 if (b->a) {... __FUNCTION__@p(..., i, ...) ...}
3 ...+>

```

However, these templates do not guarantee that the functions strictly free or reallocate the parameter when used recursively in Algorithm 1. Therefore, the algorithm begins with all strict functions (from Category 1) and stops after the first iteration. This may miss some edge cases. Additionally, the template excludes any conditions similar to $b == x$ or $b > y$. However, the script already contains 12 rules (see Appendix A.3). Addressing all cases would result in a bloated template. Furthermore, some cases may not even necessarily be detectable using simple heuristics. The function call may, for example, depend on more than one parameter or access a field of a field. Additionally, many field values determining whether a parameter is freed can be resolved only at runtime. For these reasons, the scripts in this category are only a rough approximation. Therefore, the results—and more importantly, the remaining functions—need to be reviewed manually. Fortunately, many functions are already filtered out in Category 1.

Overall, 14 dependent functions were identified for `free` and 1 for `realloc`. These functions eventually resolve to the categories `strict` or `arbitrary` (see next section) and thus share the same warnings.⁶

4.3.3 C₃: Arbitrary Functions

This category includes all remaining functions. They may free or reallocate the passed parameter, but the conditions under which this happens cannot be determined statically. An example is shown in Listing 4.19 [Pos25n]: the function `add_path` adds a new path to a relation only if it is better than the existing paths already known by the relation. Otherwise, the new path is freed. Since this decision is based on runtime data, the behavior is unpredictable for a static analyzer, and such functions are therefore referred to as *arbitrary*.

Listing 4.19: Function where freeing of a parameter is arbitrary

```

1 void
2 add_path(RelOptInfo *parent_rel, Path *new_path)
3 {
4   bool accept_new = true;      /* unless we find a superior old path */
5   ...
6   /* CHECK WHETHER new_path IS BETTER, IF SO SET accept_new = false */

```

⁶ If they do not resolve to either category, they simply do not free or reallocate at all.

```

7   ...
8   if (accept_new)
9   { ... }
10  else
11  {
12      /* Reject and recycle the new path */
13      if (!IsA(new_path, IndexPath))
14          pfree(new_path);
15  }
16 }

```

In total, 35 such functions were found for `free` and 5 for `realloc`.

Incorrect usage of these functions should produce the following warnings:

- **use-after-free:** Attempt to use potentially freed memory
- **double-free:**
 - **First freeing function is strict:** Possible attempt to free released memory
 - **First freeing function is arbitrary:** Possible attempt to free potentially released memory

4.3.4 C4: Reassigning Functions

Some functions can be identified by their usage pattern rather than their freeing behavior. For example, functions that internally call `realloc` typically require the return value to be reassigned, regardless of whether the memory was actually reallocated. As long as the returned value is used, there is no risk of using a dangling pointer. Such functions are referred to as *reassigning functions*.

An example is shown in Listing 4.20 [Pos25y], which deletes the n -th element of a list. If the list becomes empty, it is freed and `NIL` is returned (representing an empty list in PostgreSQL); otherwise, the modified list is returned.

Listing 4.20: Function where return value should be reassigned

```

1 List *
2 list_delete_nth_cell(List *list, int n)
3 {
4     ...
5     if (list->length == 1)
6     {
7         list_free(list);
8         return NIL;
9     }
10    ... /* DELETE THE NTH ELEMENT */ ...
11    return list;
12 }

```

To retrieve these functions, the previously aggregated functions were filtered for a matching return type and parameter type of the value being freed or reallocated. This logic was implemented as part of the `collectResults.sh` script (see Appendix A.5) and resulted in 18 functions for `free` and 25 functions for `realloc`. However, matching types do not guarantee that the return

value is *supposed* to be reassigned. Therefore, these functions were reviewed manually to exclude false positives.

One reallocating function was added manually because it takes a parameter of type `void *` and casts it to `char *` after reallocation [Pos25u]. So, while the passed and returned types do not match, the function still falls into this category.

For these functions, the same warnings as before apply, but the checker should additionally warn if the return value is ignored.

4.3.5 C5: Functions Returning Boolean

The function `ecpg_check_PQresult` [Pos25t] illustrates a special case: it always returns `false` if the passed parameter is freed and `true` otherwise. In this case, correct usage requires the caller to check the return value and set the variable to `NULL` if `false` is returned, representing a clear usage pattern as well.

Similar to Category 4, developers should be warned if the return value is ignored, as this can lead to undefined behavior.

4.4 SUMMARY

Table 4.1 summarizes the results of the categories discussed in this chapter.

Category	Free	Realloc
All Functions	426	30
All without static	235	25
C1: Strict Functions	186	19
C2: Dependent Functions	14	1
C3: Arbitrary Functions	35	5
C4: Functions to Reassign	18	25
C5: Functions Returning Boolean	1	0

Table 4.1: Number of functions by category

These categories describe sets of functions specific to the PostgreSQL API. Most of the identified functions behave strictly like `free` or `realloc`. However, in about 20% of cases, it cannot necessarily be determined with certainty whether a parameter has been freed or reallocated. In addition, two categories describe specific usage patterns that developers are expected to follow. These aspects influence the types of warnings a static checker should emit.

The next chapter discusses how these categories can be addressed, where possible, using Coccinelle and the Clang Static Analyzer.

IMPLEMENTING A STATIC ANALYZER

This chapter discusses the second phase of developing the `PostgresChecker`, focusing on its implementation. It applies the categories and function catalog prepared in Chapter 4 to address PostgreSQL-specific issues (see Figure 5.1). The possibilities and limitations of Coccinelle and Clang are examined, while the discussion of the feedback loop resulting from applying the checker is reserved for Chapter 6.

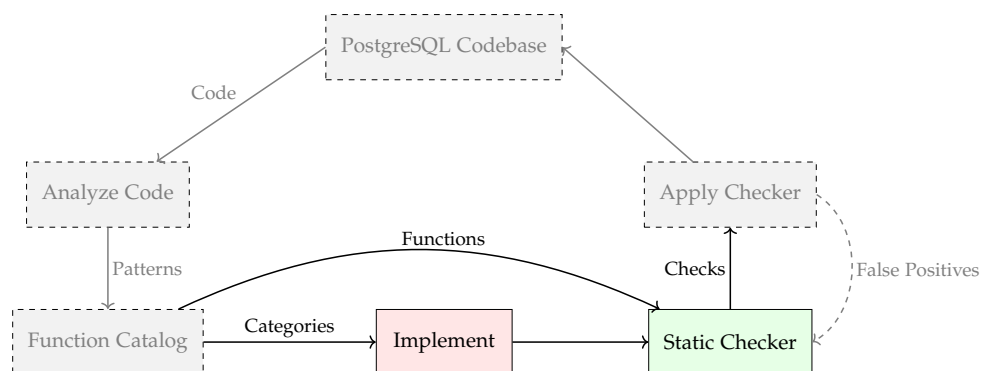


Figure 5.1: Phase 2 of developing the `PostgresChecker`

For reasons outlined in the following sections, the checker was implemented using the Clang Static Analyzer framework with Clang version 19.¹ This chapter provides an overview of the most important concepts used in the checker. It—unfortunately—cannot go into every detail. The full source code is included in Appendix B.1, and the accompanying documentation explains many aspects not covered in the main text.

5.1 DETECTING USE-AFTER-FREES: GENERAL CONSIDERATIONS

This section outlines the general capabilities of Coccinelle and Clang to detect use-after-free—and consequently double-free—issues. It thereby lays the foundation for the implementation of the `PostgresChecker`.

5.1.1 Coccinelle: Implementation by Stuart

Stuart [Stuo8] demonstrated that use-after-frees can be detected with Coccinelle. To this end, a script similar to the template shown in Listing 5.1 [Stuo8, p. 47] is used to find potential candidates. In Line 4, the script records the use of `free` on an expression `E` at position `p1`. The dot notation (`...`)

¹ This version is widely used and was also chosen by a similar project, `pg_ladybug` by Timescale, which uses Clang-Tidy to analyze PostgreSQL functions [Tim25e]. The implementation, including an install script, can be found in the fork at [Rei25h].

in Line 5 indicates any intervening code. In Line 6, the same expression is accessed again, constituting a use-after-free. The position of the use-after-free is stored in `p2`, and the results are printed using Python scripting (see also Section 4.2.2).

Listing 5.1: Coccinelle template to find use-after-free locations

```

1 @ bug exists @
2 expression E; position p1, p2;
3 @@
4 free@p1(E);
5 ...
6 E@p2
7 @ script:python @
8 e << bug.E; p1 << bug.p1; p2 << bug.p2
9 @@
10 /* OUTPUT LOGIC */

```

At first glance, this pattern seems straightforward. However, the template in Listing 5.1 also produces many false positives. For example, if `E` is re-assigned at any point between Line 4 and Line 6, the later usage would not be problematic. To address this, Stuart implemented custom data-flow analysis logic, which quickly increases the approach’s complexity. Beyond false positives, there are also inherent limitations to this approach:

- **Scope:** The pattern in Listing 5.1 works only within individual files and, within them, only inside single functions.
- **No aliasing:** The approach does not detect cases like `x = y; free(y); use(x);`, since there is no alias analysis. Handling such patterns would require additional data-flow analysis.

Addressing these points would require elaborate scripting, making the approach very time-consuming and error-prone.

5.1.2 Clang: The MallocChecker

The Clang Static Analyzer framework—as discussed in Section 3.3—offers greater flexibility. Its analysis scope covers the entire translation unit, and alias handling is straightforward because Clang uses memory regions to track symbolic values. The framework also includes a built-in checker for memory issues, the `MallocChecker` [Cla25l], which can detect, among other problems, use-after-free, double-free, use of uninitialized memory, and mismatched memory deallocations. Choosing this framework for implementing a custom checker is therefore the more flexible option, as it provides many tools that would otherwise need to be implemented from scratch in Coccinelle—namely, data-flow analysis and symbolic execution.

5.1.2.1 Code Annotations

The provided checks are modeled for standard C/C++ memory functions, but the checker is designed to be extensible. Through so-called code annotations,

developers can inform Clang which functions allocate or deallocate memory and how they relate to each other. This is done using the `__attribute()` syntax, which models memory allocation via the concept of *ownership*: allocating functions *return* ownership of memory, while deallocating functions *take* it. Using the annotations `ownership_takes()` and `ownership_returns()`, functions can be flagged as memory-handling.

Listing 5.2 shows how the `__attribute()` syntax can be used to annotate `pfree` as a memory deallocator [Cla25d]. It uses the `ownership_takes` annotation and specifies the allocation domain as `postgres`—indicating that it is part of PostgreSQL-specific memory-handling functions—as well as the position of the parameter being freed. With this information, the checker can report use-after-free issues, since it knows that `pfree` deallocates memory. It could also emit a warning if `pfree` is used on a variable that was not allocated by `palloc`, as `palloc` would likewise be assigned to the `postgres` domain.

Listing 5.2: Example of an annotation in Clang

```
1 void __attribute__((ownership_takes(postgres, 1))) pfree(void *);
```

5.1.2.2 Rationale for a Custom Checker

In this thesis, a checker independent of the `MallocChecker` and its annotations is developed for the following reasons:

- **Time constraints:** Annotating the PostgreSQL source code would require consensus and integration into the upstream project, which would be too time-consuming, making an independent solution the more practical choice for the purposes of this thesis.
- **Overreporting:** The `MallocChecker` also reports memory leaks, which is undesirable in the context of PostgreSQL, since memory leaks are not problematic when functions provided by the API are used.
- **Extensibility:** PostgreSQL-specific concepts cannot be represented adequately with existing checkers. Issues in Category 2 or 3, for instance, require more flexibility. Concepts like memory contexts, discussed in Section 4.1, also demand a similar yet distinct approach. The implementation developed here can serve as a foundation for addressing these cases in future work (see also Chapter 7).

Accordingly, a separate checker—the `PostgresChecker`—is implemented. While inspired by the conceptual foundation of the `MallocChecker`, it is adapted to address the specific categories discussed in the previous chapter. Both checkers are implemented in C++, the language used by the Static Analyzer framework.

5.2 CATEGORIES

This section describes how the `MallocChecker` was adapted and how the `PostgresChecker` addresses the challenges posed by each category from

Chapter 4. It introduces the supporting data structures used to represent program states and explains the implementation of the function catalog. Furthermore, it details how the Clang Static Analyzer framework detects use-after-free and double-free scenarios. For Categories 4 and 5, alternative solutions based on Coccinelle are also discussed.

5.2.1 C1: Strict Functions

The first category represents functions that always free passed parameters. Therefore, it addresses a “traditional” use-after-free problem. As shown in the previous section, Coccinelle can detect basic cases of this issue but suffers from significant limitations. This subsection therefore examines how the `MallocChecker` was adapted to handle these cases more effectively. Note that reallocating functions are not included here and are handled as Arbitrary Functions, as they do not strictly always free parameters.

5.2.1.1 Supporting Data Structures

STRICTMAP The checker needs to know which functions free specific parameters. To achieve this, the function catalog is implemented using a map called `StrictMap`, which stores function names as keys. The corresponding value represents the position of the freed parameter in the function signature. The checker uses the LLVM-specific data structure `llvm::StringMap<unsigned int>`. The `StrictMap` maps function names (strings) to parameter positions (unsigned integers). The definition of the map is shown in Listing 5.3. For example, in Line 2, the function `array_free_iterator` frees its first parameter, while `free_pool` in Line 4 frees its second parameter. When the checker encounters a function call, it looks up the function name in the map. If an entry exists, the checker verifies the relevant parameter position and handles the function call accordingly (see below). This map has been generated using the script in Appendix A.7² and was manually integrated into the checker.

Listing 5.3: Map storing strict function names and the position of the parameter

```

1  llvm::StringMap<unsigned int> StrictMap{
2    {"array_free_iterator"}, {0}},
3    ...
4    {"free_pool"}, {1}},
5    ...
6  };

```

PGREFSTATE When encountering a call to a freeing function, the checker needs to store information about that call. The `MallocChecker` accomplishes this using a class called `RefState`, which records the state of the freed reference. The `PostgresChecker` adopts this approach and defines the class `PGRefState`, which stores information about the statement where the function is called, the function declaration, and the original variable name.

² Internally, this uses the Coccinelle template from Appendix A.6.

REGIONSTATEPG Next, the checker needs a mechanism to associate memory regions with their `PGRefState`. The `REGISTER_MAP_WITH_PROGRAMSTATE` macro provided by Clang allows creating maps tied to the program state. These maps are available along all paths in the control-flow graph (CFG) originating from the current state. Using this mechanism, a map called `RegionStatePG` is registered, which associates symbols representing memory regions with their corresponding `PGRefState`. Whenever the checker encounters a freeing or reallocating function, it records the affected memory region in this map. Later, if the same memory region is accessed, the checker can look it up as a key in the map. If an entry exists, the region has been freed, and the checker emits a warning.

5.2.1.2 *Handling Function Calls*

With the necessary data structures in place, the next question becomes: *how* does the checker detect a function call in the first place? As discussed in Section 3.3.2, the Static Analyzer provides callbacks at specific points during the analysis of a function call. The `PostgresChecker` uses the `checkPreCall` callback,³ which always triggers before any function is executed.

The callback provides a `CallEvent`, from which the function name can be derived. The checker then distinguishes two cases: either the function exists in the `StrictMap`—in which case it handles the free—or the function does not exist in the `StrictMap` and the checker checks all parameters for potential use-after-frees. In the latter case, each parameter is looked up in the `RegionStatePG` map: if the map contains a state corresponding to the parameter’s memory region, the checker concludes that this region was freed earlier. To achieve this, the checker uses symbolic values provided by Clang’s symbolic execution engine (see also Section 3.3.2).

5.2.1.3 *Handling Frees*

A dedicated `HandleFree` function manages cases where frees are observed. Its main task is to add symbolic values to the `RegionStatePG` map. It interacts directly with the `CheckerContext`, creating a new `ProgramState` containing the updated map. If the symbolic value representing the memory region is already present in the map, the function reports a double-free.

Reporting both use-after-free and double-free cases is handled via a dedicated `emitReport` function. Two helper functions, `HandleUseAfterFree` and `HandleDoubleFree`, prepare the diagnostic messages passed to `emitReport`. These messages are of the form: “Attempt to use released memory: <variable>” or “Attempt to free released memory: <variable>”.

The `emitReport` function retrieves an error node as an `ExplodedNode`, referencing the current program point and state. The report is then emitted via the `CheckerContext`.

³ It would also be possible to use `checkPostCall`, but having access before the function call is particularly useful for Category 2.

Figure 5.2 shows the resulting output when the checker is applied to a file `test.c` containing a double-free bug. In this case, `list1` has been freed twice via `pfree`.

```
test.c:68:3: warning: Attempt to free released memory: list1 [postgres.PostgresChecker]
 68 |   pfree(list1);
    |   ~~~~~
test.c:67:3: note: Freeing function: pfree (list1)
 67 |   pfree(list1);
    |   ^
1 warning generated.
```

Figure 5.2: Resulting output of the PostgresChecker

5.2.1.4 Handling Variable Accesses

Up to this point, the checker only detects use-after-free errors in function calls. To also detect accesses to freed variables in arbitrary locations, the `checkLocation` callback is implemented. This callback triggers whenever a memory location is accessed; for example, in `y = x[0]`, it detects that `x` is being accessed. The checker then looks up the symbolic value corresponding to the accessed memory location in the `RegionStatePG` map and calls `HandleUseAfterFree` if necessary.

There is, however, an additional edge case: when a function returns a value that has already been freed. To address this, the `checkPreStmt` callback is implemented with the `ReturnStmt` template argument. This callback behaves similarly to `checkLocation` but includes extra logic to handle compound return types, such as arrays or structures.

Bringing all these components together, the signature of the `PostgresChecker` class, including all registered callback templates, is shown in Listing 5.4.

Listing 5.4: Signature of the PostgresChecker class

```
1 class PostgresChecker : public Checker<check::PreCall, check::Location,
2                               check::PreStmt<ReturnStmt>>;
```

5.2.2 C2: Dependent Functions

For this category, the checker must not only detect freeing functions and their passed parameters but also the conditions under which a free occurs. For example, if a parameter is freed only when `shouldFree == true`, the checker needs to recognize that condition. Coccinelle does not provide the means to achieve this. Clang, however, uses symbolic execution and can therefore extract concrete values from symbolic ones when they can be determined statically.

This section discusses how the basic `PostgresChecker` presented in Section 5.2.1 is extended to address this requirement, including the necessary data structures and function handling.

5.2.2.1 Supporting Data Structures

DEPENDENCYINFO To store information about dependent functions, the checker introduces a map serving a similar purpose to the `StrictMap`. However, for this category, the checker must store not only the position of the freed parameter but also information about the dependency. To this end, the structure `DependencyInfo` is introduced, as shown in Listing 5.5. It records both the position of the freed parameter and a function called `isFreeing` that returns a `Tristate`, which can be `True`, `False`, or `Undefined`.

Listing 5.5: The structure to store dependency-related information

```

1 struct DependencyInfo {
2     int positionFreed;
3     std::function<Tristate(CallEvent &, CheckerContext &)> isFreeing;
4     bool isArbitrary;
5     ...
6 };

```

This function is passed to the constructor and can be defined separately for each map entry. This allows the checker to implement custom logic for checking conditions such as `shouldFree == true`. However, it is not always possible to retrieve a concrete value from a symbolic one. For example, if a value is dynamically passed to a program, there is no way to determine its value statically. In such cases, `Undefined` is returned, and the function is treated as arbitrary. How arbitrary functions are handled will be discussed in the following section.

Even when a value is statically available, it is not always possible to determine with certainty whether a function frees a parameter. Consider the function `bms_int_members` in Listing 5.6 [Pos25q]: if `b == NULL`, the parameter `a` is always freed in Line 6. However, if `b` is not `NULL`, `a` will be freed in Line 12 depending on conditions that cannot be determined statically. Therefore, the function is either strict or arbitrary. The field `isArbitrary` in Listing 5.5 specifies whether a function should be treated as arbitrary when `isFreeing` returns `False`.

Listing 5.6: A PostgreSQL function being arbitrary when the dependent value is false

```

1 Bitmapset *
2 bms_int_members(Bitmapset *a, const Bitmapset *b) {
3     ...
4     if (b == NULL)
5     {
6         pfree(a);
7         return NULL;
8     }
9     ...
10    if (lastnonzero == -1)
11    {
12        pfree(a);
13        return NULL;
14    }
15    ...
16 }

```

DEPENDENTMAP To look up the `DependencyInfo` introduced above, a map called `DependentMap` is used, similar to the `StrictMap`. It also uses the data structure `llvm::StringMap`, but instead of mapping function names to parameter positions, it maps the function name to a `DependencyInfo`. Listing 5.7 shows an example of how the `DependencyInfo` structure can be used to implement the `DependentMap`. The example is explained below.

Listing 5.7: Example of an entry in the `DependentMap`

```

1  llvm::StringMap<DependencyInfo> DependentMap{
2    {"bms_int_members", DependencyInfo(0, true, [] (CallEvent &Call, CheckerContext &C){
3      SVal val = Call.getArgSVal(1);
4      if (val.isUnknownOrUndef()) return Undefined;
5      if (val.isZeroConstant()) return True;
6      return False;
7    }}},

```

EXAMPLE In Line 2, the entry for `bms_int_members`—used as an example earlier (see Listing 5.6)—is defined. The key of the map is the function name as a string, and the value is an object of type `DependencyInfo`. It is initialized by passing the relevant information to the constructor: the position of the freed parameter (0), whether the function becomes arbitrary when not freeing (`true`), and the `isFreeing` function. In this function, in Line 3, the argument on which the free depends—namely `b`—is retrieved. It then returns `Undefined` if the value is not defined, `True` if `b` is a zero constant, and `False` otherwise. For example, if the analyzed code calls `bms_int_members(a, NULL)`, the function would return `True`; if it calls `bms_int_members(a, b)`; and `b` is defined and not `NULL`, it would return `False`; otherwise, when the concrete value cannot be determined, it returns `Undefined`.

5.2.2.2 Handling Function Calls

When handling these maps in the callbacks discussed in Section 5.2.1.2 of Category 1, the checker now has to account for the existence of another category. To this end, an enum `Category` is introduced and passed to the handler functions. In the `checkPreCall` callback, the checker passes `Strict` when a function is inside the `StrictMap` and `Dependent` when it is in the `DependentMap`. In the latter case, the handler function retrieves the `DependencyInfo` `Info` and checks `Info.isFreeing()`. If it returns `True`, the function is handled as `Strict`; when `Undefined` is returned, it is handled as `Arbitrary` (see next section). If it returns `False`, the checker examines `Info.isArbitrary`: if this is true, the function is treated as `Arbitrary` as well; otherwise, the checker returns and does nothing.

5.2.3 C3: Arbitrary Functions

This category shares the same characteristics as Category 1 but needs to generate different warnings. To handle this category, the checker uses an `ArbitraryMap`, which is similar to the `StrictMap` and maps function names to the position of the freed parameter. An `Arbitrary` entry is added to the

enum introduced in Section 5.2.2.2, and the handler functions are extended to produce the appropriate warning messages (see Section 4.3.3).

5.2.4 C4: Reassigning Functions

Reassigning functions in Category 4 represent a problem that Coccinelle is particularly well-suited to solve: the return value of such a function should always be reassigned—typically to the parameter that is potentially freed. These cases can be expressed as a clear matching pattern. The first part of this section discusses how this issue was addressed by the TimescaleDB project using Coccinelle. The second part explains how the PostgresChecker needs to be extended to handle this category.

5.2.4.1 A Use Case for Coccinelle

Listing 5.8 illustrates how Coccinelle scripts are used in the TimescaleDB project [Tim25c] to handle the `bms_int_members` function discussed in Section 5.2.2.1. The script detects all occurrences where the function’s return value is not reassigned and replaces them with statements that assign the return value to the potentially freed parameter. Although this pattern actually modifies the code, it could easily be adapted to use Python scripting for reporting purposes only (see also Section 5.2.5). Moreover, the same approach can be applied to any function that falls into this category.

Listing 5.8: Coccinelle script to address Category 4-related issues

```

1 @rule_5@
2 expression E1, E2;
3 @@
4 + /* Result of bms_int_members has to be used */
5 + E1 = bms_int_members(E1, E2);
6 - bms_int_members(E1, E2);

```

5.2.4.2 Using the PostgresChecker

For the PostgresChecker, only minor changes are required to support this category. A `ReassigningSet`⁴ is introduced. When attaching the note to the report in the `emitReport` function, the checker issues the warning “The return value should probably be reassigned (<variable>)” instead of the standard one.⁵

5.2.5 C5: Functions Returning Boolean

Similar to Category 4, this problem can also be expressed as a pattern. Therefore, a solution using Coccinelle is presented first, followed by the required extensions to the PostgresChecker.

⁴ A map is not required, since the positional information is already available at the point where the report is emitted.

⁵ A Clang-Tidy version of this check has been implemented by Timescale [Tim25a].

5.2.5.1 Another Use Case for Coccinelle

For the function in this category (see Section 4.3.5), the return value should always be checked and—ideally—, if it is `false`, `NULL` should be assigned to the used variable. This can easily be handled using Coccinelle. The script in Listing 5.9 shows one possible approach in the form of a typical semantic patch. It detects whether the return value is checked, with `|` denoting alternative patterns; if not, it adds `if (!ret) param0 = NULL;`, where `ret` and `param0` are replaced by the actual variable names.

A complete implementation would also need to handle cases where `ret` is declared at the function call site, as well as cases where the return value is not assigned at all. The full patch script can be found in Appendix A.8.

If the goal is only to report problematic instances instead of modifying the code, the script must be slightly more elaborate. In a first rule, it has to identify all usages of the function; in a second rule, it matches cases where the return value is checked. The Python script would then receive all matches from the first rule, excluding those covered by the second one. An implementation for the basic use case, shown in Listing 5.9, can be found in Appendix A.9. This implementation would still need to be extended to cover cases involving call-site declarations and unassigned return values.

Listing 5.9: Coccinelle script to address incorrect usage of the function `ecpg_check_PQresult`

```

1 @rule@
2 expression param0, param1, param2, param3, ret;
3 statement S;
4 @@
5 ret = ecpq_check_PQresult(param0, param1, param2, param3);
6 (
7     if (ret) S
8     |
9     if (!ret) S
10    |
11    + if (!ret) param0 = NULL;
12    S
13 )

```

5.2.5.2 Using the PostgresChecker

For the `PostgresChecker`, the approach is similar to that of Category 4: introduce a `BooleanSet` containing the relevant function names. When such a function is encountered, the checker emits the note “The return value should probably be checked”.

5.3 SUMMARY

The Clang Static Analyzer framework proved to be a powerful tool for addressing the challenges posed by the categories in Chapter 4. At the same time, Coccinelle turned out to be well suited for certain pattern-based issues. The next chapter presents the findings and examines the validity and effectiveness of the `PostgresChecker`.

DISCUSSION AND EVALUATION

In this chapter, the results from Chapter 5 are discussed. The first section elaborates on the differences between Coccinelle and Clang and how well they address the issues in the categories identified in Chapter 4, thereby answering the research questions posed in Chapter 1. Additionally, the second section evaluates the implemented checker in real-world usage, discussing its validity and found bugs, the problems encountered when testing it against the PostgreSQL codebase, and potential approaches to address these issues.

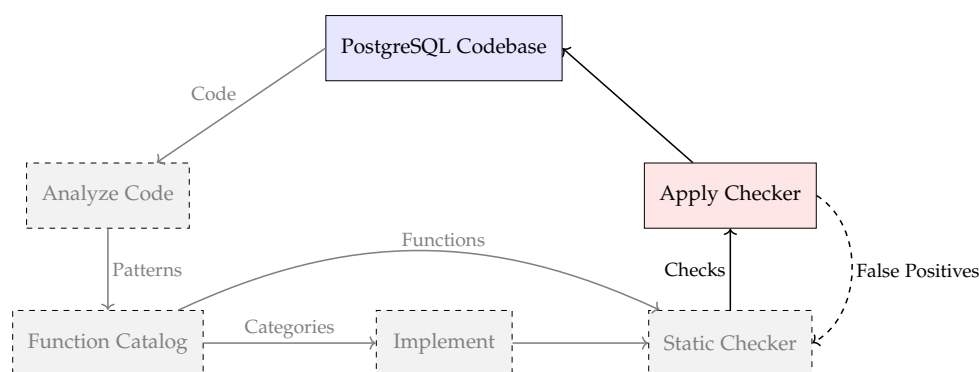


Figure 6.1: Phase 3 of developing the PostgresChecker

6.1 COMPARING COCCINELLE AND CLANG

When examining Categories 1 to 3, it becomes clear that Coccinelle is not well-suited to address the challenges posed by the functions within them. In Category 1, Coccinelle lacks the scope that Clang provides and it cannot handle aliasing adequately. Clang, by contrast, operates at the translation unit level, including everything visible to the compiler. Moreover, Clang’s symbolic execution enables it to handle aliasing correctly by using symbolic memory regions to determine whether two variables refer to the same memory. Most importantly, symbolic execution allows extracting concrete values from symbols, which is particularly valuable for addressing the challenges in Category 2. Therefore, basing the implementation of the checker on the Clang Static Analyzer framework was a justified decision.

However, implementing a static checker within this framework is far from trivial. The PostgresChecker consists of hundreds of lines of code and still lacks many features provided by the MallocChecker (see Chapter 7). Categories 4 and 5 were relatively simple to implement, but only because the foundational infrastructure was already in place. For purely pattern-based issues, Coccinelle is better suited, offering quick yet reliable solutions. Furthermore, Clang’s advantages come at the cost of performance. While this is

not critical when analyzing individual files, it can become significant when applied to large codebases.

Table 6.1 summarizes the differences between the two tools. The Clang Static Analyzer framework provides flexible solutions for a broad range of problems but requires a deep understanding of a highly specialized development environment. It is the right choice for solving intricate problems where analysis performance is not critical. Coccinelle, on the other hand, is a lightweight tool enabling fast and flexible solutions for very specific, pattern-based problems. Apart from minimal SmPL additions, its semantic patches closely resemble the original source code, making it particularly developer-friendly.

	Coccinelle	Clang
Strengths	Well-suited for pattern-based problems	Handles complex, generic problems
Limitations	Restricted to structural patterns	Requires deep framework knowledge
Performance	Lightweight and fast	Heavier and potentially slower

Table 6.1: Comparison of Coccinelle and Clang for static analysis

6.2 THE POSTGRESCHECKER

To verify the validity of the checker’s output, the tests in Appendix B.2 were used. These consist of relevant parts of the `MallocChecker` test suite, focusing on use-after-free and double-free cases, and were adapted as needed. Additional tests were created for the PostgreSQL-specific categories. These tests are designed to trigger the expected warning messages, as discussed in Section 4.3, and verify conditions where warnings should and should not be emitted. For example, calling the function `ExecForceStoreMinimalTuple` discussed in Section 4.3.2 once with `shouldFree == 1` and once with `shouldFree == 0` should either trigger or suppress a warning accordingly. The tests are executed using the LLVM Integrated Tester (LIT) [LLV25b], a tool provided by LLVM to run compiler test suites. LIT verifies whether expected warnings are shown during test execution. To enable LIT, the checker was developed in a fork of the LLVM repository [Rei25d] and was integrated into the repository mentioned in Chapter 5 after the tests completed successfully. Unfortunately, LIT can only be used when LLVM is built from source.

To evaluate the checker in a real scenario, the script in Appendix A.10 was used to analyze the PostgreSQL codebase, extensions in the `contrib` directory, and the Apache Age extension [Apa25a]. In total, 1,314 files were found in the `src` directory, of which 1,059 were successfully analyzed.¹ In the

¹ Some files failed due to missing optional headers in the PostgreSQL build (e.g., Windows-specific headers).

contrib directory 138 out of 161 files were analyzed successfully, and 49 out of 51 files in the Age repository.

Table 6.2 summarizes the results. In both PostgreSQL directories, the false-positive rate is estimated to be around 2%, while the checker found two true positives in the src directory. For the Age extension, no false positives were reported, but one true positive was identified.

Repository	Files Analyzed	False Positives	True Positives
PostgreSQL src	1,059 / 1,314	2%	2
PostgreSQL contrib	138 / 161	2%	0
Apache Age	49 / 51	0%	1

Table 6.2: Results of the experimental evaluation

The following sections discuss the findings from applying the checker in more depth, analyzing both true and false positives.

6.2.1 True Positives

The checker was able to find two true positives in the PostgreSQL src directory, both of which were confirmed. Both problems originate in the problematic use of local variables. Additionally, one potential bug, still requiring confirmation, was identified in the Apache Age extension.

6.2.1.1 Use-after-free in Partitioned Table Queries

In the function `expand_partitioned_rentry` [Pos25w], the first confirmed case of use-after-free was identified. It is part of the query planner and is used to expand partitioned tables. The problem arises in edge cases where the function handles already detached and dropped partitions—particularly when processing the last detached partition. Listing 6.1 shows the problematic pattern.

The partitioned table is passed to the function as `reinfo` in Line 2. In Line 4, the local variable `live_parts` is introduced. In Line 6, all partitions that potentially satisfy a `WHERE` condition are retrieved and stored as a `Bitmapset` both in the field `reinfo->live_parts` and in the local variable `live_parts`. The while-loop in Line 9 iterates over these partitions, retrieving the next member of the `Bitmapset` via the function `bms_next_member`. The function `try_table_open` then attempts to open the partition—the child relation—in Line 12 and stores the result in `childrel`. However, if this fails because the partition has been detached and dropped, `childrel` will be `NULL` and we enter the `if`-branch in Line 13. Here, the partition is removed from the set, and if the set becomes empty, `bms_del_member` internally frees the memory and returns `NULL`. The return value is reassigned to the field `reinfo->live_parts`, but not to the local variable `live_parts`. As a result, `live_parts` points to freed memory, making the condition check in Line 9 a case of use-after-free. A crash demonstrating this bug can be reproduced by following the steps in [Rei25e].

A patch was submitted to the `pgsql-hackers` mailing list and was accepted in Commit `da9f9f7` [Rei25b], which has been backported to Version 15.

Listing 6.1: Confirmed case of use-after-free detected by the PostgresChecker

```

1 static void
2 expand_partitioned_rtentry(..., RelOptInfo *relinfo,...)
3 {
4     Bitmapset *live_parts;
5     ...
6     relinfo->live_parts = live_parts = prune_append_rel_partitions(relinfo);
7     ...
8     i = -1;
9     while ((i = bms_next_member(live_parts, i)) >= 0)
10    {
11        ...
12        childrel = try_table_open(childOID, lockmode);
13        if (childrel == NULL)
14        {
15            relinfo->live_parts = bms_del_member(relinfo->live_parts, i);
16            continue;
17        }
18        ...
19    }
20    ...
21 }
```

6.2.1.2 Use-after-free During Parallel Vacuum

The function `parallel_vacuum_reset_dead_items` [Pos25aa] contains another case of use-after-free. This function destroys the shared store for tuple identifiers of dead items in the parallel vacuum process and creates a new one. Listing 6.2 illustrates the problematic pattern.

The parallel vacuum state `pvs` is passed to the function in Line 2, containing `pvs->dead_items`: the dead item store. In Line 4, a local variable `dead_items` is created, pointing to the same store. In Line 6, the old store is destroyed, and in Line 7 a new one is created. However, the old store is destroyed using the local variable introduced in Line 4, while the newly created one is assigned to `pvs->dead_items`. As a result, the local `dead_items` becomes dangling, making its use in `TidStoreGetDSA` in Line 10 a case of use-after-free. This issue was addressed in PostgreSQL Version 18 and backported to Version 17, with Commit `ccc8194` fixing the problem by removing the local variable altogether [Nay24].

Listing 6.2: Confirmed case of use-after-free detected by the PostgresChecker

```

1 void
2 parallel_vacuum_reset_dead_items(ParallelVacuumState *pvs, VacDeadItemsInfo **
3     dead_items_info_p)
4 {
5     TidStore *dead_items = pvs->dead_items;
6     ...
7     TidStoreDestroy(dead_items);
8     pvs->dead_items = TidStoreCreateShared(dead_items_info->max_bytes,
9         LWTRANCHE_PARALLEL_VACUUM_DSA);
10    ...
11    pvs->shared->dead_items_dsa_handle = dsa_get_handle(TidStoreGetDSA(dead_items));
```

```

11     ...
12 }

```

6.2.1.3 Possible Use-after-free in Age

In the function `age_match_two_vle_edges` [Apa25b] of the Apache Age extension, a possible case of use-after-free was detected by the checker. Listing 6.3 shows the relevant code. Here, `right_array` is assigned the same pointer as `right_path` (Line 2), which in turn points to `agt_arg_vpc` (Line 1). The pointer `agt_arg_vpc` may be freed by `PG_FREE_IF_COPY` in Line 4 if the argument was copied by PostgreSQL when derived from the function. In Line 6, `right_array` is accessed again, potentially leading to a use-after-free. This issue still needs to be confirmed.

Listing 6.3: Possible use-after-free case in the Age extension

```

1  right_path = (VLE_path_container *)agt_arg_vpc;
2  right_array = GET_GRAPHID_ARRAY_FROM_CONTAINER(right_path);
3
4  PG_FREE_IF_COPY(agt_arg_vpc, 1);
5
6  if (left_array[left_array_size - 1] != right_array[0])

```

6.2.2 False Positives

Out of the 1,059 files analyzed in the `src` directory, 22 produced warnings—about 2%. This closely matches the figure reported by Dyremark and Elfstrand for Clang-Tidy [DE25, p. 23]. Since Clang-Tidy is a widely adopted tool, this suggests that a false-positive rate of approximately 2% is generally acceptable to developers.

In the `contrib` directory, 3 out of 138 files produced warnings, again resulting in a 2% rate. As most of these warnings were false positives, this percentage approximates the overall failure rate of the checker. The following discussion highlights common causes of false positives and proposes improvements to address them.²

COUNTER-LIKE STRUCTURES Many false positives were caused by dependent functions that used a certain field of the passed parameter to determine whether memory is freed or not. In most cases, these represent counter-like functionality, such as the number of elements in a list. For example, the function `DecrTupleDescRefCount`, shown in Listing 4.5 in Section 4.1, frees a tuple descriptor only if `tdrefcount == 0`. This reference counter is handled internally, meaning the static checker does not see changes to it. Because the checker cannot determine the concrete value, it emits a *potential* error—producing false positives in a significant number of cases. To address this, the checker would need to simulate the increment and decrement behavior

² The full list of warnings can be found at [Rei25g] (PostgreSQL codebase) and [Rei25f] (`contrib` directory).

of such counters. To avoid the false positives related to this issue, functions following this pattern are currently ignored. The excluded functions are listed in the `DependentMap` (see Appendix B.1).

ERROR HANDLING Another frequent source of false positives is error handling. The function `ereport` supports multiple severity levels, such as `DEBUG`, `INFO`, `NOTICE`, `WARNING`, `ERROR`, `FATAL`, and `PANIC`, in ascending order of severity [Pos25j]. Calls with severity level `ERROR` or higher prevent the execution of subsequent code in the current scope. Certain functions, like `pg_fatal` [Pos25ab], display similar behavior.

Currently, the checker does not model this control flow. This limitation could be addressed by introducing *sink nodes* [Cla25a], which represent program points—specifically `ExplodedNodes`—where execution terminates. By using sink nodes, the checker could safely suppress warnings for code segments that are never reached.

LIST-RELATED FUNCTIONS False positives also occurred due to missing list-handling logic. Appending and removing items from lists is handled by specific functions, whose behavior—similar to counter-like structures—is not easily representable. The checker would have to implement more intricate logic simulating the behavior of certain list functions.

CONTROL-FLOW-RELATED ISSUES Control flow statements caused additional problems. When functions such as `ecpg_check_PQresult`, discussed in Category 5 (see Section 4.3.5), are used as conditionals—for instance, in `if`-statements—the checker struggles to distinguish between branches that free memory and branches that do not. A similar problem arises with reallocating functions like `ecpg_realloc` [Pos25u], which return `NULL` when memory allocation fails. These return values are often used in conditions, but the checker does not currently represent this behavior accurately. Modeling these cases would require improved handling of conditionals based on function return values.

OTHER ISSUES A peculiar issue was observed with the `exit_nicely` function [Pos25v]. Even though `exit` is called after freeing the variable, the checker still reports use-after-free warnings in subsequent code. This only occurred in the test directory and could not be reproduced elsewhere. The author was not able to determine the reason for this behavior.

Finally, standard C functions like `free` and `realloc` are currently ignored in double-free scenarios to avoid redundant warnings when used alongside the `MallocChecker`. This could be improved by allowing the user to pass options to Clang to enable or disable handling of standard C functions.

6.2.3 Suboptimal Coding Patterns

As a by-product, two false positives revealed coding patterns that are suboptimal. By avoiding these patterns and following the suggested improvements, code clarity and maintainability can be enhanced.

IF-STATEMENTS The function `get_index_paths` [Pos25x] is part of the query planner and builds index paths. The checker flagged the function as problematic because, in the section shown in Listing 6.4, the variable `ipath` is potentially freed by `add_path` in Line 3 but is used again in Line 6. However, the fields `amhasgettupple` and `amhasgetbitmap` denote different access methods for indexes that are mutually exclusive. Therefore, either the `if`-statement in Line 2 or the one in Line 5 is true, but never both for the same index. Therefore, this pattern is safe. Nevertheless, replacing the second `if` in Line 5 with an `else if` would make the intent clearer and safeguard against potential future changes to the index structure.

Listing 6.4: Suboptimal use of `if` statements detected due to false positive

```

1  ...
2  if (index->amhasgettupple)
3      add_path(rel, (Path *) ipath);
4
5  if (index->amhasgetbitmap &&
6      (ipath->path.pathkeys == NIL ||
7
8  ...

```

REASSIGNING PASSED PARAMETER In the `_bt_insertonpg` function, an index tuple `itup` is inserted on a particular page [Pos25r]. The function follows the pattern shown in Listing 6.5. In Lines 4 and 5, the pointer to the original tuple is stored in a local variable, and the passed identifier is reassigned to a copy. The copy is later freed in Line 7. The Coccinelle script in Section 4.2.2 mistakenly added this function to the overall freeing functions. If, instead of reassigning the passed identifier, a local variable like `itupcopy` were introduced, the code would be more transparent. This function was manually excluded from the checker.

Listing 6.5: Suboptimal use of local variable detected due to false positive

```

1  _bt_insertonpg(..., IndexTuple itup, ...)
2  {
3      ...
4      origitup = itup;
5      itup = CopyIndexTuple(origitup);
6      ...
7      pfree(itup);
8      ...
9  }

```

6.3 SUMMARY

Although the `PostgresChecker` can detect basic use-after-free and double-free issues and even help identify suboptimal coding patterns, incorporating the modifications discussed in this section could significantly improve its accuracy. Reducing the false-positive rate would, in turn, increase its acceptance among developers. After all, a checker should help developers improve their code—not make them question whether the checker itself is functioning correctly.

CONCLUSION

“山重水复疑无路，柳暗花明又一村”

—谚语

“After endless mountains and rivers that leave doubt whether there is a path forward, suddenly one encounters the shade of a willow, bright flowers, and a lovely village.”

—Chinese proverb

The PostgreSQL environment is a wonderfully intricate field. In addressing problems that may arise from using functions provided by the PostgreSQL API, solutions based on static analysis were explored and successfully applied. The Clang Static Analyzer framework proved to be a solid foundation for tackling PostgreSQL-specific issues related to memory management, while Coccinelle offered effective and straightforward solutions for pattern-based problems.

By addressing the five categories of functions identified in Chapter 4, the implemented Clang-based checker successfully detected relevant problems with an acceptable false-positive rate of around 2%. When applied to both the PostgreSQL codebase and its extensions, the checker uncovered real-world issues affecting production software and even resulted in a contributed patch.

Nevertheless, the checker’s application uncovered additional PostgreSQL-specific intricacies. Several potential improvements were discussed in Chapter 6, such as modeling list-handling functions and reference counters. Furthermore, the checker could benefit from more systematic testing—most notably with regard to false positives—to better understand its strengths and weaknesses, thereby guiding further refinements.

Beyond these improvements, there is also considerable potential to extend the current implementation to cover additional PostgreSQL-specific patterns as well as more general concerns.

7.1 EXTENDING THE CHECKER: POSTGRESQL-SPECIFIC ISSUES

FREEING OF FIELDS In many PostgreSQL API functions, not only memory parameters point to is manipulated, but also memory related to fields of parameters. The checker could be extended to include the freeing of these field values. This would involve detecting functions that receive `param` as a parameter and internally `free param->field`. Currently, only the freeing of the parameters themselves is considered. Identifying such functions—potentially using Coccinelle scripts—and modeling them in the checker would be a logical next step.

MEMORY CONTEXTS A concept very specific to PostgreSQL the checker could model is memory contexts (see Section 4.1). Any pointers to memory associated with a memory context become dangling once that context ends. An analyzer would therefore need to track variables tied to specific contexts to detect potential use-after-free issues. Additionally, every memory context must eventually be properly terminated—failing to do so can lead to memory leaks. Thus, any function that opens a new memory context should ensure it ends that same context before returning.

CHECK SPI RETURN CODE To allow the execution of SQL commands directly from C code, PostgreSQL provides the Server Programming Interface (SPI) [Pos25ad]. A call to `SPI_execute` will “[p]arse, plan, and execute a query string” [Pos25k]. It returns an error code that should always be checked.

LIST LENGTH CHECK The function `list_length` fetches the length of the passed list [Pos25z]. However, empty lists are always represented by null pointers in PostgreSQL [Pos25ac]. Therefore, a `list == NIL` check is preferable to `list_length(list) > 0`, as it saves a few CPU cycles.

CAST CONST `const` values in C are typically immutable. However, this immutability can be bypassed by casting, for example, `char *immutVar = (char *) constVar`. This constitutes an inherently unsafe practice because, among other issues, type safety can no longer be guaranteed. PostgreSQL provides a dedicated macro called `unconstify` [Pos25ae], which should always be used. It ensures the cast does not change the underlying data type.

POLYMORPHISM PostgreSQL provides polymorphism for some internal data types. A `Node`, for example, has a specific tag [Pos25g], which allows abstract `Node` objects to be cast to a concrete subtype. To guarantee type safety, the function `castNode` [Pos25s] should always be used.

TRANSACTION ID HANDLING Transaction identifiers (TransID) in PostgreSQL are represented by 32 bits [Pos25l]. To avoid running out of values, PostgreSQL allows overflows and implements custom functions [Pos25m] for comparing TransIDs using cyclic rather than linear logic. For example, developers should always use `TransactionIdPrecedes(transId1, transId2)` instead of `transId1 < transId2`.

SYSTEM ARCHITECTURE-DEPENDENT BIGINT PostgreSQL uses an abstraction called `Datum` [Pos25b] to represent values and other data types. “A *Datum* contains either a value of a pass-by-value type or a pointer to a value of a pass-by-reference type.” [Pos25c] This abstraction allows large values to be handled uniformly across different architectures. For example, a `BIGINT` cannot be passed by value on 32-bit architectures. By using a pointer-based abstraction, PostgreSQL unifies the handling of data types. However, this also means that casting a `Datum` directly to `BIGINT` is unsafe. Instead, the macro `PG_GETARG_INT64` [Pos25h] should be used. This macro internally calls `DatumGetInt64` [Pos25d], which checks whether the flag `USE_FLOAT8_BYVAL` is

set. If so, the integer is returned by value; otherwise, it is returned as a pointer. Simply casting the value could result in problems on 32-bit architectures.

7.2 EXTENDING THE CHECKER: ISSUES INDEPENDENT OF POSTGRESQL

USABILITY Beyond PostgreSQL-specific issues, in terms of usability, several improvements are possible. Currently, the checker does not use the `PathSensitiveBugReport`. Investigating whether this feature could produce more informative diagnostics would be worthwhile. Additionally, the checker currently emits too many warnings for issues originating from the same source; limiting output to the first occurrence would improve diagnostic quality. Integration with `clangd` [Cla25m] could also enhance usability by providing in-editor feedback for developers.

MEMORY-RELATED CHECKS The `MallocChecker` implements analyses that could be adapted for the `PostgresChecker`. These include taint analysis, zero-sized allocations, and dereferencing of non-dynamically allocated memory. The current checker does not yet model cases where memory is reallocated with a size of 0. Additionally, the `MallocChecker` ensures that memory is freed using the appropriate function, which could be adapted to ensure that memory allocated with `palloc` is freed with `pfree`.

NEW RESEARCH AVENUES A checker could also explore entirely new avenues. Fülöp and Pataki combine the idea of linear temporal logic with Clang and discuss possible mappings to the imperative layout of the C++ programming language used by Clang [FP24]. Droste, Kuhn, and Ludwig introduce MPI-specific problems relating to deadlocks within the context of PostgreSQL [DKL15]. Since the Clang Static Analyzer already provides an MPI-specific checker (see Section 2.2.2), there is potential for synergies. In this regard, Clang’s capability for cross-translation-unit analysis [Cla25j] would be a very promising topic of research. Another interesting direction would be exploring the applicability of formal static analysis to PostgreSQL-specific problems, as there are numerous formal static checkers available.

GOING DYNAMIC Lastly, beyond static analysis, dynamic tools like Valgrind [Val25] or ASan [Goo25] could also be explored. These tools may detect issues posed by arbitrary functions that are not statically detectable, effectively promoting these functions to dependent ones.

In summary, while developing a robust PostgreSQL-specific static analyzer presents technical challenges, it also opens up a broad and rewarding field for further research and practical improvements. By integrating insights from existing checkers, exploring advanced analysis techniques, and potentially incorporating dynamic methods, future work can significantly enhance the precision, usability, and scope of the `PostgresChecker`—contributing not only to the reliability of PostgreSQL itself but also to the broader field of static analysis. There are many promising paths ahead—though potentially challenging, they are well worth the endeavor.

APPENDIX



SCRIPTS AND TEMPLATES

A.1 RECURSIVE BASH SCRIPT

Listing A.1: Bash script for recursing over the code base

```
1  #!/bin/bash
2
3  #THIS SCRIPT CREATES COCCI SCRIPTS THAT RECURSIVELY SEARCH THE
4  #PROVIDED CODE BASE FOR PATTERNS AND ACCEPTS THE FOLLOWING PARAMETERS:
5  #
6  # $1: THE CODE BASE
7  # $2: THE INTIAL FUNCTION TO BE EXAMINED
8  # $3: THE TEMPLATE TO BE USED
9  #
10 #RESULTS WILL BE SAVED IN THE DEDICATED DIRECTORY DEFINED BELOW
11 #FOR EACH ITERATION A SEPARATE SUB DIRECTORY IS CREATED
12
13 if [ $# -lt 1 ]; then
14     echo "Please pass a directory or file name to be processed."
15     exit 0
16 fi
17
18
19 #DEFINE PATHS AND PARAMETERS
20 ITERATION=0
21 CODE_BASE=$1
22 FUNCTION=$2
23 TEMPLATE=$3
24 RESULTS_FILE_NAME=results.out
25 IT_PREFIX=iteration_
26 RESULTS=../results
27 RESULTS_PATH=$RESULTS/$FUNCTION$TEMPLATE
28 RESULTS_FILE=$RESULTS_PATH/$RESULTS_FILE_NAME
29 TMP_PATH=tmp
30 PRINTED_LINES=2
31 MANUALLY_ADDED=exceptions/$FUNCTION$TEMPLATE.add
32 MANUALLY_EXCLUDED=exceptions/$FUNCTION$TEMPLATE.exclude
33 MANUALLY_EXCLUDED_ALL=exceptions/${FUNCTION}all.exclude
34 SCRIPTS_CREATED=0
35
36 if [ -d $TMP_PATH ]; then
37     rm -rf $TMP_PATH
38 fi
39
40 if [ -d $RESULTS_PATH ]; then
41     echo "Results directory already exists."
42     exit 0
43 fi
44
45 #CREATE DIRECTORIES
46 mkdir $RESULTS_PATH
47 mkdir $TMP_PATH
48
49 #FUNCTION THAT CREATES A COCCI SCRIPT THAT CHECKS FOR ALL FUNCTION/TYPE
50 #NAMES PASSED TO IT IN THE TMP_PATH AS $ITERATION.cocci
51 createScripts(){
52     #COUNTER ADDED TO FUNCTION NAMES IN RULES TO BE UNAMBIGUOUS
```

```

53 counter=0
54
55 #ITERATE OVER FUNCTIONS
56 for arg in "$@"
57 do
58
59 #RETRIEVE AND SANITIZE FUNCTION AND TYPE NAMES
60 fname=$(echo "$arg" | cut -d',' -f1)
61 ftype=$(echo "$arg" | cut -d',' -f2)
62
63 #PREVENT TRYING TO ACCESS FIELDS OF char* AND char**
64 if [[ "$ftype" == "char*" && "$TEMPLATE" == "dependent" ]]; then
65     continue;
66 fi
67 if [[ "$ftype" == "char**" && "$TEMPLATE" == "dependent" ]]; then
68     continue;
69 fi
70
71 #IGNORE 'UNFREEABLE' TYPES -> THESE ARE ONLY ADDED IN REALLOC
72 if [ "$ftype" = "constint" ]; then
73     continue
74 fi
75 if [ "$ftype" = "long" ]; then
76     continue
77 fi
78 if [ "$ftype" = "size_t" ]; then
79     continue
80 fi
81 if [ "$ftype" = "yy_size_t" ]; then
82     continue
83 fi
84 if [ "$ftype" = "yy_scan_t" ]; then
85     continue
86 fi
87 if [ "$ftype" = "Size" ]; then
88     continue
89 fi
90
91 #EXPAND COMPRESSED TYPES
92 if [[ "$ftype" == "struct*" ]]; then
93     ftype=$(echo $ftype | sed "s/struct/struct /")
94 fi
95 if [[ "$ftype" == "*" ]]; then
96     ftype=$(echo "$ftype" | sed 's/*/*/')
97 fi
98
99 #PASS WILD CARD TO PYTHON SCRIPT IN THE COCCI FILE IF
100 #TYPE='void *', TYPE OTHERWISE
101 pythonType=$ftype
102 if [ "$ftype" = "void *" ]; then
103     pythonType="{t}"
104     ftype="t"
105 else
106     ftype="\${ftype}"
107 fi
108
109 #GET THE TEMPLATE -> THE IF CONDITION CHECKS, WHETHER
110 #THERE IS A FUNCTION SPECIFIC TEMPLATE (I.E. FOR SIGNATURE)
111 TEMPLATE_FILE=templates/template$TEMPLATE.cocci
112 if [ -f templates/template$FUNCTION$TEMPLATE.cocci ]; then
113     TEMPLATE_FILE=templates/template$FUNCTION$TEMPLATE.cocci
114 fi
115
116 #REPLACE THE PLACEHOLDERS WITH FUNCTION/TYPE NAMES
117 cat $TEMPLATE_FILE \
118 | sed -e "s/___FUNCTION___/$fname/g" \

```

```

119         -e "s/___CHECKTYPE___/$ftype/g" \
120         -e "s/___PRINTTYPE___/$pythonType/g" \
121         >> $TMP_PATH/$ITERATION$counter.cocci
122
123     counter=$((counter+1))
124     done
125     SCRIPTS_CREATED=$counter
126 }
127
128 #CREATE THE RESULTS FILE IF IT DOES NOT EXIST
129 if [ ! -f $RESULTS_FILE ]; then
130     touch $RESULTS_FILE
131 fi
132
133 #WE KEEP TRACK OF FUNCTIONS FOUND IN THE PREVIOUS ITERATION IN
134 #THE FILE 'functionsLastIteration'
135 #PASS INITIAL FUNCTION NAMES FOR 'ITERATION -1'
136 if [[ "$TEMPLATE" == "dependent" || "$TEMPLATE" == "ereport" || "$TEMPLATE" == "
    double" || "$TEMPLATE" == "static" ]]; then
137     grep "^>" $RESULTS/$FUNCTION/$RESULTS_FILE_NAME | sort | uniq >> $TMP_PATH/
        functionsLastIteration
138 fi
139
140 echo ">$FUNCTION,void *" >> $TMP_PATH/functionsLastIteration
141
142 #RETRIEVE FUNCTIONS TO MANUALLY ADD IF FILE EXISTS
143 if [ -f $MANUALLY_ADDED ]; then
144     if [[ "$TEMPLATE" == "dependent" ]]; then
145         cat $MANUALLY_ADDED | cut -d ',' -f 1,2,3 | sort | uniq > added.tmp
146         grep -A $PRINTED_LINES -F -f added.tmp $RESULTS/$FUNCTION/$RESULTS_FILE_NAME >>
            $RESULTS_FILE
147         awk -F ',' 'NR==FNR {key = $1 FS $2; map[key] = $0; next} {print (map[$0] ? map
            [$0] : $0)}' $MANUALLY_ADDED $RESULTS_FILE > out.tmp
148         mv out.tmp $RESULTS_FILE
149     else
150         grep -A $PRINTED_LINES -F -f $MANUALLY_ADDED $RESULTS/$FUNCTION/
            $RESULTS_FILE_NAME >> $RESULTS_FILE;
151     fi
152 fi
153
154 #LOGGING
155 echo "STARTING -> $CODE_BASE"
156 START_OVERALL=$(date +%s)
157
158 #KEEP ITERATING WHILE FILE functionsLastIteration EXISTS
159 while [ -f $TMP_PATH/functionsLastIteration ]; do
160
161     #WE'VE ALREADY LOOKED AT ALL FUNCTIONS THERE ARE
162     if [[ "$TEMPLATE" == "dependent" || "$TEMPLATE" == "static" ]]; then
163         if [ $ITERATION -gt 0 ]; then
164             break
165         fi
166     fi
167
168     #LOGGING
169     START=$(date +%s)
170     echo "##### ITERATION $ITERATION #####"
171
172     #PATH AND FILE FOR RESULTS OF ITERATION
173     IT_RESULTS_PATH=$RESULTS_PATH/$IT_PREFIX$ITERATION
174     IT_RESULTS_FILE=$IT_RESULTS_PATH/$RESULTS_FILE_NAME
175     mkdir -p $IT_RESULTS_PATH
176
177     #LOG FUNCTIONS EXAMINED IN THIS ITERATION
178     cp $TMP_PATH/functionsLastIteration $IT_RESULTS_PATH/functionsExamined.txt
179

```

```

180 #GET THE FUNCTION NAMES TO EXAMINE AND REMOVE functionsLastIteration
181 functionNames=$(grep "^>" $TMP_PATH/functionsLastIteration | cut -c2- | tr -d ' '
    | tr '\n' ' ')
182 rm $TMP_PATH/functionsLastIteration
183
184 #CREATE THE COCCINELLE SCRIPTS (SPLIT INTO SINGLE FILES TO AVOID STACK OVERFLOW)
185 createScripts $functionNames
186
187 #RUN SCRIPTS
188 for i in $(seq 0 $((SCRIPTS_CREATED-1)));
189 do
190     echo "### Function $((i+1)) of $SCRIPTS_CREATED ($FUNCTION $TEMPLATE iteration
        $ITERATION) ###"
191     spatch --sp-file $TMP_PATH/$ITERATION$i.cocci $CODE_BASE >> $IT_RESULTS_FILE
192 done
193
194 #GET ALL FUNCTION NAMES FROM THE RESULTS
195 grep '>' $IT_RESULTS_FILE | while IFS= read -r line; do
196
197     #GET FUNCTION WITHOUT EXACT POSITION IN THE FILE
198     line_file_only=$(echo $line | cut -d ':' -f 1)
199     #EXCLUDE FUNCTIONS TO BE EXCLUDED
200     if [ -f $MANUALLY_EXCLUDED ]; then
201         if grep -Fxq "${line_file_only}" "$MANUALLY_EXCLUDED"; then
202             continue;
203         fi
204     fi
205     #EXCLUDE FUNCTIONS TO BE EXCLUDED FOR ALL TEMPLATES
206     if [ -f $MANUALLY_EXCLUDED_ALL ]; then
207         if grep -Fxq "${line_file_only}" "$MANUALLY_EXCLUDED_ALL"; then
208             continue;
209         fi
210     fi
211
212     #IF THEY HAVE NOT BEEN FOUND BEFORE, ADD THEM TO THE OVERALL RESULTS
213     #AND EXAMINE THE FUNCTION WITH COCCINELLE IN THE NEXT ITERATION
214     if ! grep -Fxq "$line" $RESULTS_FILE; then
215         grep -Fx -A $PRINTED_LINES "$line" $IT_RESULTS_FILE >> $RESULTS_FILE;
216         echo "$line" >> $TMP_PATH/functionsLastIteration;
217     fi;
218 done
219
220 ITERATION=$((ITERATION+1))
221
222 #LOGGING
223 END=$(date +%s)
224 ELAPSED=$(( END - START ))
225 echo "#####"
226 echo "TIME PASSED: ${ELAPSED} SECONDS"
227 done
228
229 #LOGGING
230 END_OVERALL=$(date +%s)
231 ELAPSED_OVERALL=$(( END_OVERALL - START_OVERALL ))
232 ELAPSED_MINUTES=$(( ELAPSED_OVERALL / 60 ))
233 ELAPSED_SECONDS_REMAINING=$(( ELAPSED_OVERALL - ELAPSED_MINUTES * 60 ))
234
235 echo "#####"
236 echo "DONE"
237 echo "NUMBER OF ITERATIONS: $ITERATION"
238 echo "TIME PASSED OVERALL: $ELAPSED_MINUTES MINUTES $ELAPSED_SECONDS_REMAINING
    SECONDS"
239
240 #CLEAN UP
241 rm -rf tmp

```

A.2 STRICT FUNCTIONS COCCINELLE TEMPLATE

Listing A.2: Coccinelle template for Strict Functions

```

1  @freeing0 forall@
2  type rt, t;
3  identifier f, i;
4  position p;
5
6  @@
7  rt f(..., t i, ...) {
8      ... when != if(...){ ... return ...;}
9      __FUNCTION__@p(..., i, ...)
10     <... when any
11     __FUNCTION__(..., i, ...)
12     ...>
13 }
14
15 @script:python@
16 f << freeing0.f;
17 p << freeing0.p;
18 t << freeing0.t;
19 rt << freeing0.rt;
20
21 @@
22 if t == __CHECKTYPE__:
23     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
24     print(f"Used: __FUNCTION__")
25     print(f"Ret. Type: {rt}")
26
27 @freeing1 forall@
28 type rt, t;
29 identifier f, i;
30 position p;
31
32 @@
33 rt f(..., t i, ...) {
34     ...
35     if (i == NULL) {... return ...;}
36     ... when != if(...){ ... return ...;}
37     __FUNCTION__@p(..., i, ...)
38     <... when any
39     __FUNCTION__(..., i, ...)
40     ...>
41 }
42
43 @script:python@
44 f << freeing1.f;
45 p << freeing1.p;
46 t << freeing1.t;
47 rt << freeing1.rt;
48
49 @@
50 if t == __CHECKTYPE__:
51     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
52     print(f"Used: __FUNCTION__")
53     print(f"Ret. Type: {rt}")
54
55 @freeing2 forall@
56 type rt, t;
57 identifier f, i;
58 position p;
59
60 @@

```

```

61 rt f(..., t i, ...) {
62   ... when != if(...){ ... return ...;}
63   if (i){...
64     __FUNCTION__@p(..., i, ...)
65     ...}
66   <... when any
67     __FUNCTION__(..., i, ...)
68     ...>
69 }
70
71 @script:python@
72 f << freeing2.f;
73 p << freeing2.p;
74 t << freeing2.t;
75 rt << freeing2.rt;
76
77 @@
78 if t == __CHECKTYPE__:
79   print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
80   print(f"Used: __FUNCTION__")
81   print(f"Ret. Type: {rt}")
82
83 @freeing3 forall@
84 type rt, t;
85 identifier f, i;
86 position p;
87
88 @@
89 rt f(..., t i, ...) {
90   ...
91   if (i == NULL) {... return ...;}
92   ... when != if(...){ ... return ...;}
93   if (i){...
94     __FUNCTION__@p(..., i, ...)
95     ...}
96   <... when any
97     __FUNCTION__(..., i, ...)
98     ...>
99 }
100
101 @script:python@
102 f << freeing3.f;
103 p << freeing3.p;
104 t << freeing3.t;
105 rt << freeing3.rt;
106
107 @@
108 if t == __CHECKTYPE__:
109   print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
110   print(f"Used: __FUNCTION__")
111   print(f"Ret. Type: {rt}")
112
113 @freeing4 forall@
114 type rt, t;
115 identifier f, i;
116 position p;
117
118 @@
119 rt f(..., t i, ...) {
120   ... when != if(...){ ... return ...;}
121   while (i){...
122     __FUNCTION__@p(..., i, ...)
123     ...}
124   <... when any
125     __FUNCTION__(..., i, ...)
126     ...>

```

```

127 }
128
129 @script:python@
130 f << freeing4.f;
131 p << freeing4.p;
132 t << freeing4.t;
133 rt << freeing4.rt;
134
135 @@
136 if t == __CHECKTYPE__:
137     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
138     print(f"Used: __FUNCTION__")
139     print(f"Ret. Type: {rt}")
140
141 @freeing5 forall@
142 type rt, t;
143 identifier f, i;
144 position p;
145
146 @@
147 rt f(..., t i, ...) {
148     ...
149     if (i == NULL) {... return ...;}
150     ... when != if(...){ ... return ...;}
151     while (i){...
152     __FUNCTION__@p(..., i, ...)
153     ...}
154     <... when any
155     __FUNCTION__(..., i, ...)
156     ...>
157 }
158
159 @script:python@
160 f << freeing5.f;
161 p << freeing5.p;
162 t << freeing5.t;
163 rt << freeing5.rt;
164
165 @@
166 if t == __CHECKTYPE__:
167     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
168     print(f"Used: __FUNCTION__")
169     print(f"Ret. Type: {rt}")
170
171 @freeing6 forall@
172 type rt, t;
173 identifier f, i;
174 position p;
175
176 @@
177 rt f(..., t i, ...) {
178     ...
179     if (i == NIL) {... return ...;}
180     ... when != if(...){ ... return ...;}
181     __FUNCTION__@p(..., i, ...)
182     <... when any
183     __FUNCTION__(..., i, ...)
184     ...>
185 }
186
187 @script:python@
188 f << freeing6.f;
189 p << freeing6.p;
190 t << freeing6.t;
191 rt << freeing6.rt;
192

```

```

193 @@
194 if t == __CHECKTYPE__:
195     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
196     print(f"Used: __FUNCTION__")
197     print(f"Ret. Type: {rt}")
198
199 @freeing7 forall@
200 type rt, t;
201 identifier f, i;
202 position p;
203
204 @@
205 rt f(..., t i, ...) {
206     ...
207     if (i == NIL) {... return ...;}
208     ... when != if(...){ ... return ...;}
209     if (i){...
210     __FUNCTION__@p(..., i, ...)
211     ...}
212     <... when any
213     __FUNCTION__(..., i, ...)
214     ...>
215 }
216
217 @script:python@
218 f << freeing7.f;
219 p << freeing7.p;
220 t << freeing7.t;
221 rt << freeing7.rt;
222
223 @@
224 if t == __CHECKTYPE__:
225     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
226     print(f"Used: __FUNCTION__")
227     print(f"Ret. Type: {rt}")
228
229 @freeing8 forall@
230 type rt, t;
231 identifier f, i;
232 position p;
233
234 @@
235 rt f(..., t i, ...) {
236     ...
237     if (i == NIL) {... return ...;}
238     ... when != if(...){ ... return ...;}
239     while (i){...
240     __FUNCTION__@p(..., i, ...)
241     ...}
242     <... when any
243     __FUNCTION__(..., i, ...)
244     ...>
245 }
246
247 @script:python@
248 f << freeing8.f;
249 p << freeing8.p;
250 t << freeing8.t;
251 rt << freeing8.rt;
252
253 @@
254 if t == __CHECKTYPE__:
255     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
256     print(f"Used: __FUNCTION__")
257     print(f"Ret. Type: {rt}")

```

A.3 DEPENDENT FUNCTIONS COCCINELLE TEMPLATE

Listing A.3: Coccinelle template for Dependent Functions

```

1  @freeing0 exists@
2  type rt, t1, t2;
3  identifier f, i, b;
4  position p;
5
6  @@
7  rt f(..., t1 i, ..., t2 b, ...) {
8    <+...
9    if (b) {... __FUNCTION__@p(..., i, ...) ...}
10   ...+>
11 }
12
13 @script:python@
14 f << freeing0.f;
15 p << freeing0.p;
16 t << freeing0.t1;
17 rt << freeing0.rt;
18 b << freeing0.b;
19
20 @@
21 if t == __CHECKTYPE__:
22     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + f
23           ",{b}")
24     print(f"Used: __FUNCTION__")
25     print(f"Ret. Type: {rt}")
26
27 @freeing1 exists@
28 type rt, t1, t2;
29 identifier f, i, b;
30 position p;
31
32 @@
33 rt f(..., t2 b, ..., t1 i, ...) {
34   <+...
35   if (b) {... __FUNCTION__@p(..., i, ...) ...}
36   ...+>
37 }
38
39 @script:python@
40 f << freeing1.f;
41 p << freeing1.p;
42 t << freeing1.t1;
43 rt << freeing1.rt;
44 b << freeing1.b;
45
46 @@
47 if t == __CHECKTYPE__:
48     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + f
49           ",{b}")
50     print(f"Used: __FUNCTION__")
51     print(f"Ret. Type: {rt}")
52
53 @freeing2 exists@
54 type rt, t1, t2;
55 identifier f, i, b, a;
56 position p;
57
58 @@
59 rt f(..., t1 i, ..., t2 b, ...) {
60   <+...

```

```

59   if (b->a) {... __FUNCTION__@p(..., i, ...) ...}
60   ...+>
61 }
62
63 @script:python@
64 f << freeing2.f;
65 p << freeing2.p;
66 t << freeing2.tl;
67 rt << freeing2.rt;
68 b << freeing2.b;
69 a << freeing2.a;
70
71 @@
72 if t == __CHECKTYPE__:
73     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + F
74           ",{b}->{a}")
75     print(f"Used: __FUNCTION__")
76     print(f"Ret. Type: {rt}")
77
78 @freeing3 exists@
79 type rt, t1, t2;
80 identifier f, i, b, a;
81 position p;
82
83 @@
84 rt f(..., t2 b, ..., t1 i, ...) {
85     <+...
86     if (b->a) {... __FUNCTION__@p(..., i, ...) ...}
87     ...+>
88 }
89
90 @script:python@
91 f << freeing3.f;
92 p << freeing3.p;
93 t << freeing3.tl;
94 rt << freeing3.rt;
95 b << freeing3.b;
96 a << freeing3.a;
97
98 @@
99 if t == __CHECKTYPE__:
100     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + f
101           ",{b}->{a}")
102     print(f"Used: __FUNCTION__")
103     print(f"Ret. Type: {rt}")
104
105 @freeing4 exists@
106 type rt, t1, t2;
107 identifier f, i, b;
108 position p;
109
110 @@
111 rt f(..., t1 i, ..., t2 b, ...) {
112     <+...
113     if (b) {... __FUNCTION__@p(..., i, ...) ...}
114     else{...}
115     ...+>
116 }
117
118 @script:python@
119 f << freeing4.f;
120 p << freeing4.p;
121 t << freeing4.tl;
122 rt << freeing4.rt;
123 b << freeing4.b;

```

```

123 @@
124 if t == __CHECKTYPE__:
125     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + f
126           ",{b}")
127     print(f"Used: __FUNCTION__")
128     print(f"Ret. Type: {rt}")
129
130 @freeing7 exists@
131 type rt, t1, t2;
132 identifieur f, i, b;
133 position p;
134
135 @@
136 rt f(..., t2 b, ..., t1 i, ...) {
137     <+...
138     if (b) {... __FUNCTION__@p(..., i, ...) ...}
139     else{...}
140     ...+>
141 }
142
143 @script:python@
144 f << freeing7.f;
145 p << freeing7.p;
146 t << freeing7.t1;
147 rt << freeing7.rt;
148 b << freeing7.b;
149
150 @@
151 if t == __CHECKTYPE__:
152     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + f
153           ",{b}")
154     print(f"Used: __FUNCTION__")
155     print(f"Ret. Type: {rt}")
156
157 @freeing8 exists@
158 type rt, t1, t2;
159 identifieur f, i, b, a;
160 position p;
161
162 @@
163 rt f(..., t1 i, ..., t2 b, ...) {
164     <+...
165     if (b->a) {... __FUNCTION__@p(..., i, ...) ...}
166     else{...}
167     ...+>
168 }
169
170 @script:python@
171 f << freeing8.f;
172 p << freeing8.p;
173 t << freeing8.t1;
174 rt << freeing8.rt;
175 b << freeing8.b;
176 a << freeing8.a;
177
178 @@
179 if t == __CHECKTYPE__:
180     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + f
181           ",{b}->{a}")
182     print(f"Used: __FUNCTION__")
183     print(f"Ret. Type: {rt}")
184
185 @freeing9 exists@
186 type rt, t1, t2;
187 identifieur f, i, b, a;
188 position p;

```

```

186
187 @@
188 rt f(..., t2 b, ..., t1 i, ...) {
189     <+...
190     if (b->a) {... __FUNCTION__@p(..., i, ...) ...}
191     else{...}
192     ...+>
193 }
194
195 @script:python@
196 f << freeing9.f;
197 p << freeing9.p;
198 t << freeing9.t1;
199 rt << freeing9.rt;
200 b << freeing9.b;
201 a << freeing9.a;
202
203 @@
204 if t == __CHECKTYPE__:
205     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + f
206           ",{b}. {a}")
207     print(f"Used: __FUNCTION__")
208     print(f"Ret. Type: {rt}")
209
210 @freeing10 exists@
211 type rt, t;
212 identifier f, i, a;
213 position p;
214
215 @@
216 rt f(..., t i, ...) {
217     <+...
218     if (i->a) {... __FUNCTION__@p(..., i, ...) ...}
219     ...+>
220 }
221
222 @script:python@
223 f << freeing10.f;
224 p << freeing10.p;
225 t << freeing10.t;
226 rt << freeing10.rt;
227 a << freeing10.a;
228 i << freeing10.i;
229
230 @@
231 if t == __CHECKTYPE__:
232     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + f
233           ",{i}->{a}")
234     print(f"Used: __FUNCTION__")
235     print(f"Ret. Type: {rt}")
236
237 @freeing11 exists@
238 type rt, t;
239 identifier f, i, a;
240 position p;
241
242 @@
243 rt f(..., t i, ...) {
244     <+...
245     if (i->a) {... __FUNCTION__@p(..., i, ...) ...}
246     else{...}
247     ...+>
248 }
249
250 @script:python@
251 f << freeing11.f;

```

```

250 p << freeing11.p;
251 t << freeing11.t;
252 rt << freeing11.rt;
253 a << freeing11.a;
254 i << freeing11.i;
255
256 @@
257 if t == __CHECKTYPE__:
258     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column + f
259           ",{i}->{a}")
259     print(f"Used: __FUNCTION__")
260     print(f"Ret. Type: {rt}")

```

A.4 FUNCTIONS WITH TWO PARAMETERS OF SAME TYPE COCCINELLE TEMPLATE

Listing A.4: Coccinelle template for functions with two parameters of the same type

```

1  @freeing0 exists@
2  type rt, t;
3  identifier f, i, j;
4  position p;
5
6  @@
7  rt f(..., t i, ..., t j, ...) {
8      <+...
9      __FUNCTION__@p(..., i, ...)
10     ...+>
11 }
12
13 @script:python@
14 f << freeing0.f;
15 p << freeing0.p;
16 t << freeing0.t;
17 rt << freeing0.rt;
18
19 @@
20 if t == __CHECKTYPE__:
21     print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
22     print(f"Used: __FUNCTION__")
23     print(f"Ret. Type: {rt}")
24
25 @freeing1 exists@
26 type rt, t;
27 identifier f, i, j;
28 position p;
29
30 @@
31 rt f(..., t i, ..., t j, ...) {
32     <+...
33     __FUNCTION__@p(..., j, ...)
34     ...+>
35 }
36
37 @script:python@
38 f << freeing1.f;
39 p << freeing1.p;
40 t << freeing1.t;
41 rt << freeing1.rt;
42
43 @@
44 if t == __CHECKTYPE__:

```

```

45 print(f">{f},__PRINTTYPE__, " + p[0].file + ":" + p[0].line + ":" + p[0].column)
46 print(f"Used: __FUNCTION__")
47 print(f"Ret. Type: {rt}")

```

A.5 COLLECT RESULTS BASH SCRIPT

Listing A.5: Bash script for collecting results

```

1  #!/bin/bash
2  #COLLECT AND PREPARE THE RESULTS OF THE COCCINELLE SCRIPTS
3  #THIS SCRIPT ASSUMES THAT make HAS BEEN CALLED AND ALL THE NEEDED DIRECTORIES EXIST
4
5  RESULTS=./results
6  COLLECTED=$RESULTS/collected
7  PRINT_LINES=2
8  STRICT=C1
9  DEPENDENT=C2
10 ARBITRARY=C3
11 REASSIGN=C4
12
13 if [[ -d $COLLECTED ]]; then
14     rm -rf $COLLECTED
15 fi
16
17 mkdir $COLLECTED
18
19 #THIS FUNCTION SANITIZES THE OUPUT OF THE REALLOC SCRIPTS SINCE SOME TYPES
20 #LIKE int OR size_t HAVE BEEN ADDED TOO
21 reduceRealloc(){
22
23     awk 'NR==FNR { blacklist[$0]; next }
24     {
25         split($0, parts, ",");
26         if (!(parts[2] in blacklist)) print
27     }' exceptions/types.excluded \
28     $RESULTS/realloc$1/functionnamesonly.out > realloc.tmp && \
29     mv realloc.tmp $RESULTS/realloc$1/functionnamesonly.out
30 }
31
32 #COLLECT THE RESULTS FOR THE PASSED FUNCTION (free/realloc)
33 collect(){
34
35     #CREATE CUSTOM DIRECTORIES FOR FUNCTIONS WITH SAME RETURN TYPE
36     #AND ARBITRARY FUNCTIONS
37     if [[ -d $RESULTS/$larbitrary ]]; then
38         rm -rf $RESULTS/$larbitrary
39     fi
40
41     if [[ -d $RESULTS/$lsamereturn ]]; then
42         rm -rf $RESULTS/$lsamereturn
43     fi
44
45     if [[ -d $RESULTS/$lboolreturn ]]; then
46         rm -rf $RESULTS/$lboolreturn
47     fi
48     mkdir $RESULTS/$larbitrary
49     mkdir $RESULTS/$lsamereturn
50     mkdir $RESULTS/$lboolreturn
51
52     #DEFINE FILE STATS WILL BE PRINTED TO
53     STATS_FILE=$COLLECTED/stats_$1.out
54
55     #REDUCE RESULTS TO FUNCTION NAMES + TYPE

```

```

56 # + FILE ONLY AND ELIMINATE DUPLICATES
57 grep "^>" $RESULTS/$1/results.out | cut -d ':' -f 1 | \
58 sort | uniq > $RESULTS/$1/functionnamesonly.out
59 grep "^>" $RESULTS/$1static/results.out | cut -d ':' -f 1 | \
60 sort | uniq > $RESULTS/$1static/functionnamesonly.out
61 grep "^>" $RESULTS/$1strict/results.out | cut -d ':' -f 1 | \
62 sort | uniq > $RESULTS/$1strict/functionnamesonly.out
63 grep "^>" $RESULTS/$1signature/results.out | cut -d ':' -f 1 | \
64 sort | uniq > $RESULTS/$1signature/functionnamesonly.out
65 grep "^>" $RESULTS/$1dependent/results.out | cut -d ':' -f 1 | \
66 sort | uniq > $RESULTS/$1dependent/functionnamesonly.out
67 grep "^>" $RESULTS/$1double/results.out | cut -d ':' -f 1 | \
68 sort | uniq > $RESULTS/$1double/functionnamesonly.out
69
70 #SANITIZE REALLOC RESULTS -> REMOVES ANY SIZE, int, ETC TYPES...
71 if [[ "$1" == "realloc" ]]; then
72     reduceRealloc
73     reduceRealloc static
74     reduceRealloc strict
75     reduceRealloc dependent
76     reduceRealloc signature
77     reduceRealloc double
78 fi
79
80 #REMOVE STATIC FUNCTIONS FROM RESULTS
81 comm -23 $RESULTS/$1/functionnamesonly.out \
82 $RESULTS/$1static/functionnamesonly.out > tmp
83 mv tmp $RESULTS/$1/functionnamesonlywithoutstatic.out
84 comm -23 $RESULTS/$1strict/functionnamesonly.out \
85 $RESULTS/$1static/functionnamesonly.out > tmp
86 mv tmp $RESULTS/$1strict/functionnamesonlywithoutstatic.out
87 comm -23 $RESULTS/$1signature/functionnamesonly.out \
88 $RESULTS/$1static/functionnamesonly.out > tmp
89 mv tmp $RESULTS/$1signature/functionnamesonlywithoutstatic.out
90 comm -23 $RESULTS/$1dependent/functionnamesonly.out \
91 $RESULTS/$1static/functionnamesonly.out > tmp
92 mv tmp $RESULTS/$1dependent/functionnamesonlywithoutstatic.out
93 comm -23 $RESULTS/$1double/functionnamesonly.out \
94 $RESULTS/$1static/functionnamesonly.out > tmp
95 mv tmp $RESULTS/$1double/functionnamesonlywithoutstatic.out
96
97 #RETRIEVE ALL FUNCTIONS WITH THAT RETURN THE SAME TYPE AS THEY FREE/REALLOC
98 # -> WE CREATE A FILE WITH ALL FUNCTION NAMES + TYPE + FILE NAME
99 # AND ANOTHER ONE WITH RETURN WITH RETURN TYPES ONLY
100 # -> FOR EVERY LINE IN BOTH FILES, IF THE TYPE CORRESPONDS TO THE RETURN TYPE
101 # WE PRINT THE FUNCTIONS TO samereturn.tmp
102 grep "^>" $RESULTS/$1/results.out > allfuncs.tmp
103 grep "^Ret. Type:" $RESULTS/$1/results.out | cut -d ':' -f 2 \
104 |cut -c2- > alltypes.tmp
105
106 COUNTER=0
107 OVERALL=$(cat allfuncs.tmp | wc -l)
108
109 while [[ $COUNTER -lt $OVERALL ]]; do
110     COUNTER=$(( COUNTER + 1 ))
111     type=$(sed -n "${COUNTER}p" alltypes.tmp)
112     if [[ "$type" == $(sed -n "${COUNTER}p" allfuncs.tmp \
113 | cut -d ',' -f 2) ]]; then
114         sed -n "${COUNTER}p" allfuncs.tmp >> samereturn.tmp
115     else if [[ "$type" == "bool" ]]; then
116         if [[ -f exceptions/$1boolreturn.exclude ]]; then
117             #REMOVE EXCEPTIONS (ONLY INCLUDE FILE NAME AND
118             #ESCAPE SPECIAL CHARACTERS)
119             if grep -q "$(sed -n "${COUNTER}p" allfuncs.tmp | cut -d ':' -f 1 \
120 | sed 's/[\\.*^(){}|]/\\&/g')" exceptions/$1boolreturn.exclude;
121 \

```

```

121             then
122                 continue
123             fi
124         fi
125         sed -n "${COUNTER}p" allfuncs.tmp >> boolreturn.tmp
126     fi
127 fi
128
129 done
130
131 #ADD MANUALLY ADDED
132 if [[ -f exceptions/$lsamereturn.add ]]; then
133     cat exceptions/$lsamereturn.add >> samereturn.tmp
134 fi
135 if [[ -f exceptions/$lboolreturn.add ]]; then
136     cat exceptions/$lboolreturn.add >> boolreturn.tmp
137 fi
138
139 #GET RID OF DUPLICATES
140 cat samereturn.tmp | cut -d ':' -f 1 | sort | uniq \
141 > $RESULTS/$lsamereturn/functionnamesonly.out
142 cat boolreturn.tmp | cut -d ':' -f 1 | sort | uniq \
143 > $RESULTS/$lboolreturn/functionnamesonly.out
144
145 #SANITIZE REALLOC RESULTS
146 if [[ "$1" == "realloc" ]]; then
147     reduceRealloc samereturn
148     reduceRealloc boolreturn
149 fi
150
151 #REMOVE STATIC
152 comm -23 $RESULTS/$lsamereturn/functionnamesonly.out \
153 $RESULTS/$lstatic/functionnamesonly.out > tmp
154 mv tmp $RESULTS/$lsamereturn/functionnamesonlywithoutstatic.out
155 comm -23 $RESULTS/$lboolreturn/functionnamesonly.out \
156 $RESULTS/$lstatic/functionnamesonly.out > tmp
157 mv tmp $RESULTS/$lboolreturn/functionnamesonlywithoutstatic.out
158
159 #PRINT FUNCTIONS WITH INFORMATION
160 grep -A $PRINT_LINES -F -f \
161 $RESULTS/$lsamereturn/functionnamesonlywithoutstatic.out \
162 $RESULTS/$1/results.out > $COLLECTED/${REASSIGN}_$1.out
163
164 #REDUCE OVER FUNCTION AND EXTRACT CATEGORIES 1-3
165 grep -A $PRINT_LINES -F -f \
166 $RESULTS/$lstrict/functionnamesonlywithoutstatic.out \
167 $RESULTS/$1/results.out > $COLLECTED/${STRICT}_$1.out
168 grep -A $PRINT_LINES -F -f \
169 $RESULTS/$ldependent/functionnamesonlywithoutstatic.out \
170 $RESULTS/$1/results.out > $COLLECTED/${DEPENDENT}_$1.out
171
172 comm -23 $RESULTS/$1/functionnamesonlywithoutstatic.out \
173 $RESULTS/$lstrict/functionnamesonly.out | sort \
174 | uniq > allwithoutstaticstrict.tmp
175 comm -23 allwithoutstaticstrict.tmp \
176 $RESULTS/$ldependent/functionnamesonly.out | sort \
177 | uniq > allwithoutstaticstrictdependent.tmp
178
179 #EXPAND ABOVE RESULTS TO INCLUDE ALL INFORMATION (NOT ONLY FUNCTION NAME AND TYPE)
180 cat allwithoutstaticstrictdependent.tmp | sort \
181 | uniq > $RESULTS/$larbitrary/functionnamesonly.out
182 cp $RESULTS/$larbitrary/functionnamesonly.out \
183 $COLLECTED/${ARBITRARY}_$1_functionsonly.out
184
185 #SANITIZE REALLOC RESULTS
186 if [[ "$1" == "realloc" ]]; then

```

```

187     reduceRealloc arbitrary
188 fi
189
190 #PRINT FUNCTIONS WITH INFORMATION
191 grep -A $PRINT_LINES -F -f $RESULTS/$lbitrary/functionnamesonly.out \
192     $RESULTS/$l/results.out > $RESULTS/$lbitrary/results.out
193 cp $RESULTS/$lbitrary/results.out $COLLECTED/${ARBITRARY}_$l.out
194
195 #RETRIEVE FULL FUNCTION NAMES ONLY WITH ALL INFORMATION FOR COLLECTED
196 grep "^>" $COLLECTED/${REASSIGN}_$l.out | sort | uniq \
197     > $COLLECTED/${REASSIGN}_$l_functiononly.out
198 grep "^>" $COLLECTED/${STRICT}_$l.out | sort | uniq \
199     > $COLLECTED/${STRICT}_$l_functiononly.out
200 grep "^>" $COLLECTED/${DEPENDENT}_$l.out | sort | uniq \
201     > $COLLECTED/${DEPENDENT}_$l_functiononly.out
202 grep "^>" $COLLECTED/${ARBITRARY}_$l.out | sort | uniq \
203     > $COLLECTED/${ARBITRARY}_$l_functiononly.out
204
205 #PRINT STATS TO THE STATS_FILE
206 echo "$l" > $STATS_FILE
207 echo "ALL:" >> $STATS_FILE
208 grep "^>" $RESULTS/$l/functionnamesonly.out | wc -l >> $STATS_FILE
209 echo "ALL WITHOUT STATIC:" >> $STATS_FILE
210 grep "^>" $RESULTS/$l/functionnamesonlywithoutstatic.out | sort | uniq \
211     | wc -l >> $STATS_FILE
212 echo "SAME RETURN:" >> $STATS_FILE
213 grep "^>" $RESULTS/$lsamereturn/functionnamesonlywithoutstatic.out | cut -d ':' -f
214     1 \
215     | wc -l >> $STATS_FILE
216 echo "BOOL RETURN:" >> $STATS_FILE
217 grep "^>" $RESULTS/$lboolreturn/functionnamesonlywithoutstatic.out | cut -d ':' -f
218     1 \
219     | wc -l >> $STATS_FILE
220 echo "STRICT:" >> $STATS_FILE
221 grep "^>" $COLLECTED/${STRICT}_$l.out | cut -d ':' -f 1 | sort | uniq \
222     | wc -l >> $STATS_FILE
223 echo "DEPENDENT:" >> $STATS_FILE
224 grep "^>" $COLLECTED/${DEPENDENT}_$l.out | cut -d ':' -f 1 | sort | uniq \
225     | wc -l >> $STATS_FILE
226 echo "ARBITRARY:" >> $STATS_FILE
227 grep "^>" $RESULTS/$lbitrary/functionnamesonly.out | wc -l >> $STATS_FILE
228
229 #PRINT STATS TO THE OUTPUT
230 cat $STATS_FILE
231
232 #RETRIEVE THE INDEXES OF PARAMETERS
233 ./getArgumentPosition.sh free C1 &> /dev/null;
234 ./getArgumentPosition.sh free C3 &> /dev/null;
235 ./getArgumentPosition.sh realloc C1 &> /dev/null;
236 ./getArgumentPosition.sh realloc C3 &> /dev/null;
237
238 #CLEAN UP
239 rm *.tmp
240 }
241
242 #ACTUALLY RUN THE SCRIPT
243 collect free
244 collect realloc

```

A.6 TEMPLATE TO RETRIEVE THE POSITION OF ARGUMENTS

Listing A.6: Coccinelle template retrieving the position of arguments

```

1 @pos exists@
2 parameter list params;
3 identifier f;
4
5 @@
6 f (params) {
7     ...
8 }
9 @script:python@
10 params << pos.params;
11 f << pos.f;
12 @@
13 if f == "__FUNCTION__":
14     counter = 0
15     for param in params:
16         type = " ".join(param.split()[:-1])
17         if type == "__CHECKTYPE__":
18             print(f"__FUNCTION__: {counter}")
19             counter += 1

```

A.7 SCRIPT TO RETRIEVE ARGUMENT POSITION

Listing A.7: Bash script to retrieve the position of arguments in signatures

```

1 #!/bin/bash
2
3 FUNCTION=$1
4 CATEGORY=$2
5 RESULTS_FILE_NAME=${CATEGORY}_${FUNCTION}.position
6 RESULTS=./results
7 RESULTS_PATH=$RESULTS/positionalData
8 RESULTS_FILE=$RESULTS_PATH/$RESULTS_FILE_NAME
9 TMP_PATH=tmp
10 MANUALLY_EXCLUDED=exceptions/${CATEGORY}${FUNCTION}.position.exclude
11
12 if [[ ! -d "$RESULTS_PATH" ]]; then
13     mkdir "$RESULTS_PATH"
14 fi
15
16 if [[ -f "$RESULTS_FILE" ]]; then
17     rm "$RESULTS_FILE"
18 fi
19
20 mkdir -p $TMP_PATH
21 while read -r line; do
22     [[ -z "$line" || ! "$line" =~ ^\> ]] && continue
23
24     if [[ "$line" =~ ^\>([^\,]+),([^\,]*),(.+)\$ ]]; then
25         func=$(echo "${BASH_REMATCH[1]}" | sed 's/^[:space:]*//;s/[[:space:]]*$/')
26         ptype=$(echo "${BASH_REMATCH[2]}" | sed 's/^[:space:]*//;s/[[:space:]]*$/')
27
28         file=$(echo "${BASH_REMATCH[3]}" | sed 's/^[:space:]*//;s/[[:space:]]*$/')
29         | cut -d':' -f1)
30     else
31         continue
32     fi
33
34     [[ -z "$func" || -z "$file" || ! -f "$file" ]] && continue
35
36     cat templates/templateposition.cocci \
37         | sed -e "s/___FUNCTION___/$func/g" \
38         -e "s/___CHECKTYPE___/$ptype/g" \

```

```

37         > $TMP_PATH/tmp.cocci
38     spatch --sp-file $TMP_PATH/tmp.cocci "$file" >> "$RESULTS_FILE"
39
40 done < $RESULTS/collected/${CATEGORY}-${FUNCTION}-functionsonly.out
41
42 grep -vFf exceptions/${CATEGORY}${FUNCTION}.exclude "$RESULTS_FILE" > temp && mv temp "
    $RESULTS_FILE"
43
44 sed 's/^\([^:]*\):\([0-9]*\)$/{{"\1"}, {\2}},/' "$RESULTS_FILE" | sort | uniq > "
    $RESULTS_FILE".extracted
45
46 rm -r tmp

```

A.8 COCCINELLE SCRIPT TO ADDRESS CATEGORY 5

Listing A.8: Full Coccinelle script to address Category 5

```

1  @rule1a@
2  expression param0, param1, param2, param3, ret;
3  statement S;
4  @@
5  ret = ecpg_check_PQresult(param0, param1, param2, param3);
6  (
7      if (ret) S
8  |
9      if (!ret) S
10 |
11 + if (!ret) param0 = NULL;
12   S
13 )
14
15 @rule1b@
16 expression param0, param1, param2, param3;
17 identifier ret;
18 statement S;
19 @@
20 bool ret = ecpg_check_PQresult(param0, param1, param2, param3);
21 (
22     if (ret) S
23 |
24     if (!ret) S
25 |
26 + if (!ret) param0 = NULL;
27   S
28 )
29
30 @rule2@
31 expression param0, param1, param2, param3;
32 @@
33
34 - ecpg_check_PQresult(param0, param1, param2, param3);
35 + ret = ecpg_check_PQresult(param0, param1, param2, param3);
36 + if (!ret) param = NULL;

```

A.9 COCCINELLE SCRIPT TO REPORT ISSUES FROM CATEGORY 5

Listing A.9: Coccinelle script to report Category 5 issues

```

1  @all_calls@
2  expression param0, param1, param2, param3;
3  identifier ret;
4  position p;
5  @@
6  ret = ecpg_check_PQresult@p(param0, param1, param2, param3);
7
8  @checked exists@
9  expression param0, param1, param2, param3;
10 identifier ret;
11 position all_calls.p;
12 statement S;
13 @@
14 ret = ecpg_check_PQresult@p(param0, param1, param2, param3);
15 ...
16 (
17 if (ret) S
18 |
19 if (!ret) S
20 )
21
22 @script:python depends on all_calls && !checked@
23 ret << all_calls.ret;
24 p << all_calls.p;
25 @@
26 print(f"Warning: {p[0].file}:{p[0].line}: Return value '{ret}' not checked")

```

A.10 SCRIPT TO ANALYZE DIRECTORIES

Listing A.10: Bash script to run Clang against a directory

```

1  #!/bin/bash
2
3  # PostgresChecker Directory Analyzer Script
4  # Usage: ./analyze_directory.sh [directory] [output_file]
5
6  DIRECTORY=${1:-"."}
7  # Generate output filename based on directory name
8  DIR_NAME=$(basename "$DIRECTORY")
9  TMP_DIR=PGtmp
10 # Replace . with current_directory for readability
11 if [ "$DIR_NAME" = "." ]; then
12     DIR_NAME="current_directory"
13 fi
14 OUTPUT_FILE=${2:-"analysis_${DIR_NAME}_results.out"}
15
16 # PostgreSQL source path
17 if [ -z "${PG_PATH}" ]; then
18     PG_PATH=/usr/local/src/postgresql
19 fi
20 if [ ! -d "$PG_PATH" ]; then
21     echo "Need to set valid PG_PATH!"
22     exit 1
23 fi
24 if [ ! -d "$TMP_DIR" ]; then
25     mkdir "$TMP_DIR"
26 fi
27
28 # Get absolute path of the directory being analyzed
29 ANALYSIS_DIR=$(realpath "$DIRECTORY")
30
31 cd "$TMP_DIR"
32

```

```

33 # Colors for output
34 RED='\033[0;31m'
35 GREEN='\033[0;32m'
36 YELLOW='\033[1;33m'
37 BLUE='\033[0;34m'
38 NC='\033[0m' # No Color
39
40 echo "=====
41 echo "PostgreSQL Directory Analysis"
42 echo "=====
43 echo "Directory: $ANALYSIS_DIR"
44 echo "Output file: $OUTPUT_FILE"
45 echo "PostgreSQL path: $PG_PATH"
46 echo "Started: $(date)"
47 echo "=====
48
49 # Clear output file
50 > ../"$OUTPUT_FILE"
51
52 # Write header to output file
53 {
54     echo "PostgresChecker Static Analysis Results"
55     echo "=====
56     echo "Directory: $ANALYSIS_DIR"
57     echo "Path: $PG_PATH"
58     echo "Started: $(date)"
59     echo "Clang version: $(clang-19 --version | head -1)"
60     echo ""
61 } >> ../"$OUTPUT_FILE"
62
63 # Function to build include paths dynamically
64 build_include_paths() {
65     local includes=""
66
67     # Include server
68     includes="$includes -I$(pg_config --includedir-server)"
69
70     # Include client
71     includes="$includes -I$(pg_config --includedir)"
72
73     # Extension-specific includes
74     if [ -d "$ANALYSIS_DIR/src/include" ]; then
75         includes="$includes -I$ANALYSIS_DIR/src/include"
76     fi
77
78     # Additional backend includes
79     if [ -d "$ANALYSIS_DIR/src/backend" ]; then
80         includes="$includes -I$ANALYSIS_DIR/src/backend"
81     fi
82
83     # Try common build locations
84     if [ -d "$ANALYSIS_DIR/./build" ]; then
85         includes="$includes -I$ANALYSIS_DIR/./build"
86         includes="$includes -I$ANALYSIS_DIR/./build/src"
87     fi
88
89     echo "$includes"
90 }
91
92 # Function to check if checker library exists
93 check_checker_lib() {
94     if [ -f "/usr/local/lib/libPostgresChecker.so" ]; then
95         echo -e "${GREEN}[OK] PostgreSQL checker library found${NC}"
96         return 0
97     else

```

```

98         echo -e "${YELLOW}! PostgreSQL checker library not found at /usr/local/lib/
        libPostgresChecker.so${NC}"
99         return 1
100     fi
101 }
102
103 # Function to analyze a single file
104 analyze_file() {
105     local file="$1"
106     local temp_output="$2"
107     local includes="$3"
108     local use_checker="$4"
109
110     clang-19 --analyze \
111         $includes \
112         -Xclang -load -Xclang /usr/local/lib/libPostgresChecker.so \
113         --analyzer-no-default-checks \
114         -Xclang -analyzer-checker=postgres.PostgresChecker \
115         "$file" > "$temp_output" 2>&1
116 }
117
118 # Main analysis
119 echo -e "${BLUE}Preparing analysis...${NC}"
120
121 # Build include paths
122 INCLUDE_PATHS=$(build_include_paths)
123 echo "Include paths: $INCLUDE_PATHS"
124
125 # Check for checker library
126 if check_checker_lib; then
127     echo "Command: clang-19 --analyze $INCLUDE_PATHS -Xclang -load -Xclang /usr/local/
        lib/libPostgresChecker.so --analyzer-no-default-checks -Xclang -analyzer-checker
        =postgres.PostgresChecker"
128 else
129     echo "Checker lib has not been found."
130     return 2;
131 fi
132
133 # Write command info to output file
134 {
135     echo "Include paths: $INCLUDE_PATHS"
136     echo "Using custom checker: $USE_CHECKER"
137     echo "Command: clang-19 --analyze $INCLUDE_PATHS -Xclang -load -Xclang /usr/local/
        lib/libPostgresChecker.so --analyzer-no-default-checks -Xclang -analyzer-checker
        =postgres.PostgresChecker"
138     echo ""
139 } >> "$OUTPUT_FILE"
140
141 # Counters
142 total_files=0
143 analyzed_files=0
144 failed_files=0
145 files_with_warnings=0
146
147 echo -e "${BLUE}Starting analysis...${NC}"
148
149 # Find and analyze all .c files
150 while IFS= read -r file; do
151     ((total_files++))
152
153     echo -e "${YELLOW}Analyzing ($total_files):${NC} $file"
154
155     # Create temporary file for analysis output
156     temp_output=$(mktemp)
157
158     # Run analysis

```

```

159 if analyze_file "$file" "$temp_output" "$INCLUDE_PATHS" "$USE_CHECKER"; then
160     echo -e "${GREEN} [OK] Analysis completed${NC}"
161     ((analyzed_files++))
162
163     # Check if there were any warnings or errors in the output
164     if grep -q -E "(warning:|error:|note:)" "$temp_output"; then
165         echo -e "${RED} ! Issues found${NC}"
166         ((files_with_warnings++))
167
168         # Count different types of issues
169         warnings=$(grep -c "warning:" "$temp_output" 2>/dev/null)
170         errors=$(grep -c "error:" "$temp_output" 2>/dev/null)
171
172         echo -e "${RED} Errors: $errors, Warnings: $warnings"
173
174         # Write file header and content to output file
175         {
176             echo "=====
177             echo "File: $file"
178             echo "Time: $(date)"
179             echo "Status: ISSUES FOUND"
180             echo "Errors: $errors, Warnings: $warnings"
181             echo "=====
182             cat "$temp_output"
183             echo ""
184         } >> ../"$OUTPUT_FILE"
185     else
186         echo -e "${GREEN} [OK] Clean (no issues)${NC}"
187     fi
188 else
189     echo -e "${RED} [X] Analysis failed${NC}"
190     ((failed_files++))
191     cat "$temp_output"
192
193     # Always write failed files to output
194     #{
195         #echo "=====
196         #echo "File: $file"
197         #echo "Time: $(date)"
198         #echo "Status: ANALYSIS FAILED"
199         #echo "=====
200         #cat "$temp_output"
201         #echo ""
202     #} >> "$OUTPUT_FILE"
203 fi
204
205 # Clean up temporary file
206 rm -f "$temp_output"
207
208 # Progress update every 10 files
209 if (( total_files % 10 == 0 )); then
210     echo -e "${BLUE}Progress: $total_files files processed...${NC}"
211 fi
212 done < <(find "$ANALYSIS_DIR" -name "*.c" -type f)
213
214 # Write summary to output file
215 {
216     echo ""
217     echo "=====
218     echo "ANALYSIS SUMMARY"
219     echo "=====
220     echo "Completed: $(date)"
221     echo "Total files found: $total_files"
222     echo "Successfully analyzed: $analyzed_files"
223     echo "Failed to analyze: $failed_files"
224     echo "Files with warnings/errors: $files_with_warnings"

```

```

225     #echo "Success rate: $(( analyzed_files * 100 / total_files ))%"
226     echo "=====
227 } >> "$OUTPUT_FILE"
228
229 # Display summary
230 echo ""
231 echo "=====
232 echo -e "${GREEN}ANALYSIS COMPLETE${NC}"
233 echo "=====
234 echo "Total files found: $total_files"
235 echo "Successfully analyzed: $analyzed_files"
236 echo "Failed to analyze: $failed_files"
237 echo "Files with warnings/errors: $files_with_warnings"
238 #echo "Success rate: $(( analyzed_files * 100 / total_files ))%"
239 echo ""
240 echo "Results saved to: $OUTPUT_FILE"
241 echo ""
242
243 echo "=====
244
245 cd ..
246 rm -r "$TMP_DIR"

```

THE POSTGRESchecker

B.1 IMPLEMENTATION

Listing B.1: Implementation in the Clang Static Analyzer framework

```

1  //== PostgresChecker.cpp -----* C++ -*==//
2  //
3  // Part of the LLVM Project, under the Apache License v2.0 with LLVM Exceptions.
4  // See https://llvm.org/LICENSE.txt for license information.
5  // SPDX-License-Identifier: Apache-2.0 WITH LLVM-exception
6  //
7  //===-----//
8  //
9  // This file defines PostgresChecker, a checker for memory management related
10 // issues when using the PostgreSQL API. It includes checks for use-after-free
11 // and double-free.
12 //
13 // It divides PostgreSQL functions that handle memory into five categories:
14 //
15 // 1. Strict functions:
16 //     These functions will always free memory and, therefore, are treated
17 //     like traditional free functions.
18 // 2. Dependent functions:
19 //     These functions will free memory depending on another parameter.
20 //     This could, for example, be a boolean 'shouldFree' being passed to the
21 //     function, deciding whether another parameter is freed.
22 // 3. Arbitrary functions:
23 //     Like dependent functions, the freeing of the parameter is dependent on
24 //     certain circumstances, which, however, cannot be checked statically.
25 // 4. Reassigning functions:
26 //     Functions like realloc.
27 // 5. Functions returnign boolean:
28 //     Functions that always return true or false when a parameter is freed.
29 //
30 // The first three categories partition the set of all memory managment related
31 // PostgreSQL functions. For these the checker implements logic similar to the
32 // MallocChecker. Categories 4 and 5 are pattern based and only serve to provide
33 // better feedback to the developer.
34 //
35 //===-----//
36
37 #include "clang/StaticAnalyzer/Frontend/CheckerRegistry.h"
38 #include "clang/StaticAnalyzer/Core/PathSensitive/CallEvent.h"
39 #include "clang/StaticAnalyzer/Core/PathSensitive/CheckerContext.h"
40 #include <llvm/Support/raw_ostream.h>
41
42 using namespace clang;
43 using namespace ento;
44
45 class PGRefState;
46 struct DependencyInfo;
47 enum Category {Strict, Dependent, Arbitrary};
48 enum Tristate {True, False, Undefined};
49
50 namespace{
51 class PostgresChecker :
52     public Checker<check::PreCall, check::Location, check::PreStmt<ReturnStmt>> {

```

```

53 mutable std::unique_ptr<BugType> BT_Free_Strict;
54 mutable std::unique_ptr<BugType> BT_Free_Arbitrary;
55
56 public:
57 void checkPreCall(const CallEvent &Call, CheckerContext &C) const;
58 void checkLocation(SVal l, bool isLoad, const Stmt *S,
59                  CheckerContext &C) const;
60 void checkPreStmt(const ReturnStmt *S, CheckerContext &C) const;
61 bool checkUseAfterFree(SymbolRef Sym, std::string varName,
62                      CheckerContext &C, const Stmt * S) const;
63 private:
64 void HandleFree(const CallEvent &Call, CheckerContext &C,
65               Category Cat) const;
66 void HandleUseAfterFree(CheckerContext &C, SourceRange Range, const
67                      PGRRefState *RS, std::string varName,
68                      Category Cat) const;
69 void HandleDoubleFree(CheckerContext &C, SourceRange Range,
70                      const PGRRefState *RS, std::string varName,
71                      Category FirstFreeCat, Category Cat) const;
72 void emitReport(const PGRRefState *RS, BugType *BT, CheckerContext &C,
73               std::string message) const;
74 };
75 } // end of anonymous namespace
76
77 // Map that tracks PGRRefStates for symbol references. Entries are defined in the
78 // state of the CheckContext and can be accessed on all successive paths.
79 REGISTER_MAP_WITH_PROGRAMSTATE(RegionStatePG, SymbolRef, PGRRefState)
80
81 // Storing the state at the function call.
82 class PGRRefState {
83 // Keep track of whether the call was strict or arbitrary.
84 enum Kind {
85     // Reference to released/freed memory.
86     Released,
87     // Possible reference to released/freed memory.
88     PossiblyReleased
89 };
90
91 const Stmt *S;
92 Kind K;
93 const FunctionDecl *FD;
94 std::string OriginalVarName;
95
96 PGRRefState(Kind k, const Stmt *s, const FunctionDecl *fd, std::string varName)
97     : S(s), K(k), FD(fd), OriginalVarName(varName) {}
98
99 public:
100 const std::string& getVarName() const { return OriginalVarName; }
101 bool isReleased() const { return K == Released; }
102 bool isPossiblyReleased() const { return K == PossiblyReleased; }
103 const Stmt *getStmt() const { return S; }
104 const FunctionDecl *getFunction() const { return FD; }
105
106 bool operator==(const PGRRefState &X) const {
107     return K == X.K && S == X.S && FD == X.FD
108         && OriginalVarName == X.OriginalVarName;
109 }
110
111 static PGRRefState getReleased(const Stmt *s, const FunctionDecl *fd,
112                             std::string varName = "") {
113     return PGRRefState(Released, s, fd, varName);
114 }
115 static PGRRefState getPossiblyReleased(const Stmt *s,
116                                       const FunctionDecl *fd,
117                                       std::string varName = "") {
118     return PGRRefState(PossiblyReleased, s, fd, varName);

```

```

119     }
120
121     void Profile(llvm::FoldingSetNodeID &ID) const {
122         ID.AddInteger(K);
123         ID.AddPointer(S);
124         ID.AddPointer(FD);
125     }
126
127     LLVM_DUMP_METHOD void dump(raw_ostream &OS) const {
128         switch (K) {
129 #define CASE(ID) case ID: OS << #ID; break;
130             CASE(Released)
131             CASE(PossiblyReleased)
132         }
133     }
134
135     LLVM_DUMP_METHOD void dump() const { dump(llvm::errs()); }
136
137 };
138
139 // This map defines standard C memory management functions. We look out for them
140 // to not get in the way of the MallocChecker.
141 llvm::StringMap<int> CMemoryMap{
142     {"free"}, {0}},
143     {"realloc"}, {0}},
144 };
145
146 // Map representing strict functions: the function names are mapped to the index
147 // of the parameter being freed.
148 llvm::StringMap<unsigned int> StrictMap{
149     // C function
150     {"free"}, {0}},
151     // PostgreSQL-specific functions
152     {"arabic_UTF_8_close_env"}, {0}},
153     {"armenian_UTF_8_close_env"}, {0}},
154     {"array_free_iterator"}, {0}},
155     {"basque_ISO_8859_1_close_env"}, {0}},
156     {"basque_UTF_8_close_env"}, {0}},
157     {"binaryheap_free"}, {0}},
158     {"BipartiteMatchFree"}, {0}},
159     {"BlockRefTableFreeEntry"}, {0}},
160     {"bloom_free"}, {0}},
161     {"bms_free"}, {0}},
162     {"brin_free_tuple"}, {0}},
163     {"brinRevmapTerminate"}, {0}},
164     {"BufFileClose"}, {0}},
165     {"BumpDelete"}, {0}},
166     {"catalan_ISO_8859_1_close_env"}, {0}},
167     {"catalan_UTF_8_close_env"}, {0}},
168     {"CatalogCloseIndexes"}, {0}},
169     {"closedir"}, {0}},
170     {"close_tsvector_parser"}, {0}},
171     {"conditional_stack_destroy"}, {0}},
172     {"core_yyfree"}, {0}},
173     {"danish_ISO_8859_1_close_env"}, {0}},
174     {"danish_UTF_8_close_env"}, {0}},
175     {"DeCloneArchive"}, {0}},
176     {"DestroyBlockRefTableReader"}, {0}},
177     {"DestroyBlockRefTableWriter"}, {0}},
178     {"DestroyParallelContext"}, {0}},
179     {"destroyPQExpBuffer"}, {0}},
180     {"destroyStringInfo"}, {0}},
181     {"DestroyTupleQueueReader"}, {0}},
182     {"disconnectDatabase"}, {0}},
183     {"dsa_detach"}, {0}},
184     {"dshash_destroy"}, {0}},

```

```

185     {"dshash_detach"}, {0}},
186     {"dsm_detach"}, {0}},
187     {"dutch_ISO_8859_1_close_env"}, {0}},
188     {"dutch_UTF_8_close_env"}, {0}},
189     {"ecpg_do_epilogue"}, {0}},
190     {"ecpg_free"}, {0}},
191     {"ECPGfree_type"}, {0}},
192     {"EndCompressFileHandle"}, {0}},
193     {"EndCompressor"}, {1}},
194     {"EndCopyFrom"}, {0}},
195     {"EndCopyTo"}, {0}},
196     {"end_tup_output"}, {0}},
197     {"english_ISO_8859_1_close_env"}, {0}},
198     {"english_UTF_8_close_env"}, {0}},
199     {"ExecDropSingleTupleTableSlot"}, {0}},
200     {"ExecHashTableDestroy"}, {0}},
201     {"ExecParallelCleanup"}, {0}},
202     {"finnish_ISO_8859_1_close_env"}, {0}},
203     {"finnish_UTF_8_close_env"}, {0}},
204     {"FreeAccessStrategy"}, {0}},
205     {"free_attrmap"}, {0}},
206     {"FreeBulkInsertState"}, {0}},
207     {"free_chromo"}, {1}},
208     {"free_city_table"}, {1}},
209     {"free_conversion_map"}, {0}},
210     {"free_edge_table"}, {1}},
211     {"FreeErrorData"}, {0}},
212     {"FreeExprContext"}, {0}},
213     {"FreeFakeRelcacheEntry"}, {0}},
214     {"freeGinBtreeStack"}, {0}},
215     {"free_object_addresses"}, {0}},
216     {"free_parsestate"}, {0}},
217     {"free_pool"}, {1}},
218     {"FreeQueryDesc"}, {0}},
219     {"FreeSubscription"}, {0}},
220     {"FreeTriggerDesc"}, {0}},
221     {"FreeTupleDesc"}, {0}},
222     {"FreeWaitEventSet"}, {0}},
223     {"FreeWaitEventSetAfterFork"}, {0}},
224     {"french_ISO_8859_1_close_env"}, {0}},
225     {"french_UTF_8_close_env"}, {0}},
226     {"GenerationDelete"}, {0}},
227     {"GenericXLogAbort"}, {0}},
228     {"GenericXLogFinish"}, {0}},
229     {"german_ISO_8859_1_close_env"}, {0}},
230     {"german_UTF_8_close_env"}, {0}},
231     {"ginInsertValue"}, {1}},
232     {"greek_UTF_8_close_env"}, {0}},
233     {"guc_free"}, {0}},
234     {"heap_free_minimal_tuple"}, {0}},
235     {"heap_freetuple"}, {0}},
236     {"hindi_UTF_8_close_env"}, {0}},
237     {"_h_spooldestroy"}, {0}},
238     {"hungarian_ISO_8859_2_close_env"}, {0}},
239     {"hungarian_UTF_8_close_env"}, {0}},
240     {"index_endscan"}, {0}},
241     {"IndexScanEnd"}, {0}},
242     {"indonesian_ISO_8859_1_close_env"}, {0}},
243     {"indonesian_UTF_8_close_env"}, {0}},
244     {"inv_close"}, {0}},
245     {"irish_ISO_8859_1_close_env"}, {0}},
246     {"irish_UTF_8_close_env"}, {0}},
247     {"italian_ISO_8859_1_close_env"}, {0}},
248     {"italian_UTF_8_close_env"}, {0}},
249     {"jit_release_context"}, {0}},
250     {"json_parse_manifest_incremental_shutdown"}, {0}},

```

```

251     {"jsonpath_yyfree"}, {0}},
252     {"list_free"}, {0}},
253     {"list_free_deep"}, {0}},
254     {"lithuanian_UTF_8_close_env"}, {0}},
255     {"LogicalTapeClose"}, {0}},
256     {"LogicalTapeSetClose"}, {0}},
257     {"mbuf_free"}, {0}},
258     {"nepali_UTF_8_close_env"}, {0}},
259     {"norwegian_ISO_8859_1_close_env"}, {0}},
260     {"norwegian_UTF_8_close_env"}, {0}},
261     {"output_deallocate_prepare_statement"}, {0}},
262     {"output_prepare_statement"}, {0}},
263     {"output_simple_statement"}, {0}},
264     {"output_statement"}, {0}},
265     {"PageRestoreTempPage"}, {0}},
266     {"pairingheap_free"}, {0}},
267     {"parallel_vacuum_end"}, {0}},
268     {"pfree"}, {0}},
269     {"pgfnames_cleanup"}, {0}},
270     {"pg_free"}, {0}},
271     {"pgp_cfb_free"}, {0}},
272     {"pgp_free"}, {0}},
273     {"pgp_key_free"}, {0}},
274     {"pgp_mpi_free"}, {0}},
275     {"PGTYPESchar_free"}, {0}},
276     {"PGTYPESdate_free"}, {0}},
277     {"PGTYPESdecimal_free"}, {0}},
278     {"PGTYPESinterval_free"}, {0}},
279     {"PGTYPESnumeric_free"}, {0}},
280     {"pg_tzenumerate_end"}, {0}},
281     {"pg_xml_done"}, {0}},
282     {"PortalDrop"}, {0}},
283     {"porter_ISO_8859_1_close_env"}, {0}},
284     {"porter_UTF_8_close_env"}, {0}},
285     {"portuguese_ISO_8859_1_close_env"}, {0}},
286     {"portuguese_UTF_8_close_env"}, {0}},
287     {"PQconninfoFree"}, {0}},
288     {"PQfinish"}, {0}},
289     {"PQfreeCancel"}, {0}},
290     {"PQfreemem"}, {0}},
291     {"PQfreeNotify"}, {0}},
292     {"psql_scan_destroy"}, {0}},
293     {"pullf_free"}, {0}},
294     {"pushf_free"}, {0}},
295     {"pushf_free_all"}, {0}},
296     {"QTNFree"}, {0}},
297     {"read_stream_end"}, {0}},
298     {"ReorderBufferReturnChange"}, {1}},
299     {"ReorderBufferReturnRelids"}, {1}},
300     {"ReorderBufferReturnTupleBuf"}, {0}},
301     {"ResourceOwnerDelete"}, {0}},
302     {"restorePsetInfo"}, {1}},
303     {"romanian_ISO_8859_2_close_env"}, {0}},
304     {"romanian_UTF_8_close_env"}, {0}},
305     {"russian_KOI8_R_close_env"}, {0}},
306     {"russian_UTF_8_close_env"}, {0}},
307     {"serbian_UTF_8_close_env"}, {0}},
308     {"shm_mq_detach"}, {0}},
309     {"SlabDelete"}, {0}},
310     {"SN_close_env"}, {0}},
311     {"spanish_ISO_8859_1_close_env"}, {0}},
312     {"spanish_UTF_8_close_env"}, {0}},
313     {"SPI_cursor_close"}, {0}},
314     {"SPI_freetuple"}, {0}},
315     {"SPI_pfree"}, {0}},
316     {"swedish_ISO_8859_1_close_env"}, {0}},

```

```

317     {"swedish_UTF_8_close_env"}, {0}},
318     {"systable_endscan"}, {0}},
319     {"systable_endscan_ordered"}, {0}},
320     {"tamil_UTF_8_close_env"}, {0}},
321     {"tbn_end_iterate"}, {0}},
322     {"tbn_end_shared_iterate"}, {0}},
323     {"tbn_free"}, {0}},
324     {"TidStoreDestroy"}, {0}},
325     {"TidStoreDetach"}, {0}},
326     {"TidStoreEndIterate"}, {0}},
327     {"toast_close_indexes"}, {0}},
328     {"tuplestore_end"}, {0}},
329     {"turkish_UTF_8_close_env"}, {0}},
330     {"updateAclDependencies"}, {5}},
331     {"updateAclDependencies"}, {7}},
332     {"updateInitAclDependencies"}, {4}},
333     {"updateInitAclDependencies"}, {6}},
334     {"vac_close_indexes"}, {1}},
335     {"XLogPrefetcherFree"}, {0}},
336     {"XLogReaderFree"}, {0}},
337     {"yiddish_UTF_8_close_env"}, {0}},
338 };
339
340 // DEFINE HELPER STRUCTURES AND FUNCTIONS FOR THE DEPENDENT MAP
341
342 // This struct contains information about a dependent function.
343 struct DependencyInfo{
344     // The position of the freed argument
345     unsigned int positionFreed;
346     // The condition for making this function strict
347     std::function<Tristate(CallEvent &, CheckerContext &)> isFreeing;
348     // Defines whether the function becomes arbitrary when isFreeing is false
349     bool isArbitrary = false;
350
351     // Default constructor
352     DependencyInfo() : positionFreed(0), isFreeing([](CallEvent &,
353     CheckerContext &) { return False; }) {}
354
355     DependencyInfo(int positionFreed, bool isArbitrary,
356     std::function<Tristate(CallEvent &, CheckerContext &)> f)
357     : positionFreed(positionFreed), isFreeing(f), isArbitrary(isArbitrary) {}
358 };
359
360 // Method to retrieve a field of a structure represented by the symbolic value
361 // via its field name.
362 SVal getFieldSVal(CheckerContext &C, SVal val, std::string fieldName){
363
364     ProgramStateRef state = C.getState();
365     const MemRegion *baseRegion = val.getAsRegion();
366     if (baseRegion == nullptr)
367         return UndefinedVal();
368     // cast to TypedValueRegion to get the struct type
369     const TypedValueRegion *typedRegion = dyn_cast<TypedValueRegion>(baseRegion);
370     if (!typedRegion)
371         return UndefinedVal();
372     // retrieve the type to detect objects like structs and retrieve the
373     // declaration of the record
374     QualType structType = typedRegion->getValueType();
375     const RecordType *recordType = structType->getAsStructureType();
376     if (!recordType)
377         return UndefinedVal();
378     const RecordDecl *recordDecl = recordType->getDecl();
379     if (!recordDecl)
380         return UndefinedVal();
381
382     // iterate over the field declarations in the record to find the field

```

```

383     for (const FieldDecl *field : recordDecl->fields()) {
384         // we found it, return the field as a symbolic value
385         if (field->getName() == fieldName) {
386             const FieldRegion *fieldRegion = C.getSValBuilder().getRegionManager()
387                                     .getFieldRegion(field, typedRegion);
388             return state->getSVal(fieldRegion);
389         }
390     }
391
392     // field has not been found
393     return UndefinedVal();
394 }
395
396 // Check whether a symbolic value represents a concrete integer and satisfies
397 // the conditions defined by the comparator. The comparator should be defined as
398 // follows: [](const llvm::ASPInt &x){ condition -> e.g. return x==0; }
399 template<typename Comparator>
400 Tristate checkConcreteInt(SVal SValToCheck, Comparator comparator){
401     // if the value is a concrete integer, compare it, return Undefined otherwise
402     if (std::optional<nonloc::ConcreteInt> intVal =
403         SValToCheck.getAs<nonloc::ConcreteInt>()) {
404         const llvm::APInt &val = intVal->getValue();
405         if (comparator(val))
406             return True;
407         else
408             return False;
409     } else {
410         return Undefined;
411     }
412 }
413
414 // helper function for debugging SVals
415 void printSVal(const SVal &sval) {
416     if (auto concreteInt = sval.getAs<nonloc::ConcreteInt>()) {
417         llvm::errs() << "ConcreteInt: " << concreteInt->getValue() << "\n";
418     } else if (auto symbol = sval.getAs<nonloc::SymbolVal>()) {
419         llvm::errs() << "SymbolVal: ";
420         symbol->getSymbol()->dump();
421     } else if (auto loc = sval.getAs<Loc>()) {
422         llvm::errs() << "Location: ";
423         loc->dump();
424     } else if (sval.isUndef()) {
425         llvm::errs() << "Undefined\n";
426     } else if (sval.isUnknown()) {
427         llvm::errs() << "Unknown\n";
428     } else {
429         llvm::errs() << "Other SVal type\n";
430         sval.dump();
431     }
432 }
433
434 // Map holding all dependent functions: maps function names to DependencyInfo.
435 // The callback to isFreeing will always extract the parameter the free is
436 // dependent on and determine the condition. When the condition can not be
437 // determined, Undefined is returned. For clarity reasons before every entry a
438 // comment briefly describes what we check for.
439 // To better understand what is happening here, it is highly recommended to look
440 // at the source code of the functions at https://github.com/postgres/postgres or
441 // https://doxygen.postgresql.org/.
442 llvm::StringMap<DependencyInfo> DependentMap{
443     // 'b' == NULL'
444     {"bms_int_members", DependencyInfo(0, true, [](CallEvent &Call,
445                                                     CheckerContext &C){
446         if (Call.getNumArgs() < 2) return Undefined;
447         SVal val = Call.getArgSVal(1);
448         if (val.isUnknownOrUndef()) return Undefined;

```

```

449     if (val.isZeroConstant()) return True;
450     return False;
451   }},
452   // 'b == NULL'
453   {"bms_replace_members", DependencyInfo(0, true, [])(CallEvent &Call,
454                                           CheckerContext &C){
455     if (Call.getNumArgs() < 2) return Undefined;
456     SVal val = Call.getArgSVal(1);
457     if (val.isUnknownOrUndef()) return Undefined;
458     if (val.isZeroConstant()) return True;
459     return False;
460   }},
461   // 'mode != 0'
462   {"dump_variables", DependencyInfo(0, false, [])(CallEvent &Call,
463                                           CheckerContext &C){
464     if (Call.getNumArgs() < 2) return Undefined;
465     SVal mode = Call.getArgSVal(1);
466     if (mode.isUnknownOrUndef())
467       return Undefined;
468     return checkConcreteInt(mode, [])(const llvm::APInt &a){return a != 0;});
469   }},
470   // 'shouldFree != 0'
471   // never return True -> we can only tell, that the parameter is NOT freed
472   // -> if we wanted to check this condition we would have to simulate the tuple
473   // creation to check the tuple type, which is possible but not implemented yet
474   {"ExecForceStoreMinimalTuple", DependencyInfo(0, false, [])(CallEvent &Call,
475                                           CheckerContext &C){
476     if (Call.getNumArgs() < 3) return Undefined;
477     SVal shouldFree = Call.getArgSVal(2);
478     if (shouldFree.isUnknownOrUndef()) return Undefined;
479     Tristate notFreeing = checkConcreteInt(shouldFree, [])(const llvm::APInt &a){
480                                                         return a == 0;});
481     if (notFreeing == True) return False;
482     return Undefined;
483   }},
484   // 'shouldFree != 0'
485   // never return True -> see note in last entry
486   {"ExecForceStoreHeapTuple", DependencyInfo(0, false, [])(CallEvent &Call,
487                                           CheckerContext &C){
488     if (Call.getNumArgs() < 3) return Undefined;
489     SVal shouldFree = Call.getArgSVal(2);
490     if (shouldFree.isUnknownOrUndef()) return Undefined;
491     Tristate notFreeing = checkConcreteInt(shouldFree, [])(const llvm::APInt &a)
492                                                         {return a == 0;});
493     if (notFreeing == True) return False;
494     return Undefined;
495   }},
496   // 'shouldFree != 0'
497   {"ExecResetTupleTable", DependencyInfo(0, false, [])(CallEvent &Call,
498                                           CheckerContext &C){
499     if (Call.getNumArgs() < 2) return Undefined;
500     SVal shouldFree = Call.getArgSVal(1);
501     if (shouldFree.isUnknownOrUndef()) return Undefined;
502     return checkConcreteInt(shouldFree, [])(const llvm::APInt &a){return a != 0;});
503   }},
504   // 'flags[0] != 0'
505   {"freeJsonLexContext", DependencyInfo(0, false, [])(CallEvent &Call,
506                                           CheckerContext &C){
507     if (Call.getNumArgs() < 1) return Undefined;
508     SVal lexContext = Call.getArgSVal(0);
509     if (lexContext.isUnknownOrUndef()) return Undefined;
510     SVal flags = getFieldSVal(C, lexContext, "flags");
511     if (flags.isUnknownOrUndef()) return Undefined;
512     return checkConcreteInt(flags, [])(const llvm::APInt &a){return a[0];});
513   }},
514   // 'clear != 0'

```

```

515 {"pgfdw_report_error", DependencyInfo(1, false, [])(CallEvent &Call,
516                                     CheckerContext &C){
517     if (Call.getNumArgs() < 4) return Undefined;
518     SVal clear = Call.getArgSVal(3);
519     if (clear.isUnknownOrUndef()) return Undefined;
520     return checkConcreteInt(clear, [])(const llvm::APSIInt &a){return a != 0;});
521 }},
522 // '(const PGresult *) res != &OOM_result'
523 {"PQclear", DependencyInfo(0, false, [])(CallEvent &Call, CheckerContext &C){
524     if (Call.getNumArgs() != 1) return Undefined;
525     SVal res = Call.getArgSVal(0);
526     if (res.isUnknownOrUndef()) return Undefined;
527     ProgramStateRef State = C.getState();
528     if (auto Loc = res.getAs<loc::MemRegionVal>()) {
529         const MemRegion *Region = Loc->getRegion();
530         // check if this is the OOM_result global variable
531         if (auto *VR = dyn_cast<VarRegion>(Region))
532             if (VR->getDecl()->getName() == "OOM_result")
533                 return False;
534         if (Region->getKindStr() == "SymbolicRegion")// actual value is not known
535             return Undefined;
536         return True; // we are confident we freed something
537     }
538     return Undefined; // we don't know what we are dealing with
539 }},
540 // '*destsz == 0 || lex > *destsz'
541 // We never return True since the function is reallocating!
542 {"brin_copy_tuple", DependencyInfo(2, false, [])(CallEvent &Call,
543                                     CheckerContext &C){
544     if (Call.getNumArgs() < 4) return Undefined;
545     // get the pointer to destsz
546     SVal destszValPtr = Call.getArgSVal(3);
547                                     ProgramStateRef State = C.getState();
548     // dereference the pointer
549     SVal destszVal = State->getSVal(destszValPtr.castAs<Loc>(),
550                                     C.getASTContext().getSizeType());
551     if (destszVal.isUnknownOrUndef()) return Undefined;
552
553     // check *destsz == 0
554     auto checkDestszResult = checkConcreteInt(
555         destszVal, [])(const llvm::APSIInt &a){return a == 0;});
556
557     if (checkDestszResult == True || checkDestszResult == Undefined)
558         return Undefined;
559
560     // get len
561     SVal lenVal = Call.getArgSVal(1);
562     if (lenVal.isUnknownOrUndef()) return Undefined;
563
564     // extract concrete integers and compare them
565     std::optional<nonloc::ConcreteInt> lenConcrete = lenVal.getAs<nonloc::ConcreteInt>();
566     std::optional<nonloc::ConcreteInt> destszConcrete = destszVal.getAs<nonloc::ConcreteInt>();
567     const llvm::APSIInt &lenAPS = lenConcrete->getValue();
568     const llvm::APSIInt &destszAPS = destszConcrete->getValue();
569     int cmpResult = lenAPS.compareValues(lenAPS, destszAPS);
570
571     // len > *destsz
572     if (cmpResult > 0)
573         return Undefined;
574     // no reallocation detected
575     return False;
576 }},
577 // WE IGNORE THE REST FOR NOW: the checker will not actually see the values,
578 // associated with the relevant fields, since they involve counting of

```

```

579 // characteristics like 'size' that is handled internally. We would have
580 // to simulate this behavior, which is possible but not implemented yet.
581 // As is the inclusion of these functions leads to false positives.
582 // 'tdrefcount == 1'
583 {"DecrTupleDescRefCount", DependencyInfo(0, false, [])(CallEvent &Call,
584                                     CheckerContext &C){
585     return False;
586 }},
587 // 'pstate->numWorkers == 1'
588 {"ParallelBackupEnd,ParallelState", DependencyInfo(1, false, [])(CallEvent &Call,
589                                     CheckerContext &C){
590     return False;
591 }},
592 // 'snap->active_count == 1'
593 {"SnapBuildSnapDecRefCount", DependencyInfo(0, false, [])(CallEvent &Call,
594                                     CheckerContext &C){
595     return False;
596 }},
597 // 'snapshot->regd_count == 0 && snapshot->active_count == 0'
598 {"UnregisterSnapshotFromOwner", DependencyInfo(0, false, [])(CallEvent &Call,
599                                     CheckerContext &C){
600     return False;
601 }},
602 // 'snapshot->regd_count == 0 && snapshot->active_count == 0'
603 {"UnregisterSnapshot", DependencyInfo(0, false, [])(CallEvent &Call,
604                                     CheckerContext &C){
605     return False;
606 }},
607 };
608
609 // Map for arbitrary functions: maps the function name to the index being
610 // (potentially) freed
611 llvm::StringMap<unsigned int> ArbitraryMap{
612     // C function
613     {"realloc"}, {0}},
614     {"calloc"}, {0}},
615     // PostgreSQL-specific functions
616     {"add_partial_path"}, {1}},
617     {"add_path"}, {1}},
618     {"bms_add_member"}, {0}},
619     {"bms_add_members"}, {0}},
620     {"bms_add_range"}, {0}},
621     {"bms_del_member"}, {0}},
622     {"bms_del_members"}, {0}},
623     {"bms_join"}, {0}},
624     {"bms_join"}, {1}},
625     {"bms_replace_members"}, {0}},
626     {"CheckAttributeType"}, {3}},
627     {"core_yyrealloc"}, {0}},
628     {"ecpg_check_PQresult"}, {0}},
629     {"ecpg_realloc"}, {0}},
630     {"findsubquery"}, {0}},
631     {"FreeDir"}, {0}},
632     {"gistjoinvector"}, {0}},
633     {"index_close"}, {0}},
634     {"InsertPgAttributeTuples"}, {4}},
635     {"_int_unique"}, {0}},
636     {"resize_intArrayType"}, {0}},
637     {"jsonpath_yyrealloc"}, {0}},
638     {"list_delete"}, {0}},
639     {"list_delete_cell"}, {0}},
640     {"list_delete_first"}, {0}},
641     {"list_delete_first_n"}, {0}},
642     {"list_delete_int"}, {0}},
643     {"list_delete_last"}, {0}},
644     {"list_delete_nth_cell"}, {0}},

```

```

645     {"list_delete_oid"}, {0}},
646     {"list_delete_ptr"}, {0}},
647     {"ParallelSlotsAdoptConn"}, {1}},
648     {"pa_xact_finish"}, {0}},
649     {"pg_realloc"}, {0}},
650     {"relation_close"}, {0}},
651     {"RelationClose"}, {0}},
652     {"relation_is_updatable"}, {1}},
653     {"relation_is_updatable"}, {3}},
654     {"ReleaseCatCacheList"}, {0}},
655     {"ReorderBufferQueueChange"}, {3}},
656     {"repalloc"}, {0}},
657     {"repalloc0"}, {0}},
658     {"rewrite_heap_tuple"}, {2}},
659     {"sequence_close"}, {0}},
660     {"SPI_repalloc"}, {0}},
661     {"table_close"}, {0}},
662     {"TableCommandResultHandler"}, {0}},
663     {"yyrealloc"}, {0}},
664 };
665
666 // Set containing functions where the return value
667 // should be reassigned.
668 llvm::StringSet<> ReassigningSet{
669     {"bms_add_member"},
670     {"bms_add_members"},
671     {"bms_add_range"},
672     {"bms_del_member"},
673     {"bms_del_members"},
674     {"bms_int_members"},
675     {"bms_join"},
676     {"bms_replace_members"},
677     {"brin_copy_tuple"},
678     {"findsubquery"},
679     {"list_delete_cell"},
680     {"list_delete_first"},
681     {"list_delete_first_n"},
682     {"list_delete_int"},
683     {"list_delete_last"},
684     {"list_delete"},
685     {"list_delete_nth_cell"},
686     {"list_delete_oid"},
687     {"list_delete_ptr"},
688 };
689
690 // Set containing functions where the return value
691 // signifies whether the parameter has been freed.
692 llvm::StringSet<> BooleanSet{
693     {"ecpg_check_PQresult"},
694 };
695
696 // Set of functions to be ignored for double-free
697 llvm::StringSet<> IgnoreDouble{
698 };
699
700 // Map of functions to be ignored for use-after-free
701 llvm::StringMap<unsigned int> IgnoreUse{
702 };
703
704 // Map of functions to be ignored altogether
705 llvm::StringSet<> IgnoreAlltogether{
706 };
707
708 // Helper function: retrieves the variable declaration from an expression and
709 // returns the name.
710 std::string getNameFromExpression(const Expr * Expr){

```

```

711     if (!Expr)
712         return "";
713     if (const DeclRefExpr *DRE = dyn_cast<DeclRefExpr>(Expr)) {
714         if (const ValueDecl *VD = DRE->getDecl()) {
715             return VD->getNameAsString();
716         }
717     }
718     // recurse until there are no casts
719     else if (const CastExpr *CE = dyn_cast<CastExpr>(Expr)) {
720         return getNameFromExpression(CE->getSubExpr());
721     }
722     return "";
723 }
724
725 void PostgresChecker::HandleUseAfterFree(CheckerContext &C, SourceRange Range,
726                                         const PGRefState *RS,
727                                         std::string varName,
728                                         Category Cat) const{
729
730     BugType *BT;
731     // the message to be emitted as a warning
732     std::string message;
733     switch(Cat){
734     case (Strict):
735         message = "Attempt to use released memory";
736         if (!BT_Free_Strict)
737             BT_Free_Strict.reset(new BugType(this, message));
738         BT = BT_Free_Strict.get();
739         break;
740         // we should not be here, this could be handled better (maybe introduce
741         // another enum entry?)
742     case (Dependent):
743         return;
744     case (Arbitrary):
745         message = "Attempt to use potentially released memory";
746         if (!BT_Free_Arbitrary)
747             BT_Free_Arbitrary.reset(new BugType(this, message));
748         BT = BT_Free_Arbitrary.get();
749         break;
750     }
751     // add the variable to the message name if it exists
752     if (!varName.empty())
753         message = message + ": " + varName;
754     emitReport(RS, BT, C, message);
755 }
756
757 void PostgresChecker::HandleDoubleFree(CheckerContext &C, SourceRange Range,
758                                       const PGRefState *RS, std::string varName,
759                                       Category FirstFreeCat, Category Cat) const{
760
761     BugType *BT;
762     // the message to be emitted as a warning
763     std::string message;
764     switch(Cat){
765     case (Strict):
766         if (FirstFreeCat == Arbitrary)
767             message = "Attempt to free potentially released memory";
768         else
769             message = "Attempt to free released memory";
770         if (!BT_Free_Strict)
771             BT_Free_Strict.reset(new BugType(this, message));
772         BT = BT_Free_Strict.get();
773         break;
774         // we should not be here, this could be handled better (maybe introduce
775         // another enum entry?)
776     case (Dependent):

```

```

777     return;
778 case (Arbitrary):
779     if (FirstFreeCat == Arbitrary)
780         message = "Possible attempt to free potentially released memory";
781     else
782         message = "Possible attempt to free released memory";
783     if (!BT_Free_Arbitrary)
784         BT_Free_Arbitrary.reset(new BugType(this, message));
785     BT = BT_Free_Arbitrary.get();
786     break;
787 }
788 // add the variable to the message name if it exists
789 if (!varName.empty())
790     message = message + ": " + varName;
791 emitReport(RS, BT, C, message);
792 }
793
794 // If the function is freeing -> call HandleFree; otherwise, call
795 // checkUseAfterFree
796 void PostgresChecker::checkPreCall(const CallEvent &Call, CheckerContext &C) const {
797
798     const FunctionDecl *FD = dyn_cast_or_null<FunctionDecl>(Call.getDecl());
799     if (!FD)
800         return;
801     // if the function has a body, we return. If there is a call to a freeing
802     // function inside we see it anyways
803     if (FD->hasBody())
804         return;
805
806     if (IgnoreAlltogether.contains(FD->getNameAsString()))
807         return;
808
809     // look up the function name in the maps
810     if (StrictMap.contains(FD->getName())
811         || DependentMap.contains(FD->getName())
812         || ArbitraryMap.contains(FD->getName())){
813
814         // check whether we have access to a valid expression (and not, e.g., void)
815         if (!Call.getOriginExpr())
816             return;
817
818         // handle strict functions
819         if (StrictMap.contains(FD->getName())){
820             HandleFree(Call, C, Strict);
821             return;
822         }
823         // handle dependent functions
824         if (DependentMap.contains(FD->getName())){
825             HandleFree(Call, C, Dependent);
826             return;
827         }
828         // handle arbitrary functions
829         if (ArbitraryMap.contains(FD->getName())){
830             HandleFree(Call, C, Arbitrary);
831             return;
832         }
833         // this is pure paranoia
834         return;
835     }
836
837     // we encountered a non-freeing function, for every parameter retrieve the
838     // symbolic value and, if it is a location, check for use-after-free
839     for (unsigned I = 0, E = Call.getNumArgs(); I != E; ++I) {
840         // ignore special cases

```

```

842     if (IgnoreUse.contains(FD->getNameAsString()) && I == IgnoreUse.lookup(FD->
      getNameAsString()))
843         continue;
844     SVal ArgSVal = Call.getArgSVal(I);
845     if (isa<Loc>(ArgSVal)) {
846         SymbolRef Sym = ArgSVal.getAsSymbol();
847         if (!Sym)
848             continue;
849         checkUseAfterFree(Sym, getNameFromExpression(Call.getArgExpr(I)), C,
      Call.getArgExpr(I));
851     }
852 }
853 }
854
855 void PostgresChecker::HandleFree(const CallEvent &Call, CheckerContext &C,
      Category Cat) const{
856
857     // no argument means nothing to check
858     if (Call.getNumArgs() == 0)
859         return;
860     const FunctionDecl *FD = dyn_cast_or_null<FunctionDecl>(Call.getDecl());
861     if (!FD)
862         return;
863
864     SVal ArgVal;
865     // keep track of the expression we found to add the name of the variable to
866     // a potential warning
867     const Expr *ArgExpr;
868     switch (Cat){
869     // strict function -> get the position of the argument and retrieve it
870     case (Strict): {
871         auto Position = StrictMap.lookup(FD->getName());
872         if (Call.getNumArgs() <= Position)
873             return;
874         ArgExpr = Call.getArgExpr(Position);
875         if (!ArgExpr)
876             return;
877         ArgVal = C.getSVal(ArgExpr);
878         break;
879     }
880     // dependent functions:
881     // - retrieve the DependencyInfo
882     // - call Info.isFreeing(): if True, handle function as strict
883     // - if Undefined or not freeing and the Info.isArbitrary fallback option
884     //   is set, treat the function as arbitrary
885     // - otherwise do nothing
886     // - if we do something, get position of the argument and retrieve it
887     case (Dependent): {
888         auto It = DependentMap.find(FD->getName());
889         if (It != DependentMap.end()) {
890             DependencyInfo Info = It->second;
891             if (Call.getNumArgs() <= Info.positionFreed)
892                 return;
893             auto result = Info.isFreeing(const_cast<CallEvent &>(Call), C);
894             //function is freeing
895             if (result == True){
896                 Cat = Strict;
897                 // function is not freeing
898             } else if (result == False){
899                 //fall back to arbitrary?
900                 if (Info.isArbitrary)
901                     Cat = Arbitrary;
902                 //function does not free at all
903                 else
904                     return;
905             }
906             // ifFreeing could not be determined

```

```

907         } else
908             Cat = Arbitrary;
909         ArgExpr = Call.getArgExpr(Info.positionFreed);
910         if (!ArgExpr)
911             return;
912         ArgVal = C.getSVal(ArgExpr);
913         break;
914     }
915     // we did not retrieve anything
916     return;
917 }
918 // arbitrary function -> get the position of the argument and retrieve it
919 case (Arbitrary): {
920     auto Position = ArbitraryMap.lookup(FD->getName());
921     if (Call.getNumArgs() <= Position)
922         return;
923     ArgExpr = Call.getArgExpr(Position);
924     if (!ArgExpr)
925         return;
926     ArgVal = C.getSVal(ArgExpr);
927     break; // doing this for safety reasons in case enum gets expanded
928 }
929 }
930
931 if (ArgVal.isUnknownOrUndef())
932     return;
933 const MemRegion *R = ArgVal.getAsRegion();
934 if (!R)
935     return;
936 // get parent expression to have access to the source location
937 const Expr *ParentExpr = Call.getOriginExpr();
938 if (!ParentExpr)
939     return;
940
941 // remove casts and retrieve the base symbolic region
942 R = R->StripCasts();
943 const SymbolicRegion *SrBase = dyn_cast<SymbolicRegion>(R->getBaseRegion());
944 if (!SrBase)
945     return;
946
947 ProgramStateRef State = C.getState();
948 if (!State)
949     return;
950 // look up the symbol reference in the RegionStatePG map. If found, handle
951 // double-free
952 SymbolRef SymBase = SrBase->getSymbol();
953 const PGRState *RsBase = State->get<RegionStatePG>(SymBase);
954 if (RsBase){
955     // if both the old and the new functions are C functions ignore
956     // -> we don't want to get in the way of the MallocChecker
957     if (CMemoryMap.contains(RsBase->getFunction()->getName())
958         && CMemoryMap.contains(FD->getName()))
959         return;
960     // determine the quality of the double-free
961     Category CatFirst = RsBase->isReleased() ? Strict : Arbitrary;
962     if (!IgnoreDouble.contains(FD->getNameAsString()))
963         HandleDoubleFree(C, ParentExpr->getSourceRange(), RsBase,
964             getNameFromExpression(ArgExpr), CatFirst, Cat);
965 }
966
967 // the variable name to be used in a potential warning
968 std::string VarName = getNameFromExpression(ArgExpr);
969
970 // depending on the category of the freeing function set a released or possibly
971 // released state
972 switch (Cat){

```

```

973     case Strict:
974         State = State->set<RegionStatePG>(SymBase, PGRefState::getReleased(
975             ParentExpr, FD, VarName));
976         break;
977     case Dependent:
978         return; // We should never get here (could this be handled better?)
979     case Arbitrary:
980         State = State->set<RegionStatePG>(SymBase, PGRefState::getPossiblyReleased(
981             ParentExpr, FD, VarName));
982         break;
983     default:
984         return;
985 }
986
987 // add the state to the checker context
988 C.addTransition(State);
989 }
990
991 // Check whether the location accessed is a freed symbolic region.
992 void PostgresChecker::checkLocation(SVal l, bool isLoad, const Stmt *S,
993     CheckerContext &C) const {
994     SymbolRef Sym = l.getLocSymbolInBase();
995     if (Sym) {
996         std::string varName = "";
997         // try to extract the variable name from the variable declaration
998         if (auto loc = l.getAs<Loc>()) {
999             if (const MemRegion *region = loc->getAsRegion()) {
1000                 if (const VarRegion *varRegion = dyn_cast<VarRegion>(region)) {
1001                     const VarDecl *varDecl = varRegion->getDecl();
1002                     varName = varDecl->getName();
1003                 }
1004             }
1005         }
1006         checkUseAfterFree(Sym, varName, C, S);
1007     }
1008 }
1009
1010 // Check for use-after-free for return value.
1011 void PostgresChecker::checkPreStmt(const ReturnStmt *S, CheckerContext &C) const {
1012     if (!S)
1013         return;
1014     const Expr *E = S->getRetVal();
1015     if (!E)
1016         return;
1017     SVal RetVal = C.getSVal(E);
1018     SymbolRef Sym = RetVal.getAsSymbol();
1019     // we might deal with composite objects -> look at the region, if it is a field
1020     // or array element, retrieve the base region and extract the symbol
1021     if (!Sym)
1022         if (const MemRegion *MR = RetVal.getAsRegion())
1023             if (isa<FieldRegion, ElementRegion>(MR))
1024                 if (const SymbolicRegion *BMR =
1025                     dyn_cast<SymbolicRegion>(MR->getBaseRegion()))
1026                     Sym = BMR->getSymbol();
1027     if (Sym)
1028         checkUseAfterFree(Sym, getNameFromExpression(E), C, E);
1029 }
1030
1031 // Check whether the memory region the symbolic reference is pointing to has been
1032 // freed -> look into the RegionStatePG map and handle strict and arbitrary cases
1033 bool PostgresChecker::checkUseAfterFree(SymbolRef Sym, std::string varName,
1034     CheckerContext &C, const Stmt *S) const {
1035     const PGRefState *RS = C.getState()->get<RegionStatePG>(Sym);
1036     // no use-after-free
1037     if (!RS)
1038         return false;

```

```

1039     if (RS && RS->isReleased()) {
1040         HandleUseAfterFree(C, S->getSourceRange(), RS, varName, Strict);
1041         return true;
1042     }
1043     if (RS && RS->isPossiblyReleased()) {
1044         HandleUseAfterFree(C, S->getSourceRange(), RS, varName, Arbitrary);
1045         return true;
1046     }
1047     return false;
1048 }
1049
1050 // Emit a basic bug report with message
1051 void PostgresChecker::emitReport(const PGRState *RS, BugType *BT,
1052                                 CheckerContext &C, std::string message) const{
1053     if (!BT)
1054         return;
1055     // One could also argue, that we should use a fatal error node here for strict
1056     // functions and a non-fatal one for arbitrary ones. Since MallocChecker uses
1057     // non-fatal for 'strict' cases, we follow suite.
1058     ExplodedNode *EN = C.generateNonFatalErrorNode(C.getState(), this);
1059     if (!EN)
1060         return;
1061
1062     // get location information for the current site and create the report
1063     PathDiagnosticLocation PDLoc = PathDiagnosticLocation::create(
1064         EN->getLocation(),
1065         C.getSourceManager()
1066     );
1067     auto R = std::make_unique<BasicBugReport>(*BT, message, PDLoc);
1068
1069     // if, for some reason, the PGRState is invalid, emit a report without
1070     // information about the free site
1071     if (!RS || !RS->getStmt() || !RS->getFunction()){
1072         C.emitReport(std::move(R));
1073         return;
1074     }
1075
1076     // get location information for the free site
1077     PathDiagnosticLocation PDLocOriginal = PathDiagnosticLocation::createBegin(
1078         RS->getStmt(),
1079         C.getSourceManager(),
1080         C.getLocationContext()
1081     );
1082     const FunctionDecl *FD = RS->getFunction();
1083
1084     // if the variable name exists add (varName) to the message
1085     std::string addition = RS->getVarName().empty() ? "" :
1086         (" (" + RS->getVarName() + ")");
1087
1088     // add warning note for reassigning functions
1089     if (ReassigningSet.contains(FD->getNameAsString())){
1090         R->addNote(("The return value should probably be reassigned" + addition),
1091             PDLocOriginal);
1092     } else if (BooleanSet.contains(FD->getNameAsString())){
1093         R->addNote(("The return value should probably be checked"), PDLocOriginal);
1094         // add a note about the free site to the report
1095     } else {
1096         R->addNote("Freeing function" + (FD ? (" (" + FD->getNameAsString() + addition)
1097             : "<unknown>"), PDLocOriginal);
1098     }
1099
1100     C.emitReport(std::move(R));
1101 }
1102
1103 namespace clang {

```

```

1104 namespace ento {
1105 // Register the checker within the llvm-project context
1106 //void registerPostgresChecker(CheckerManager &mgr) {
1107 //mgr.registerChecker<PostgresChecker>();
1108 //}
1109
1110 //bool shouldRegisterPostgresChecker(const CheckerManager &mgr) {
1111 //return true;
1112 //}
1113 // Register the checker as an extern .so file:
1114 // - uncomment all the lines below
1115 // - comment out the line above up to but not including the namespaces
1116 extern "C" void clang_registerCheckers(CheckerRegistry &registry) {
1117     registry.addChecker<PostgresChecker>(
1118         "postgres.PostgresChecker",
1119         "Checks for use-after-free and double-free in PostgreSQL",
1120         "");
1121 }
1122 extern "C"
1123 const char clang_analyzerAPIVersionString[] = CLANG_ANALYZER_API_VERSION_STRING;
1124
1125 } // namespace ento
1126 } // namespace clang

```

B.2 TESTS

Listing B.2: Tests used to validate the checker

```

1 // RUN: %clang_analyze_cc1 -Wno-strict-prototypes -Wno-error=implicit-int -verify %s \
2 // RUN: -analyzer-checker=postgres.PostgresChecker
3
4 #include "Inputs/system-header-simulator.h"
5
6 void clang_analyzer_eval(int);
7 void clang_analyzer_dump(int);
8 void clang_analyzer_dumpExtent(void *);
9
10 // Without -fms-compatibility, wchar_t isn't a builtin type. MSVC defines
11 // _WCHAR_T_DEFINED if wchar_t is available. Microsoft recommends that you use
12 // the builtin type: "Using the typedef version can cause portability
13 // problems", but we're ok here because we're not actually running anything.
14 // Also of note is this cryptic warning: "The wchar_t type is not supported
15 // when you compile C code".
16 //
17 // See the docs for more:
18 // https://msdn.microsoft.com/en-us/library/dh8che7s.aspx
19 #if !defined(_WCHAR_T_DEFINED)
20 // "Microsoft implements wchar_t as a two-byte unsigned value"
21 typedef unsigned short wchar_t;
22 #define _WCHAR_T_DEFINED
23 #endif // !defined(_WCHAR_T_DEFINED)
24
25 typedef __typeof(sizeof(int)) size_t;
26 void *malloc(size_t);
27 void *calloc(size_t);
28 void free(void *);
29 void pfree(void *);
30 void *realloc(void *ptr, size_t size);
31 void *repalloc(void *ptr, size_t size);
32 char *strdup(const char *s);
33 wchar_t *wcsdup(const wchar_t *s);
34 char *strndup(const char *s, size_t n);

```

```

35 int memcmp(const void *s1, const void *s2, size_t n);
36
37 // Windows variants
38 char *_strdup(const char *strSource);
39 wchar_t *_wcsdup(const wchar_t *strSource);
40
41 void myfoo(int *p);
42 void myfoo(int p);
43 char *fooRetPtr(void);
44
45 //HANDLE NORMAL FREE
46 void normal_free(void){
47     int *p = malloc(12);
48     free(p); // expected-note{{Freeing function: free (p)}}
49     pfree(p); // expected-warning{{Attempt to free released memory: p}}
50 }
51 //HANDLE NORMAL REALLOC
52 //PFREE
53 void f2(void) {
54     int *p = malloc(12);
55     pfree(p); // expected-note{{Freeing function: pfree (p)}}
56     pfree(p); // expected-warning{{Attempt to free released memory: p}}
57 }
58
59 void f7(void) {
60     char *x = (char*) malloc(4);
61     pfree(x); // expected-note{{Freeing function: pfree (x)}}
62     x[0] = 'a'; // expected-warning{{Attempt to use released memory}}
63 }
64
65 void f8(void) {
66     char *x = (char*) malloc(4);
67     pfree(x); // expected-note{{Freeing function: pfree (x)}}
68     char *y = strdup(x, 4); // expected-warning{{Attempt to use released memory: x}}
69 }
70
71 void paramFree(int *p) {
72     myfoo(p);
73     pfree(p); // expected-note{{Freeing function: pfree (p)}}
74     myfoo(p); // expected-warning {{Attempt to use released memory: p}}
75 }
76
77
78 void mallocEscapeFree(void) {
79     int *p = malloc(12);
80     myfoo(p);
81     pfree(p);
82 }
83
84 void mallocEscapeFreeFree(void) {
85     int *p = malloc(12);
86     myfoo(p);
87     pfree(p); // expected-note{{Freeing function: pfree (p)}}
88     pfree(p); // expected-warning{{Attempt to free released memory: p}}
89 }
90
91 void mallocEscapeFreeUse(void) {
92     int *p = malloc(12);
93     myfoo(p);
94     pfree(p); // expected-note{{Freeing function: pfree (p)}}
95     myfoo(p); // expected-warning{{Attempt to use released memory: p}}
96 }
97
98 int *myalloc(void);
99 void myalloc2(int **p);
100

```

```

101 void mallocEscapeFreeCustomAlloc(void) {
102     int *p = palloc(12);
103     myfoo(p);
104     pfree(p);
105     p = myalloc();
106     pfree(p); // no warning
107 }
108
109 //avoid false positives
110 void mallocEscapeFreeCustomAlloc2(void) {
111     int *p = palloc(12);
112     myfoo(p);
113     pfree(p);
114     myalloc2(&p);
115     pfree(p); // no warning
116 }
117
118 void mallocBindFreeUse(void) {
119     int *x = palloc(12);
120     int *y = x;
121     pfree(y); // expected-note{{Freeing function: pfree (y)}}
122     myfoo(x); // expected-warning{{Attempt to use released memory: x}}
123 }
124
125 void mallocFreeMalloc(void) {
126     int *p = palloc(12);
127     pfree(p);
128     p = palloc(12);
129     pfree(p);
130 }
131
132 void mallocFreeUse_params(void) {
133     int *p = palloc(12);
134     pfree(p); // expected-note{{Freeing function: pfree (p)}}
135     myfoo(p); //expected-warning{{Attempt to use released memory: p}}
136 }
137
138 void mallocFreeUse_params2(void) {
139     int *p = palloc(12);
140     pfree(p); // expected-note{{Freeing function: pfree (p)}}
141     myfoo(int(*p)); //expected-warning{{Attempt to use released memory}}
142 }
143
144 struct StructWithInt {
145     int g;
146 };
147
148 int *mallocReturnFreed(void) {
149     int *p = palloc(12);
150     pfree(p); // expected-note{{Freeing function: pfree (p)}}
151     return p; // expected-warning {{Attempt to use released memory: p}}
152 }
153
154 int useAfterFreeStruct(void) {
155     struct StructWithInt *px= palloc(sizeof(struct StructWithInt));
156     px->g = 5;
157     pfree(px); // expected-note{{Freeing function: pfree (px)}}
158     return px->g; // expected-warning {{Attempt to use released memory}}
159 }
160
161 int *GL;
162 struct GLStTy {
163     int *x;
164 };
165
166 struct GLStTy GLS = {0};

```

```

167
168 void GlobalFree(void) {
169     pfree(GL);
170 }
171
172 void GlobalMalloc(void) {
173     GL = palloc(12);
174 }
175
176 void GlobalStructMalloc(void) {
177     int *a = palloc(12);
178     GLS.x = a;
179 }
180
181 void GlobalStructMallocFree(void) {
182     int *a = palloc(12);
183     GLS.x = a;
184     pfree(GLS.x);
185 }
186
187 char *ArrayG[12];
188
189 void globalArrayTest(void) {
190     char *p = (char*)palloc(12);
191     ArrayG[0] = p;
192 }
193
194 void testElemRegion1(void) {
195     char *x = (void*)palloc(2);
196     int *ix = (int*)x;
197     pfree(&(x[0]));
198 }
199
200 void testElemRegion2(int **pp) {
201     int *p = palloc(12);
202     *pp = p;
203     pfree(pp[0]);
204 }
205
206 void testElemRegion3(int **pp) {
207     int *p = palloc(12);
208     *pp = p;
209     pfree(*pp);
210 }
211
212 // Make sure we catch errors when we free in a function which does not allocate
213 // memory.
214 void freeButNoMalloc(int *p, int x){
215     if (x) {
216         pfree(p); // expected-note{{Freeing function: pfree (p)}}
217         //user forgot a return here.
218     }
219     pfree(p); // expected-warning {{Attempt to free released memory: p}}
220 }
221
222 void testOffsetZeroDoubleFree(void) {
223     int *array = palloc(sizeof(int)*2);
224     int *p = &array[0];
225     pfree(p); // expected-note{{Freeing function: pfree (p)}}
226     pfree(&array[0]); // expected-warning{{Attempt to free released memory}}
227 }
228
229 char **_vectorSegments;
230 int _nVectorSegments;
231 void poolFreeC(void* s) {

```

```

232     pfree(s); // no-warning
233 }
234 void freeMemory(void) {
235     while (_nVectorSegments) {
236         poolFreeC(_vectorSegments[_nVectorSegments++]);
237     }
238 }
239
240 struct IntAndPtr {
241     int x;
242     int *p;
243 };
244
245 void constEscape(const void *ptr);
246
247 void testConstEscapeThroughAnotherField(void) {
248     struct IntAndPtr s;
249     s.p = malloc(sizeof(int));
250     constEscape(&(s.x)); // could free s->p!
251 } // no-warning
252
253 // PR15623
254 int testNoCheckerDataPropogationFromLogicalOpOperandToOpResult(void) {
255     char *param = malloc(10);
256     char *value = malloc(10);
257     int ok = (param && value);
258     pfree(param);
259     pfree(value);
260     // Previously we ended up with 'Attempt to use released memory' on return.
261     return ok; // no warning
262 }
263
264 typedef struct {
265     // no members
266 } RelOptInfo;
267 typedef struct {
268     // no members
269 } Path;
270 void add_partial_path(RelOptInfo *parent_rel, Path *new_path);
271 void use_path(Path *path);
272 void arbitrary_use(void){
273     Path *new_path= malloc(sizeof(Path));
274     RelOptInfo *parent_rel= malloc(sizeof(Path));
275     add_partial_path(parent_rel, new_path); // expected-note{{Freeing function:
        add_partial_path (new_path)}}
276     use_path(new_path); // expected-warning{{Attempt to use potentially released memory:
        new_path}}
277 }
278
279 void arbitrary_double_free(void){
280     Path *new_path= malloc(sizeof(Path));
281     RelOptInfo *parent_rel= malloc(sizeof(Path));
282     add_partial_path(parent_rel, new_path); // expected-note{{Freeing function:
        add_partial_path (new_path)}}
283     pfree(new_path); // expected-warning{{Attempt to free potentially released memory:
        new_path}}
284 }
285
286 void arbitrary_potential_double_free(void){
287     Path *new_path= malloc(sizeof(Path));
288     RelOptInfo *parent_rel= malloc(sizeof(Path));
289     pfree(new_path); // expected-note{{Freeing function: pfree (new_path)}}
290     add_partial_path(parent_rel, new_path); // expected-warning{{Possible attempt to
        free released memory: new_path}}
291 }
292

```

```

293 void arbitrary_possibly_potential_double_free(void){
294     Path *new_path= malloc(sizeof(Path));
295     RelOptInfo *parent_rel= malloc(sizeof(Path));
296     add_partial_path(parent_rel, new_path); // expected-note{{Freeing function:
        add_partial_path (new_path)}}
297     add_partial_path(parent_rel, new_path); // expected-warning{{Possible attempt to
        free potentially released memory: new_path}}
298 }
299
300 typedef struct {} Bitmapset;
301 Bitmapset *bms_int_members(Bitmapset *a, const Bitmapset *b);
302 void use_bms(Bitmapset *a);
303
304 void bitmapset(void){
305     Bitmapset *a = malloc(sizeof(Bitmapset));
306     Bitmapset *b = NULL;
307     bms_int_members(a, b); // expected-note{{The return value should probably be
        reassigned (a)}}
308     use_bms(a); // expected-warning{{Attempt to use released memory: a}}
309
310     a = malloc(sizeof(Bitmapset));
311     b = malloc(sizeof(Bitmapset));
312     bms_int_members(a, b); // expected-note{{The return value should probably be
        reassigned (a)}}
313     use_bms(a); // expected-warning{{Attempt to use potentially released memory: a}}
314 }
315
316 void bitmapset_argument(Bitmapset *b){
317     Bitmapset *a = malloc(sizeof(Bitmapset));
318     bms_int_members(a, b); // expected-note{{The return value should probably be
        reassigned (a)}}
319     use_bms(a); // expected-warning{{Attempt to use potentially released memory: a}}
320 }
321
322 struct arguments {};
323 void dump_variables(struct arguments *list, int mode);
324 void use_arguments(struct arguments *list);
325 void dump_vars(void){
326     struct arguments *list = malloc(sizeof(struct arguments));
327     dump_variables(list, 0);
328     use_arguments(list);
329     dump_variables(list, 1); // expected-note{{Freeing function: dump_variables (list)}}
330
331     use_arguments(list); // expected-warning{{Attempt to use released memory: list}}
332 }
333 void dump_vars_argument(int mode){
334     struct arguments *list = malloc(sizeof(struct arguments));
335     dump_variables(list, mode); // expected-note{{Freeing function: dump_variables (
        list)}}
336     use_arguments(list); // expected-warning{{Attempt to use potentially released
        memory: list}}
337 }
338
339 typedef int bool;
340 #define true 1
341 #define false 0
342
343 typedef struct {} TupleTableSlot;
344 typedef struct {} MinimalTupleStruct;
345 typedef MinimalTupleStruct * MinimalTuple;
346 typedef struct {} HeapTupleStruct;
347 typedef HeapTupleStruct * HeapTuple;
348
349 void ExecForceStoreMinimalTuple(MinimalTuple mtup, TupleTableSlot *slot, bool
    shouldFree);
350 void ExecForceStoreHeapTuple(HeapTuple mtup, TupleTableSlot *slot, bool shouldFree);

```

```

350 void useTupleMin(MinimalTuple tup);
351 void useTupleHeap(HeapTuple tup);
352
353 void exec_force_minimal(void){
354     MinimalTuple tuple = malloc(sizeof(MinimalTuple));
355     TupleTableSlot *slot = malloc(sizeof(TupleTableSlot));
356     ExecForceStoreMinimalTuple(tuple, slot, false);
357     useTupleMin(tuple);
358     ExecForceStoreMinimalTuple(tuple, slot, true); // expected-note{{Freeing function:
        ExecForceStoreMinimalTuple (tuple)}}
359     useTupleMin(tuple); // expected-warning{{Attempt to use potentially released memory:
        tuple}}
360
361 }
362
363 void exec_force_minimal_argument(bool shouldFree){
364     MinimalTuple tuple = malloc(sizeof(MinimalTuple));
365     TupleTableSlot *slot = malloc(sizeof(TupleTableSlot));
366     ExecForceStoreMinimalTuple(tuple, slot, shouldFree); // expected-note{{Freeing
        function: ExecForceStoreMinimalTuple (tuple)}}
367     useTupleMin(tuple); // expected-warning{{Attempt to use potentially released memory:
        tuple}}
368
369 }
370
371 void exec_force_heap(void){
372     HeapTuple tuple = malloc(sizeof(HeapTuple));
373     TupleTableSlot *slot = malloc(sizeof(TupleTableSlot));
374     ExecForceStoreHeapTuple(tuple, slot, false);
375     useTupleHeap(tuple);
376     ExecForceStoreHeapTuple(tuple, slot, true); // expected-note{{Freeing function:
        ExecForceStoreHeapTuple (tuple)}}
377     useTupleHeap(tuple); // expected-warning{{Attempt to use potentially released
        memory: tuple}}
378
379 }
380
381 void exec_force_heap_argument(bool shouldFree){
382     HeapTuple tuple = malloc(sizeof(HeapTuple));
383     TupleTableSlot *slot = malloc(sizeof(TupleTableSlot));
384     ExecForceStoreHeapTuple(tuple, slot, shouldFree); // expected-note{{Freeing
        function: ExecForceStoreHeapTuple (tuple)}}
385     useTupleHeap(tuple); // expected-warning{{Attempt to use potentially released
        memory: tuple}}
386
387 }
388
389 typedef struct {}List;
390
391 void ExecResetTupleTable(List *tupleTable,      bool shouldFree);
392 void useList(List *list);
393
394 void exec_reset_tt(void){
395     List *list = malloc(sizeof(List));
396     ExecResetTupleTable(list, false);
397     useList(list);
398     ExecResetTupleTable(list, true); // expected-note{{Freeing function:
        ExecResetTupleTable (list)}}
399     useList(list); // expected-warning{{Attempt to use released memory: list}}
400 }
401
402 void exec_reset_tt_argument(bool shouldFree){
403     List *list = malloc(sizeof(List));
404     ExecResetTupleTable(list, shouldFree); // expected-note{{Freeing function:
        ExecResetTupleTable (list)}}

```

```

405     useList(list); // expected-warning{{Attempt to use potentially released memory:
        list}}
406 }
407
408 typedef unsigned int bits32;
409 typedef struct {
410     bits32 flags;
411 }JsonLexContext;
412 void freeJsonLexContext(JsonLexContext *lex);
413 void useJsonLexContext(JsonLexContext *lex);
414
415 void free_json_lc(void){
416     JsonLexContext *lexContext = malloc(sizeof(JsonLexContext));
417     lexContext->flags = 0;
418     freeJsonLexContext(lexContext);
419     useJsonLexContext(lexContext);
420     lexContext->flags |= (1 << 0);
421     lexContext->flags = (1 << 0);
422     freeJsonLexContext(lexContext); // expected-note{{Freeing function:
        freeJsonLexContext (lexContext)}}
423     useJsonLexContext(lexContext); // expected-warning{{Attempt to use released memory:
        lexContext}}
424 }
425
426 void free_json_lc_argument(bits32 flags){
427     JsonLexContext *lexContext = malloc(sizeof(JsonLexContext));
428     lexContext->flags = flags;
429     freeJsonLexContext(lexContext); // expected-note{{Freeing function:
        freeJsonLexContext (lexContext)}}
430     useJsonLexContext(lexContext); // expected-warning{{Attempt to use potentially
        released memory: lexContext}}
431 }
432
433 typedef struct {}PGresult;
434 typedef struct {}PGconn;
435
436 void pgfdw_report_error(int elevel, PGresult *res, PGconn *conn, bool clear, const
    char *sql);
437
438 void usePGresult(PGresult *res);
439
440 void free_pgfdw_report_error(void){
441     PGresult *res = malloc(sizeof(PGresult));
442     PGconn *conn = malloc(sizeof(PGconn));
443     pgfdw_report_error(0, res, conn, false, "PG is great");
444     usePGresult(res);
445     pgfdw_report_error(0, res, conn, true, "PG is great"); // expected-note{{Freeing
        function: pgfdw_report_error (res)}}
446     usePGresult(res); // expected-warning{{Attempt to use released memory: res}}
447 }
448
449 void free_pgfdw_report_error_argument(bool clear){
450     PGresult *res = malloc(sizeof(PGresult));
451     PGconn *conn = malloc(sizeof(PGconn));
452     pgfdw_report_error(0, res, conn, clear, "PG is great"); // expected-note{{Freeing
        function: pgfdw_report_error (res)}}
453     usePGresult(res); // expected-warning{{Attempt to use potentially released memory:
        res}}
454 }
455
456 static const PGresult OOM_result = {};
457
458 void PQclear(PGresult *res);
459
460 void free_PQclear(void){
461     PGresult *res = malloc(sizeof(PGresult));

```

```

462     const PGresult *resOOM = &OOM_result;
463     PQclear(res); // expected-note{{Freeing function: PQclear (res)}}
464     usePGresult(res); // expected-warning{{Attempt to use released memory: res}}
465     PQclear((PGresult *) resOOM);
466     usePGresult((PGresult *) resOOM);
467 }
468 }
469
470 void free_argument(PGresult * res){
471     PQclear(res); // expected-note{{Freeing function: PQclear (res)}}
472     usePGresult(res); // expected-warning{{Attempt to use potentially released memory:
        res}}
473 }
474
475 typedef struct{} BrinTuple;
476 typedef unsigned int Size;
477
478 BrinTuple *brin_copy_tuple(BrinTuple *tuple, Size len, BrinTuple *dest, Size *destsz);
479
480 void useBrinTuple(BrinTuple * tuple);
481
482 void free_brin_copy_tuple(void){
483     BrinTuple *tuple = malloc(sizeof(BrinTuple));
484     BrinTuple *otherTuple = malloc(sizeof(BrinTuple));
485     Size destsz = 5;
486     Size *destszPtr = &destsz;
487     Size len = 3;
488     brin_copy_tuple(tuple, len, otherTuple, destszPtr);
489     useBrinTuple(otherTuple);
490     destsz = 0;
491     brin_copy_tuple(tuple, len, otherTuple, destszPtr); // expected-note{{The return
        value should probably be reassigned (otherTuple)}}
492     useBrinTuple(otherTuple); // expected-warning{{Attempt to use potentially released
        memory: otherTuple}}
493     destsz = 0;
494     otherTuple = malloc(sizeof(BrinTuple));
495     destsz = 2;
496     brin_copy_tuple(tuple, len, otherTuple, destszPtr); // expected-note{{The return
        value should probably be reassigned (otherTuple)}}
497     useBrinTuple(otherTuple); // expected-warning{{Attempt to use potentially released
        memory: otherTuple}}
498 }
499
500 enum COMPAT_MODE{MODE};
501
502 bool ecpg_check_PQresult (PGresult *results, int lineno, PGconn *      connection,
        enum COMPAT_MODE compat);
503
504 void free_ecpg_check(void){
505     PGresult *res = malloc(sizeof(PGresult));
506     PGconn *conn = malloc(sizeof(PGconn));
507     ecpg_check_PQresult(res, 0, conn, MODE); // expected-note{{The return value should
        probably be checked}}
508     usePGresult(res); // expected-warning{{Attempt to use potentially released memory:
        res}}
509 }

```

BIBLIOGRAPHY

- [ACB16] Marcelo Arroyo, Francisco Chiotta, and Francisco Bavera. “An user configurable clang static analyzer taint checker.” In: *2016 35th International Conference of the Chilean Computer Science Society (SCCC)*. 2016, pp. 1–12. DOI: 10.1109/SCCC.2016.7835996.
- [AGo4] Andrew W. Appel and Maia Ginsburg. *Modern Compiler Implementation in C*. Cambridge: University Press, 2004. ISBN: 0 521 58390 X.
- [Aho+07] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, & Tools*. Boston: Pearson, 2007. ISBN: 0-321-48681.
- [Albo3] Laurenz Albe. *Memory context: private memory management in PostgreSQL*. 2003. URL: <https://www.cybertec-postgresql.com/en/memory-context-for-postgresql-memory-management/> (visited on 07/22/2025).
- [Apa25a] The Apache Software Foundation. *AGE*. 2025. URL: <https://github.com/apache/age> (visited on 08/25/2025).
- [Apa25b] The Apache Software Foundation. *age_match_two_vle_edges*. 2025. URL: https://github.com/apache/age/blob/4cc02ad2525172b7417ee4d9afa5ee8015b467cb/src/backend/utils/adt/age_vle.c#L1930C7-L1930C30 (visited on 08/25/2025).
- [BPT17] Jasmin Christian Blanchette, Andrei Popescu, and Dmitriy Traytel. “Soundness and Completeness Proofs by Coinductive Methods.” In: *J. Autom. Reason.* 58.1 (Jan. 2017), 149–179. ISSN: 0168-7433. DOI: 10.1007/s10817-016-9391-3. URL: <https://doi.org/10.1007/s10817-016-9391-3>.
- [Bar+21] Md Abdullah Shahneous Bari, Ujjwal Arora, Varun Hegde, Tony Curtis, and Barbara Chapman. “OpenSHMEM Checker - A Clang Based Static Checker for OpenSHMEM.” In: *2021 20th International Symposium on Parallel and Distributed Computing (ISPD)*. 2021, pp. 41–48. DOI: 10.1109/ISPD52870.2021.9521645.
- [Bru+09] Julien Brunel, Damien Doligez, René Rydhof Hansen, Julia L. Lawall, and Gilles Muller. “A foundation for flow-based program matching: using temporal logic and model checking.” In: *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL ’09. Savannah, GA, USA: Association for Computing Machinery, 2009, 114–126. ISBN: 9781605583792. DOI: 10.1145/1480881.1480897. URL: <https://doi.org/10.1145/1480881.1480897>.

- [CEA25] CEA List and Inria. *Frama-C*. 2025. URL: <https://frama-c.com/> (visited on 08/05/2025).
- [Cla25a] The Clang Team. *CheckerContext::addSink*. 2025. URL: https://clang.llvm.org/doxygen/classclang_1_1ento_1_1CheckerContext.html#a320b78287a2690096751244235277d03 (visited on 08/05/2025).
- [Cla25b] The Clang Team. *Checker.h Source File*. 2025. URL: https://clang.llvm.org/doxygen/Checker_8h_source.html (visited on 08/05/2025).
- [Cla25c] The Clang Team. *Clang 21.0.git documentation*. 2025. URL: <https://clang.llvm.org/docs/index.html> (visited on 06/10/2025).
- [Cla25d] The Clang Team. *Clang Attribute Reference - Ownership*. 2025. URL: <https://clang.llvm.org/docs/AttributeReference.html#ownership-holds-ownership-returns-ownership-takes-clang-static-analyzer> (visited on 08/05/2025).
- [Cla25e] The Clang Team. *Clang Doxygen Documentation: Classes*. 2025. URL: <https://clang.llvm.org/doxygen/classes.html> (visited on 08/05/2025).
- [Cla25f] The Clang Team. *Clang Static Analyzer Check Namespace*. 2025. URL: https://clang.llvm.org/doxygen/namespaceclang_1_1ento_1_1check.html (visited on 08/05/2025).
- [Cla25g] The Clang Team. *Clang Static Analyzer*. 2025. URL: <https://clang-analyzer.llvm.org/> (visited on 06/12/2025).
- [Cla25h] The Clang Team. *Clang-Tidy*. 2025. URL: <https://clang.llvm.org/extra/clang-tidy/> (visited on 06/12/2025).
- [Cla25i] The Clang Team. *Clang: a C language family frontend for LLVM*. 2025. URL: <https://clang.llvm.org/> (visited on 08/05/2025).
- [Cla25j] The Clang Team. *Cross Translation Unit (CTU) Analysis*. 2025. URL: <https://clang.llvm.org/docs/analyzer/user-docs/CrossTranslationUnit.html> (visited on 08/01/2025).
- [Cla25k] The Clang Team. *MallocChecker Class Template Declaration*. 2025. URL: <https://github.com/llvm/llvm-project/blob/2cb32e29408a6c598072ea0f066a246957be69f9/clang/lib/StaticAnalyzer/Checkers/MallocChecker.cpp#L343> (visited on 08/05/2025).
- [Cla25l] The Clang Team. *MallocChecker.cpp Source File*. 2025. URL: https://clang.llvm.org/doxygen/MallocChecker_8cpp_source.html (visited on 08/05/2025).
- [Cla25m] The Clang Team. *What is clangd?* 2025. URL: <https://clangd.llvm.org/> (visited on 08/01/2025).
- [Coc25] Coccinelle Development Team. *Coccinelle*. 2025. URL: <https://github.com/coccinelle/coccinelle> (visited on 08/05/2025).

- [DE25] Edvin Dyremark and Tobias Elfstrand. “False Positives, Real Problems : Evaluating Static Analysis Tools.” MA thesis. Linköping University, Department of Computer and Information Science, 2025.
- [DKL15] Alexander Droste, Michael Kuhn, and Thomas Ludwig. “MPI-checker: static analysis for MPI.” In: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. LLVM ’15. Austin, Texas: Association for Computing Machinery, 2015. ISBN: 9781450340052. DOI: 10.1145/2833157.2833159. URL: <https://doi.org/10.1145/2833157.2833159>.
- [DR24] Alexandru Dura and Christoph Reichenbach. “Clog: A Declarative Language for C Static Code Checkers.” In: *Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction*. CC 2024. Edinburgh, United Kingdom: Association for Computing Machinery, 2024, 186–197. ISBN: 9798400705076. DOI: 10.1145/3640537.3641579. URL: <https://doi.org/10.1145/3640537.3641579>.
- [Dec25] Decathlon. *dblinter*. 2025. URL: <https://github.com/Decathlon/dblinter> (visited on 08/05/2025).
- [FP24] Endre Fülöp and Norbert Pataki. “Temporal Logic-driven Symbolic Execution with the Clang Static Analyzer.” In: *2024 7th International Conference on Software and System Engineering (ICoSSE)*. 2024, pp. 78–82. DOI: 10.1109/ICoSSE62619.2024.00021.
- [Fac25] Steven Fackler. *rust-postgres*. 2025. URL: <https://github.com/sfackler/rust-postgres> (visited on 08/05/2025).
- [For25] Chris Forbes. *Sparse*. 2025. URL: <https://github.com/chrisforbes/sparse> (visited on 08/05/2025).
- [GNU25a] GNU C Library. *strcat.c*. 2025. URL: <https://github.com/bminor/glibc/blob/39183b953c68a489cc0b9aefb8974711c834fb38/string/strcat.c#L28> (visited on 08/05/2025).
- [GNU25b] GNU Project. *GCC Static Analyzer Internals*. 2025. URL: <https://gcc.gnu.org/onlinedocs/gccint/Analyzer-Internals.html> (visited on 08/05/2025).
- [Goo25] Google LLC. *AddressSanitizer*. 2025. URL: <https://github.com/google/sanitizers/wiki/addresssanitizer> (visited on 08/01/2025).
- [HR04] Michael R. A. Huth and Mark Ryan. *Logic in computer science: modelling and reasoning about systems*. USA: Cambridge University Press, 2004. ISBN: 0521656028.
- [Hol25] Holistic Development. *Holistic SQL Analysis*. 2025. URL: <https://holistic.dev/playground> (visited on 08/05/2025).

- [ISO07] Joint Technical Committee. *ISO/IEC 9899:TC3*. 2007. URL: <http://www.open-std.org/jtc1/sc22/WG14/www/docs/n1256.pdf> (visited on 06/12/2025).
- [ISO15] *ISO/IEC 2382:2015(en)*. 2015. URL: <https://www.iso.org/obp/ui/#iso:std:iso-iec:2382:ed-1:v2:en> (visited on 06/12/2025).
- [ISO24] The Information Technology Vocabulary Maintenance Team. *ISO/IEC 9899:2024 (en) — N3220 working draft*. 2024. URL: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf> (visited on 08/31/2025).
- [Kin76] James C. King. “Symbolic execution and program testing.” In: *Commun. ACM* 19.7 (July 1976), 385–394. ISSN: 0001-0782. DOI: 10.1145/360248.360252. URL: <https://doi.org/10.1145/360248.360252>.
- [LLV25a] LLVM Project. *The LLVM Compiler Infrastructure*. 2025. URL: <http://llvm.org/> (visited on 06/12/2025).
- [LLV25b] LLVM Project. *lit - LLVM Integrated Tester*. 2025. URL: <https://llvm.org/docs/CommandGuide/lit.html> (visited on 06/12/2025).
- [LM18] Julia Lawall and Gilles Muller. “Coccinelle: 10 years of automated evolution in the Linux kernel.” In: *Proceedings of the 2018 USENIX Conference on Usenix Annual Technical Conference*. USENIX ATC ’18. Boston, MA, USA: USENIX Association, 2018, 601–613. ISBN: 9781931971447.
- [LM22] Julia Lawall and Gilles Muller. “Automating Program Transformation with Coccinelle.” In: *NASA Formal Methods: 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24–27, 2022, Proceedings*. Pasadena, CA, USA: Springer-Verlag, 2022, 71–87. ISBN: 978-3-031-06772-3. DOI: 10.1007/978-3-031-06773-0_4. URL: https://doi.org/10.1007/978-3-031-06773-0_4.
- [Lun25] Lund University. *Clog*. 2025. URL: <https://github.com/lu-cs-sde/clog> (visited on 08/05/2025).
- [MS25] Anders Møller and Michael I. Schwartzbach. *Static Program Analysis*. Aarhus University, Denmark: Department of Computer Science, 2025. URL: <https://cs.au.dk/~amoeller/spa/spa.pdf> (visited on 08/31/2025).
- [Met25] Meta Platforms, Inc. *Infer Static Analyzer*. 2025. URL: <https://github.com/facebook/infer> (visited on 08/05/2025).
- [Mey19] Bertrand Meyer. *Soundness and Completeness: Defined With Precision*. 2019. URL: <https://cacm.acm.org/blogcacm/soundness-and-completeness-defined-with-precision/> (visited on 06/10/2025).
- [Moo25] Moose Technology. *Moose*. 2025. URL: <https://github.com/mooseetechnology/Moose> (visited on 08/05/2025).

- [Mul+06] Gilles Muller, Yoann Padioleau, Julia L. Lawall, and René Rydhof Hansen. “Semantic patches considered helpful.” In: *SIGOPS Oper. Syst. Rev.* 40.3 (July 2006), 90–92. ISSN: 0163-5980. DOI: 10.1145/1151374.1151392. URL: <https://doi.org/10.1145/1151374.1151392>.
- [Mur24] Ivan Murashko. *Clang Compiler Frontend: Get to grips with the internals of a C/C++ compiler frontend and create your own tools*. Birmingham: Packt, 2024. ISBN: 978-1-83763-098-1.
- [Nay24] John Naylor. *Commit ccc8194*. 2024. URL: <https://github.com/postgres/postgres/commit/ccc8194e4275d0be557a9b94126a12b389520593> (visited on 08/05/2025).
- [OP23] Paul C. van Oorschot and Frank Piessens. “Memory Errors and Memory Safety: C as a Case Study.” In: *IEEE Security and Privacy* 21.2 (Mar. 2023), 70–76. ISSN: 1540-7993. DOI: 10.1109/MSEC.2023.3236542. URL: <https://doi.org/10.1109/MSEC.2023.3236542>.
- [Ole+10] Mads Olesen, Rene Hansen, Julia Lawall, and Nicolas Palix. “Clang and Coccinelle: Synergising program analysis tools for CERT C Secure Coding Standard certification.” In: *Electronic Communications of the EASST* 33 (2010). DOI: 10.14279/tuj.eceasst.33.455. URL: <https://eceasst.org/index.php/eceasst/article/view/1725>.
- [PLMo7] Yoann Padioleau, Julia L. Lawall, and Gilles Muller. “SmPL: A Domain-Specific Language for Specifying Collateral Evolutions in Linux Device Drivers.” In: *Electronic Notes in Theoretical Computer Science* 166 (2007). Proceedings of the ERCIM Working Group on Software Evolution (2006), pp. 47–62. ISSN: 1571-0661. DOI: <https://doi.org/10.1016/j.entcs.2006.07.022>. URL: <https://www.sciencedirect.com/science/article/pii/S1571066106005287>.
- [Pos25a] PostgreSQL Global Development Group. *Bitmapset Implementation Warning*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/backend/nodes/bitmapset.c#L19> (visited on 08/05/2025).
- [Pos25b] PostgreSQL Global Development Group. *Datum*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/include/postgres.h#L69> (visited on 08/05/2025).
- [Pos25c] PostgreSQL Global Development Group. *Datum*. 2025. URL: <https://github.com/postgres/postgres/blob/e0f373ee42a40a41bdfc025a1641d351580991c4/src/include/postgres.h#L60C4-L61C38> (visited on 08/05/2025).

- [Pos25d] PostgreSQL Global Development Group. *DatumGetInt64*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/include/postgres.h#L390> (visited on 08/05/2025).
- [Pos25e] PostgreSQL Global Development Group. *DecrTupleDescRefCount*. 2025. URL: <https://github.com/postgres/postgres/blob/afa5c365ec5ad978878e5d26c536d8f865abf1ae/src/backend/access/common/tupdesc.c#L577> (visited on 08/05/2025).
- [Pos25f] PostgreSQL Global Development Group. *ExecForceStoreMinimalTuple*. 2025. URL: <https://github.com/postgres/postgres/blob/1dbe6f76677c26096518998fdc72dab771a98913/src/backend/executor/execTuples.c#L1701> (visited on 08/05/2025).
- [Pos25g] PostgreSQL Global Development Group. *NodeTag*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/include/nodes/nodes.h#L26> (visited on 08/05/2025).
- [Pos25h] PostgreSQL Global Development Group. *PG_GETARG_INT64*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/include/fmgr.h#L283> (visited on 08/05/2025).
- [Pos25i] PostgreSQL Global Development Group. *PostgreSQL 17.4 Documentation*. 2025. URL: <https://www.postgresql.org/files/documentation/pdf/17/postgresql-17-A4.pdf> (visited on 06/10/2025).
- [Pos25j] PostgreSQL Global Development Group. *PostgreSQL Error Reporting and Logging - Severity Levels*. 2025. URL: <https://www.postgresql.org/docs/current/runtime-config-logging.html#RUNTIME-CONFIG-SEVERITY-LEVELS> (visited on 08/05/2025).
- [Pos25k] PostgreSQL Global Development Group. *SPI_execute*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/backend/executor/spi.c#L595> (visited on 08/05/2025).
- [Pos25l] PostgreSQL Global Development Group. *TransactionId*. 2025. URL: <https://github.com/postgres/postgres/blob/2fd3e2fa5c908d05ac4d8b6c2c80fd50c506f32e/src/include/c.h#L623> (visited on 08/05/2025).
- [Pos25m] PostgreSQL Global Development Group. *TransactionIdPrecedes*. 2025. URL: <https://github.com/postgres/postgres/blob/2fd3e2fa5c908d05ac4d8b6c2c80fd50c506f32e/src/backend/access/transam/transam.c#L276> (visited on 08/05/2025).
- [Pos25n] PostgreSQL Global Development Group. *add_path*. 2025. URL: <https://github.com/postgres/postgres/blob/1dbe6f76677c26096518998fdc72dab771a98913/src/backend/optimizer/util/pathnode.c#L461> (visited on 08/05/2025).

- [Pos25o] PostgreSQL Global Development Group. *bitmapset.h*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/include/nodes/bitmapset.h> (visited on 08/05/2025).
- [Pos25p] PostgreSQL Global Development Group. *bms_add_member*. 2025. URL: <https://github.com/postgres/postgres/blob/ef5b87b970dc28adeeb88191fbf66c9d6298b112/src/backend/nodes/bitmapset.c#L815> (visited on 08/27/2025).
- [Pos25q] PostgreSQL Global Development Group. *bms_int_members*. 2025. URL: <https://github.com/postgres/postgres/blob/196063d6761d2c8d6f78cc03afad08efc95a0708/src/backend/nodes/bitmapset.c#L1109> (visited on 08/05/2025).
- [Pos25r] PostgreSQL Global Development Group. *_bt_insertonpg*. 2025. URL: <https://github.com/postgres/postgres/blob/6551a05d9cf8ea75c0db232b661dadd16e595854/src/backend/access/nbtree/nbtinsert.c#L1105> (visited on 08/05/2025).
- [Pos25s] PostgreSQL Global Development Group. *castNode*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/include/nodes/nodes.h#L180> (visited on 08/05/2025).
- [Pos25t] PostgreSQL Global Development Group. *ecpg_check_PQresult*. 2025. URL: <https://github.com/postgres/postgres/blob/1dbe6f76677c26096518998fdc72dab771a98913/src/interfaces/ecpg/ecpglib/error.c#L281> (visited on 08/05/2025).
- [Pos25u] PostgreSQL Global Development Group. *ecpg_realloc*. 2025. URL: <https://github.com/postgres/postgres/blob/ee924698d566223e927cf9d505c1ccdacd7061c8/src/interfaces/ecpg/ecpglib/memory.c#L33> (visited on 08/05/2025).
- [Pos25v] PostgreSQL Global Development Group. *exit_nicely*. 2025. URL: https://github.com/postgres/postgres/blob/3151c264d460c0be09131ce90529073631d70ae8/src/test/modules/libpq_pipeline/libpq_pipeline.c#L59 (visited on 08/05/2025).
- [Pos25w] PostgreSQL Global Development Group. *expand_partitioned_rteentry*. 2025. URL: <https://github.com/postgres/postgres/blob/6551a05d9cf8ea75c0db232b661dadd16e595854/src/backend/optimizer/util/inherit.c#L318> (visited on 08/05/2025).
- [Pos25x] PostgreSQL Global Development Group. *get_index_paths*. 2025. URL: <https://github.com/postgres/postgres/blob/6551a05d9cf8ea75c0db232b661dadd16e595854/src/backend/optimizer/path/indxpath.c#L715> (visited on 08/05/2025).
- [Pos25y] PostgreSQL Global Development Group. *list_delete_nth_cell*. 2025. URL: <https://github.com/postgres/postgres/blob/1dbe6f76677c26096518998fdc72dab771a98913/src/backend/nodes/list.c#L767> (visited on 08/05/2025).

- [Pos25z] PostgreSQL Global Development Group. *list_length*. 2025. URL: https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/include/nodes/pg_list.h#L152 (visited on 08/05/2025).
- [Pos25aa] PostgreSQL Global Development Group. *parallel_vacuum_reset_dead_items*. 2025. URL: <https://github.com/postgres/postgres/blob/6551a05d9cf8ea75c0db232b661dadd16e595854/src/backend/commands/vacuumparallel.c#L475> (visited on 08/05/2025).
- [Pos25ab] PostgreSQL Global Development Group. *pg_fatal*. 2025. URL: https://github.com/postgres/postgres/blob/4300d8b6a79d61abb5ca9f901df7bde7a49322b6/src/bin/pg_upgrade/util.c#L270 (visited on 08/05/2025).
- [Pos25ac] PostgreSQL Global Development Group. *pg_list.h*. 2025. URL: https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/include/nodes/pg_list.h#L13 (visited on 08/05/2025).
- [Pos25ad] PostgreSQL Global Development Group. *spi.c*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/backend/executor/spi.c> (visited on 08/05/2025).
- [Pos25ae] PostgreSQL Global Development Group. *unconstify*. 2025. URL: <https://github.com/postgres/postgres/blob/0ff95e0a5be1372bfba9db284ea17c8e0e5da3a0/src/include/c.h#L1219> (visited on 08/05/2025).
- [Rei25a] Bernd Reiß. *Coccinelle Scripts and Results*. 2025. URL: <https://github.com/berndreiss/pg-dev-container/tree/main/coccinelle> (visited on 08/05/2025).
- [Rei25b] Bernd Reiß. *Commit da9f9f7*. 2025. URL: <https://github.com/postgres/postgres/commit/da9f9f75e5ce27a45878ffa262156d18f0046188> (visited on 08/29/2025).
- [Rei25c] Bernd Reiß. *PostgreSQL Global Development Group*. 2025. URL: <https://github.com/berndreiss/pg-dev-container/tree/main> (visited on 08/05/2025).
- [Rei25d] Bernd Reiß. *PostgresChecker.cpp*. 2025. URL: <https://github.com/berndreiss/llvm-project/blob/7645be0f346655d301863073b4e24258b4e0c4bb/clang/lib/StaticAnalyzer/Checkers/PostgresChecker.cpp> (visited on 08/05/2025).
- [Rei25e] Bernd Reiß. *Use-after-free in expand_partitioned_rentry*. 2025. URL: <https://www.postgresql.org/message-id/6b88f27a-c45c-4826-8e37-d61a04d90182@gmx.at> (visited on 08/29/2025).
- [Rei25f] Bernd Reiß. *analysis_contrib_results.out*. 2025. URL: https://github.com/berndreiss/pg-dev-container/blob/d81e217fcfcbd3af2792cc9d4b4f09921d3cd027/clang-tests/analysis_contrib_results.out (visited on 08/05/2025).

- [Rei25g] Bernd Reiß. *analysis_src_results.out*. 2025. URL: https://github.com/berndreiss/pg-dev-container/blob/d81e217fcfcdbd3af2792cc9d4b4f09921d3cd027/clang-tests/analysis_src_results.out (visited on 08/05/2025).
- [Rei25h] Bernd Reiß. *pg_ladybug Fork with Install Script*. 2025. URL: https://github.com/berndreiss/pg_ladybug (visited on 08/05/2025).
- [Ric53] Henry G. Rice. “Classes of Recursively Enumerable Sets and Their Decision Problems.” In: *Transactions of the American Mathematical Society* 74.2 (1953), pp. 358–366. ISSN: 00029947, 10886850. URL: <http://www.jstor.org/stable/1990888> (visited on 06/27/2025).
- [Rus25a] Rust Foundation. *Rust Programming Language*. 2025. URL: <https://www.rust-lang.org/> (visited on 08/05/2025).
- [Rus25b] Rust Foundation. *Understanding Ownership*. 2025. URL: <https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html> (visited on 08/05/2025).
- [SMK21] Stefanie Scherzinger, Wolfgang Mauerer, and Haridimos Kondylakis. “DeBinelle: Semantic Patches for Coupled Database-Application Evolution.” In: *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 2021, pp. 2697–2700. DOI: 10.1109/ICDE51399.2021.00307.
- [STM25a] Andreas Seltenreich, Bo Tang, and Sjoerd Mullender. *sqlsmith: Score List*. 2025. URL: <https://github.com/anse1/sqlsmith/wiki#score-list> (visited on 08/08/2025).
- [STM25b] Andreas Seltenreich, Bo Tang, and Sjoerd Mullender. *sqlsmith*. 2025. URL: <https://github.com/anse1/sqlsmith> (visited on 08/08/2025).
- [Sem25] Semgrep Inc. *Semgrep*. 2025. URL: <https://github.com/semgrep/semgrep> (visited on 08/05/2025).
- [Sil25] Silicon Graphics International Corp. *Welcome to OpenSHMEM*. 2025. URL: <http://openshmem.org/> (visited on 08/08/2025).
- [Sof25] Software Systems Lab. *CPAchecker*. 2025. URL: <https://github.com/sosy-lab/cpachecker> (visited on 08/05/2025).
- [Sta25] StackScale. *Linux Kernel Surpasses 40 Million Lines of Code*. 2025. URL: <https://www.stackscale.com/blog/linux-kernel-surpasses-40-million-lines-code/> (visited on 08/05/2025).
- [Ste25] Pavel Stehule. *plpgsql_check*. 2025. URL: https://github.com/okbob/plpgsql_check (visited on 08/05/2025).

- [Stu+07] Henrik Stuart, René Rydhof Hansen, Julia L. Lawall, Jesper Andersen, Yoann Padioleau, and Gilles Muller. “Towards easing the diagnosis of bugs in OS code.” In: *Proceedings of the 4th Workshop on Programming Languages and Operating Systems*. PLOS ’07. Stevenson, Washington: Association for Computing Machinery, 2007. ISBN: 9781595939227. DOI: 10.1145/1376789.1376792. URL: <https://doi.org/10.1145/1376789.1376792>.
- [Stu08] Henrik Stuart. *Hunting bugs with Coccinelle*. Master’s thesis. Copenhagen, 2008. URL: https://coccinelle.gitlabpages.inria.fr/website/papers/stuart_thesis.pdf (visited on 08/31/2025).
- [Tim25a] Timescale Development Team. *PostgresCheck.cpp*. 2025. URL: https://github.com/timescale/pg_ladybug/blob/3a50bfd767a3e0dad61b0bd7a6bd9542d2a4ae6e/src/postgres/PostgresCheck.cpp (visited on 08/05/2025).
- [Tim25b] Timescale Development Team. *TimescaleDB Coccinelle Scripts*. 2025. URL: <https://github.com/timescale/timescaledb/tree/main/coccinelle> (visited on 08/05/2025).
- [Tim25c] Timescale Development Team. *bms_result.cocci*. 2025. URL: https://github.com/timescale/timescaledb/blob/main/coccinelle/bms_result.cocci (visited on 08/05/2025).
- [Tim25d] Timescale Development Team. *pg_ladybug*. 2025. URL: https://github.com/timescale/pg_ladybug/tree/main (visited on 08/05/2025).
- [Tim25e] Timescale Development Team. *pg_ladybug*. 2025. URL: https://github.com/timescale/pg_ladybug (visited on 08/05/2025).
- [Tim25f] Timescale Inc. *Timescale*. 2025. URL: <https://www.timescale.com/> (visited on 08/05/2025).
- [Tim25g] Timescale Inc. *Top 8 PostgreSQL Extensions*. 2025. URL: <https://www.timescale.com/blog/top-8-postgresql-extensions> (visited on 08/05/2025).
- [Uni25] University of Virginia. *Splint*. 2025. URL: <https://splint.org/> (visited on 08/05/2025).
- [Val25] Valgrind Developers. *Valgrind*. 2025. URL: <https://valgrind.org/> (visited on 08/01/2025).
- [Van25] Ivan Vanek. *pg-index-health*. 2025. URL: <https://github.com/mfvaneek/pg-index-health> (visited on 08/05/2025).